THOMAS WALKER

A SPLINE THEORY OF DEEP LEARNING

RICE UNIVERSITY

Contents

I The Theoretical Foundations 9

1 Deep Learning 11
- 1.1 Introduction 11
- 1.2 Deep Networks 11
- 1.3 Paradigm 12
- 1.4 Operators and Layers 12
  - Fully Connected Operator 12
  - Convolutional Operator 13
  - Activation Operators 13
  - Pooling Operators 14
  - Normalisation Operators 15
  - Skip Connection Operator 16
  - Recurrent Connection Operator 17
  - Output Operator 17
  - Layer 17
- 1.5 References 17

2 Spline Theory 18
- 2.1 Introduction 18
- 2.2 Affine Spline Functions 18
- 2.3 Affine Spline Operators 19
- 2.4 Max-Affine Spline Functions 20
- 2.5 Max-Affine Spline Operators 22
- 2.6 References 23

3 Deep Networks and Spline Operators 24
3.1 Introduction 24
3.2 Deep Network Operators as Spline Operators 24
3.3 Deep Networks as Spline Operators 28
3.4 Constructions 29
Convolutional Neural Network 29
ResNet 30
Recurrent Neural Networks 30
Transformers 31
3.5 References 31

4 Properties of Continuous Piecewise Linear Deep Network 32
4.1 Introduction 32
4.2 Convexity 32
4.3 Lipschitz 34
4.4 Universality 36
4.5 Template Matchers 37
Filter Template Matching 37
Filter Template versus Bias 38
Collinear Templates 38
Orthogonal Templates 40
4.6 Input Space Partitioning 41
Max-Affine Spline Partition 41
Deep Network Partition 42
Functional Decomposition 43
4.7 Fourier Analysis 45
4.8 References 46

5 Locally Non-Linear Deep Networks 47
5.1 Introduction 47
5.2 Vector Quantization 47

5.3 Soft Max-Affine Spline Operators 48
5.4 Vector Quantization Inference 49
Hard-VQ Inference 49
Soft-VQ Inference 50
β-VQ Inference 52
5.5 Constructions 52
5.6 References 53

6 Partition Geometry 54
6.1 Introduction 54
6.2 Power Diagrams 54
6.3 Max-Affine Splines 54
6.4 Max-Affine Spline Operator 57
6.5 Deep Network 57
6.6 Decision Boundary 59
6.7 References 59

7 Initialisation and Regularisation 60
7.1 Introduction 60
7.2 Batch Normalisation 60
Single Layer Analysis 60
Multilayer Analysis 62
Initialization Scheme 65
Jitter 65
7.3 Layer Normalisation 66
7.4 References 66

8 Optimization 67
8.1 Introduction 67

9 Data Distribution 68
9.1 Introduction 68

II The Practical Implications 69

10 Visualisation 71
10.1 Introduction 71
10.2 Exact Computation of Partition Boundaries 71
10.3 SplineCam 72
10.4 References 74

11 Complexity Measures 75
11.1 Introduction 75
11.2 Local Complexity 75
11.3 Absolute Deviation 76
Adaptive Linear Region Discovery 76
Quantifying Nonlinearity 77
11.4 Hessian Based Rugosity Measure 79
Smooth Activation Deep Networks 79
Continuous Piecewise Affine Deep Networks 80
Connection to Data Augmentation 80
11.5 References 82

12 Constraining Optimization 83
12.1 Introduction 83
12.2 POLICE 83
12.3 Instance Robustness 84
12.4 References 85

13 Formal Verification 86
13.1 Introduction 86
13.2 Predictions 86
13.3 References 88

14 Pruning 89
14.1 Introduction 89
14.2 Characterisation of Techniques 89
14.3 Impact on Partition 90
14.4 References 90

15 Fine-tuning 91
15.1 Introduction 91
15.2 Smoothing Out Region Assignment 91
15.3 Curvature Tuning 92
References 92

16 Generative Modelling 93
16.1 Introduction 93
16.2 Continuous Piecewise Affine Autoencoders 93
Reconstruction Guarantees 94
Autoencoder Jacobian 96
16.3 Continuous Piecewise Affine Variational Autoencoders 97
Variational Bayes 97
Variational Autoencoders 98
Conditional, Posterior and Marginal Distributions 98
Gaussian Integration on Partitions 99
Expectation-Maximization Learning 100
16.4 Density of Generated Manifold 101
16.5 Polarity Sampling 104
Practical Implementation 104
16.6 ScaLES 105
Latent Space Optimisation 105
Scalable Latent Exploration Score 106
16.7 References 109

17 *Applications to Language Models* 110

17.1 *Introduction* 110

17.2 *The Architecture* 110

17.3 *The Geometry of Attention* 111

17.4 *Expressivity* 112

17.5 *References* 113

18 *Interpretability* 114

18.1 *Introduction* 114

18.2 *Neurons as Fundamental Units* 114

18.3 *Directions as Fundamental Units* 114

18.4 *Polytopes as Fundamental Units* 115

18.5 *Mechanistic Interpretability* 117

18.6 *References* 117

19 *Computation* 118

19.1 *Introduction* 118

19.2 *Jacobian-Vector Products* 118

19.3 *References* 120

Introduction

Part I

The Theoretical Foundations

1 Deep Learning

1.1 Introduction

Deep learning investigates the utilisation of deep networks to represent distributions on an input domain. Using a collection of training data and iterative learning algorithms, a deep network can be tuned to functional represent the distribution of the training data on its domain. Deep networks consist of a series of affine transformations separated by non-linear functions. Despite their architectural simplicity, deep networks are very expressive, representing a large function class. In this chapter we introduce the structure of deep networks and the paradigm they operate. We introduce some common components of deep networks and how they are combined to form expressive architectures.

1.2 Deep Networks

A deep network parameterised by parameters θ, is an operator $f_\theta : \mathbb{R}^d \to \mathbb{R}^{d^{(L)}}$ constructed as a composition of layers

$$f_\theta = \left(f^{(L)}_{\theta^{(L)}} \circ \cdots \circ f^{(1)}_{\theta^{(1)}} \right).$$

In this instance the deep network has L layers, where layer ℓ is parametersied by the set of parameters $\theta^{(\ell)} \subseteq \theta$.

A deep network can be thought of as taking a signal $\mathbf{x} \in \mathbb{R}^d$ and predicting an output $\mathbf{y} \in \mathbb{R}^{d^{(L)}}$; with each intermediary layer extracting *features* of the signal that are pertinent to making the prediction.

More specifically, the operator at layer ℓ takes as input $\mathbf{z}^{(\ell-1)}(\mathbf{x}) \in \mathbb{R}^{d^{(\ell-1)}}$ and generates a feature map $\mathbf{z}^{(\ell)}(\mathbf{x})$.

- Either, one can think of $\mathbf{z}^{(\ell)}$ as a prediction of the features in the signal $\mathbf{x}$, since

$$\mathbf{z}^{(\ell)}(\mathbf{x}) = \left(f^{(\ell)}_{\theta^{(\ell)}} \circ \cdots \circ f^{(1)}_{\theta^{(1)}} \right)(\mathbf{x}).$$

- Or, one can think of $\mathbf{z}^{(\ell)}$ as a signal that is inputted into the deep network $\left(f^{(L)}_{\theta^{(L)}} \circ \cdots \circ f^{(\ell+1)}_{\theta^{(\ell+1)}} \right)$ to form the prediction $\mathbf{y}$.

For clarity, we let $\mathbf{z}^{(0)} := \mathbf{x}$ and $d^{(0)} = d$.

1.3 *Paradigm*

The parameters θ of the deep network need to be learned so that its prediction is meaningful[1].

[1] For ease of exposition, we will consider the case of supervised learning.

Typically a loss function $\mathcal{L}$ is used to quantitatively capture the error of the deep network's prediction. Since the the layers of the deep network are differentiable almost everywhere with respect to their parameters and inputs, we can use this error to optimise the parameters θ through a first-order optimization policy such as gradient descent.

The exact form of the loss function is dependent on the task of focus and the type of data available.

- For regression problems, we are typically represented with a set of data $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ and we are required to optimise θ such that $f_\theta(\mathbf{x}_i) \approx \mathbf{y}_i$. A commonly used loss function in this scenario is the mean squared error loss function $\mathcal{L}_{\text{MSE}} : \mathbb{R}^{d^{(L)}} \times \mathbb{R}^{d^{(L)}} \to \mathbb{R}_{\geq 0}$ given by
$$\mathcal{L}_{\text{MSE}}(\mathbf{y}_i, f_\theta(\mathbf{x}_i)) = \|\mathbf{y}_i - f_\theta(\mathbf{x}_i)\|_2^2.$$
- For classification problems, we are typically presented with a set of data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and we are required to optimise θ such that $\operatorname{argmax}(f_\theta(\mathbf{x}_i)) = y_i$. A commonly used loss function in this scenario is the cross entropy loss function $\mathcal{L}_{\text{CE}} : \mathbb{R}_{\geq 0} \times \mathbb{R}^{d^{(L)}} \to \mathbb{R}_{\geq 0}$ given by
$$\mathcal{L}_{\text{CE}}(y_i, f_\theta(\mathbf{x}_i)) = -[f_\theta(\mathbf{x}_i)]_{y_i} + \log\left(\sum_{j=1}^{d^{(L)}} \exp\left([f_\theta(\mathbf{x}_i)]_j\right)\right).$$

1.4 *Operators and Layers*

The form of the intermediary operators $f_{\theta^{(\ell)}}^{(\ell)}$ is dependent on the type of signals to be observed by the deep network, the computational constraints of the deep network, and prior knowledge of the features the deep network should extract.

Fully Connected Operator

A fully connected operator performs an affine transformation on the input signal by multiplying by a dense matrix $W_{\text{fc}}^{(\ell)} \in \mathbb{R}^{d^{(\ell)} \times d^{(\ell-1)}}$ and adding a bias vector $b_{\text{fc}}^{(\ell)} \in \mathbb{R}^{d^{(\ell)}}$. So that,
$$f_{\text{fc}}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right) = W_{\text{fc}}^{(\ell)} \mathbf{z}^{(\ell-1)}(\mathbf{x}) + b_{\text{fc}}^{(\ell)}.$$

Convolutional Operator

A convolutional operator deals with signals exhibiting channels. More specifically, we consider our input signal as a tensor x rather than as a vector $\mathbf{x}$. We contextualise this by thinking of x as an image, by supposing that x has $C^{(0)}$ channels across $\left(I^{(0)} \times J^{(0)}\right)$ pixels. The ℓth feature map then contains $C^{(\ell)}$ channels across $\left(I^{(\ell)} \times J^{(\ell)}\right)$ pixels. We can form a vector representation $\mathbf{x}$ of x by flattening its dimensions, and we can retrieve the ith dimension of this representation with $[\mathbf{x}]_i$.

A convolution operator performs a constrained affine transformation on the input signal by multiplying by a multichannel block-circulant convolution matrix $\mathbf{C}_{\text{conv}}^{(\ell)} \in \mathbb{R}^{d^{(\ell)} \times d^{(\ell-1)}}$ and adding a bias vector $b_{\text{conv}}^{(\ell)} \in \mathbb{R}^{d^{(\ell)}}$. So that,

$$f_{\text{conv}}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right) = \mathbf{C}_{\text{conv}}^{(\ell)}\mathbf{z}^{(\ell-1)}(\mathbf{x}) + b_{\text{conv}}.$$

Activation Operators

Activation operators apply a non-linear function σ to its input either element-wise or across the whole input.

In the element-wise case we have $\sigma : \mathbb{R} \to \mathbb{R}$, so that

$$\left[f_{\sigma}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right)\right]_i = \sigma\left(\left[\mathbf{z}^{(\ell-1)}(\mathbf{x})\right]\right)$$

for $i = 1, \ldots, d^{(\ell)}$.

Example 1.4.1. *Commonly used activation functions include the following.*

1. *The rectified linear unit (ReLU), given by*

$$\sigma_{ReLU}(u) = \max(u, 0).$$

2. *The leaky rectified linear unit (leaky ReLU), given by*

$$\sigma_{LReLU}(u) = \max(\eta u, u)$$

   for some $\eta > 0$.

3. *The absolute value, given by*

$$\sigma_{abs}(u) = |u|.$$

4. *The sigmoid, given by*

$$\sigma_{sig}(u) = \frac{1}{1 + e^{-u}}.$$

5. *The swish, given by*

$$\sigma_{swish}(u;\eta) = \sigma_{sig}(\eta u)\, u$$

for $0 \leq \eta \leq 1$.

6. *The hyperbolic tangent, given by*

$$\sigma_{tanh}(u) = 2\sigma_{sig}(2u) - 1.$$

Observe how examples 1, 2, and 3 are piecewise affine and convex, whereas 4 and 5 have neither of these properties.

Activation operators acting on the whole input have $\sigma : \mathbb{R}^{d^{(\ell-1)}} \to \mathbb{R}^{d^{(\ell)}}$, so that

$$f_\sigma^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right) = \sigma\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right).$$

Example 1.4.2.

1. *The softmax non-linearity has*

$$\sigma_{softmax}(\mathbf{x}) = \begin{pmatrix} \frac{\exp([\mathbf{x}]_1)}{\sum_{j=1}^{d^{(\ell)}} \exp\left([\mathbf{x}]_j\right)} \\ \vdots \\ \frac{\exp\left([\mathbf{x}]_{d^{(\ell)}}\right)}{\sum_{j=1}^{d^{(\ell)}} \exp\left([\mathbf{x}]_j\right)} \end{pmatrix}.$$

2. *The maxout non-linearity has*

$$\sigma_{maxout}(\mathbf{x}) = \max_{j=1,\ldots,d^{(\ell)}} \mathbf{1}.$$

In subsequent analyses we will consider activation operators applying the non-linearity element-wise. could we rectify this with further analyses.

Pooling Operators

A pooling operator subsamples an input to reduce its dimensionality. Pooling operators enact a policy ρ over $d^{(\ell)}$ pooling regions given by a collection of indices $\left\{\mathcal{R}_i^{(\ell)}\right\}_{i=1}^{d^{(\ell)}}$.[2]

[2] It is necessary that $\left|\mathcal{R}_i^{(\ell)}\right| = \left|\mathcal{R}_{i'}^{(\ell)}\right|$ for each $i, i' \left\{1, \ldots, d^{(\ell)}\right\}$.

Max-Pooling Max-pooling has policy $\rho_{\max}$ so that

$$\left[f_{\rho_{\max}}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right)\right]_i = \max_{t \in \mathcal{R}_i^{(\ell)}} \left[\mathbf{z}^{(\ell-1)}(\mathbf{x})\right]_t$$

for $i = 1, \ldots, d^{(\ell)}$.

Mean-Pooling Mean-pooling has policy ρ_{mean} so that

$$\left[f^{(\ell)}_{\rho_{\text{mean}}}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right)\right]_i = \frac{1}{\left|\mathcal{R}^{(\ell)}_i\right|}\sum_{t\in\mathcal{R}^{(\ell)}_i}\left[\mathbf{z}^{(\ell-1)}(\mathbf{x})\right]_t$$

for $i = 1, \ldots, d^{(\ell)}$.

Local-Importance-Pooling Local-importance-pooling (Gao et al., 2023) has policy ρ_g so that

$$\left[f^{(\ell)}_{\rho_g}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right)\right]_i = \frac{\sum_{j\in\mathcal{R}^{(\ell)}_i} g\left(\left[\mathbf{z}^{(\ell-1)}(\mathbf{x})\right]_j\right)\left[\mathbf{z}^{(\ell-1)}(\mathbf{x})\right]_j}{\sum_{t\in\mathcal{R}^{(\ell)}_i} g\left(\left[\mathbf{z}^{(\ell-1)}(\mathbf{x})\right]_t\right)}$$

Normalisation Operators

Let ℓ be a deep network layer consisting of an affine transformation and activation operator σ_{act}, say

$$\mathbf{z}^{(\ell)} = \sigma_{\text{act}}\left(W^{(\ell)}_{\text{fc}}\mathbf{z}^{(\ell-1)} + b^{(\ell)}_{\text{fc}}\right).$$

Normalisation operators utilise parameters $\mu^{(\ell)}$, $\sigma^{(\ell)}$, $\beta^{(\ell)}$ and $\gamma^{(\ell)}$ to augment layer ℓ's mapping as

$$\left[\mathbf{z}^{(\ell)}\right]_i = \sigma_{\text{act}}\left(\frac{\left\langle\left[W^{(\ell)}_{\text{fc}}\right]_{i,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle - \left[\mu^{(\ell)}\right]_i}{\left[\sigma^{(\ell)}\right]_i}\left[\gamma^{(\ell)}\right]_i + \left[\beta^{(\ell)}\right]_i\right)$$

for $i = 1, \ldots, d^{(\ell)}$.

Typically, $\mu^{(\ell)}$ and $\sigma^{(\ell)}$ are computed for the deep network,[3] whereas $\gamma^{(\ell)}$ and $\beta^{(\ell)}$ are learned. For simplicity, we set $\gamma^{(\ell)} = \mathbf{1}$ and $\beta^{(\ell)} = \mathbf{0}$. The latter assumption can be empirically shown to have little effect on the performance of the deep network, we justify the former assumption theoretically with Proposition 1.4.3.

[3] The nature of the computations may vary depending on whether the deep network is being trained or tested.

Proposition 1.4.3. *The batch normalisation parameter $\gamma^{(\ell)} \neq \mathbf{0}$ does not impact the expressivity of the deep network layer.*

Proof. We proceed by showing that $\gamma^{(\ell)}$ can be absorbed into the

parameters of $W^{(\ell+1)}$ and $\beta^{(\ell)}$. Observe that

$$\begin{aligned}
\left[\mathbf{z}^{(\ell+1)}\right]_i &= \sigma_{\text{act}}\left(\frac{\sum_{j=1}^{d^{(\ell)}}\left[W^{(\ell+1)}\right]_{i,j}\sigma_{\text{act}}\left(\frac{\left\langle\left[W^{(\ell)}\right]_{j,\cdot},\mathbf{z}^{(\ell)}\right\rangle-\left[\mu^{(\ell)}\right]_j}{\left[\sigma^{(\ell)}\right]_j}\left[\gamma^{(\ell)}\right]_j+\left[\beta^{(\ell)}\right]_j\right)-\left[\mu^{(\ell+1)}\right]_i}{\left[\sigma^{(\ell+1)}\right]_i}\left[\gamma^{(\ell+1)}\right]_i+\left[\beta^{(\ell+1)}\right]_i\right)\\
&= \sigma_{\text{act}}\left(\frac{\sum_{j=1}^{d^{(\ell)}}\left[\gamma^{(\ell)}\right]_j\left[W^{(\ell+1)}\right]_{i,j}\sigma_{\text{act}}\left(\frac{\left\langle\left[W^{(\ell)}\right]_{j,\cdot},\mathbf{z}^{(\ell)}\right\rangle-\left[\mu^{(\ell)}\right]_j}{\left[\sigma^{(\ell)}\right]_j}+\frac{\left[\beta^{(\ell)}\right]_j}{\left[\gamma^{(\ell)}\right]_j}\right)-\left[\mu^{(\ell+1)}\right]_i}{\left[\sigma^{(\ell+1)}\right]_i}\left[\gamma^{(\ell+1)}\right]_i+\left[\beta^{(\ell+1)}\right]_i\right)\\
&= \sigma_{\text{act}}\left(\frac{\sum_{j=1}^{d^{(\ell)}}\left[\tilde{W}^{(\ell+1)}\right]_{i,j}\sigma_{\text{act}}\left(\frac{\left\langle\left[W^{(\ell)}\right]_{j,\cdot},\mathbf{z}^{(\ell)}\right\rangle-\left[\mu^{(\ell)}\right]_j}{\left[\sigma^{(\ell)}\right]_j}+\left[\tilde{\beta}^{(\ell)}\right]_j\right)-\left[\mu^{(\ell+1)}\right]_i}{\left[\sigma^{(\ell+1)}\right]_i}\left[\gamma^{(\ell+1)}\right]_i+\left[\beta^{(\ell+1)}\right]_i\right).
\end{aligned}$$

□

Batch Normalisation Let $\mathcal{X}$ be the training set, and let $\mathcal{B} \subseteq \mathcal{X}$. Let $\mathcal{X}^{(\ell)}$ be the features maps $\mathbf{z}^{(\ell)}$ formed at layer ℓ from the inputs $\mathbf{x} \in \mathcal{X}$, and similarly for $\mathcal{B}^{(\ell)}$. So that, during training, at mini-batch $\mathcal{B}$ we set

$$\begin{cases}\mu^{(\ell)} = \frac{1}{\left|\mathcal{B}^{(\ell)}\right|}\sum_{\mathbf{z}^{(\ell)}\in\mathcal{B}^{(\ell)}} W_{\text{fc}}^{(\ell)}\mathbf{z}^{(\ell)}\\ \sigma^{(\ell)} = \sqrt{\frac{1}{\left|\mathcal{B}^{(\ell)}\right|}\sum_{\mathbf{z}^{(\ell)}\in\mathcal{B}^{(\ell)}}\left(W_{\text{fc}}^{(\ell)}\mathbf{z}^{(\ell)}-\mu^{(\ell)}\right)^2}.\end{cases} \tag{1.1}$$

At test time, we compute the parameters $\bar{\mu}^{(\ell)}$ and $\bar{\sigma}^{(\ell)}$ by replacing $\mathcal{B}^{(\ell)}$ in (1.1) with $\mathcal{X}^{(\ell)}$.

Layer Normalisation During training a testing, we set

$$\begin{cases}\mu^{(\ell)} = \frac{1}{d^{(\ell)}}\sum_{i=1}^{d^{(\ell)}}\left[W_{\text{fc}}^{(\ell)}\mathbf{z}^{(\ell)}\right]_i\\ \sigma^{(\ell)} = \sqrt{\frac{1}{d^{(\ell)}}\sum_{i=1}^{d^{(\ell)}}\left(\left[W_{\text{fc}}^{(\ell)}\mathbf{z}^{(\ell)}\right]_i-\mu^{(\ell)}\right)^2}.\end{cases}$$

Skip Connection Operator

A skip-connection operator performs an affine transformation on the input signal of an operator $f_{\theta^{(\ell)}}^{(\ell)}$ with the output of the same operator. So that,

$$f_{\text{skip}}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x});f_{\theta^{(\ell)}}^{(\ell)}\right) = W_{\text{skip}}^{(\ell)}\mathbf{z}^{(\ell-1)}(\mathbf{x}) + f_{\theta^{(\ell)}}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right) + b_{\text{skip}}^{(\ell)}.$$

The skip connection can also be cast as a convolution between the input signal and the output signal of the operator by replacing the variables.

Recurrent Connection Operator

A recurrent operator acts on a time-series by performing a transformation on the input, its output at the previous time-step, as

$$f_{\text{rec}}^{(1,t)}(\mathbf{x}) = W_{\text{in}}^{(1)} x^t + W_{\text{rec}}^{(1)} \mathbf{z}^{(1,t-1)}(\mathbf{x}) + b^{(1)},$$

or when there is a previous layer, as

$$f_{\text{rec}}^{(\ell,t)}(\mathbf{x}) = W_{\text{in}}^{(\ell)} x^t + W_{\text{up}}^{(\ell)} \mathbf{z}^{(\ell-1,t)}(\mathbf{x}) + W_{\text{rec}}^{(\ell)} \mathbf{z}^{(\ell,t-1)}(\mathbf{x}) + b^{(\ell)}.$$

Output Operator

The output operator forms the output of the deep network by taking the prediction $\mathbf{y} \in \mathbb{R}^{d^{(L)}}$ and passing it through a non-linearity.

Layer

A layer of a deep network refers to an activation or pooling operator composed with an affine operator[4]. Typically, $f^{(\ell)}$ will depict a layer, so that a deep network is a composition of layers.

[4] Affine operators include fully connected operators, convolution operators, skip connection operators and recurrent connection operators. Typically, we will only consider piecewise affine activation operators, such as ReLU, leaky ReLU and absolute value.

1.5 *References*

Chapter 1 from Balestriero and Baraniuk (2018).

2 Spline Theory

2.1 Introduction

Spline theory is a framework for studying global function approximation using locally constructed simple functions. By partitioning the input domain judiciously, a more complex globally defined function can be well approximated using a collection of simple function constructed locally on each region of the partition. In this chapter, we will focus on the case where the simple functions are affine functions. We will start with spline operators where the underlying partition is explicitly set, and then move onto operators where the underlying partition is implicitly constructed from other parameters.

2.2 Affine Spline Functions

Let $\Omega = \{\omega_1, \ldots, \omega_r\}$ be a set of regions partitioning the domain $\mathbb{R}^d$. Let $\Phi = \{\phi_1, \ldots, \phi_r\}$ be a set of mappings operating on the regions Ω, via

$$\phi_j(\mathbf{x}) = \left\langle [A]_{j,\cdot}, \mathbf{x} \right\rangle + [B]_j$$

for $\mathbf{x} \in \omega_j$ and $j = 1, \ldots, r$, where $A \in \mathbb{R}^{r \times d}$ is a matrix of slopes and $B \in \mathbb{R}^r$ is a vector of offsets.[1]

[1] Also referred to as a vector of biases.

Definition 2.2.1. *An affine spline function $s[A, B, \Omega] : \mathbb{R}^d \to \mathbb{R}$ is given by*

$$s[A, B, \Omega](\mathbf{x}) = \sum_{j=1}^{r} \left(\left\langle [A]_{j,\cdot}, \mathbf{x} \right\rangle + [B]_j \right) \mathbf{1}_{\{\mathbf{x} \in \omega_j\}}.$$

Remark 2.2.2.

1. *More generally, the functions ϕ_j can be arbitrary, however we focus on the case when they are affine mappings.*

2. *For ease of notation we will often write*

$$s[A, B, \Omega] = \langle A[\mathbf{x}], \mathbf{x} \rangle + B[\mathbf{x}],$$

where $A[x] = [A]_{j,\cdot}$ for $\mathbf{x} \in \omega_j$ and similarly for $B[\mathbf{x}]$.

Proposition 2.2.3. *Let s be an affine spline function.*

1. *s is piecewise affine, and globally affine for $r = 1$.*

2. *s is piecewise convex, and global convex for $r = 1$.*

Remark 2.2.4.

1. *The case for $r = 1$ is usually referred to as degenerate, since it just corresponds to an affine mapping.*

2. *Note that an affine spline function need not be continuous along the boundaries between the regions Ω. However, in the case that it is, we refer to the function as a continuous piecewise affine spline function.*

2.3 *Affine Spline Operators*

Let $\Omega = \{\omega_1, \ldots, \omega_r\}$ be a set of regions partitioning the domain $\mathbb{R}^d$. Let $\Phi = \{\phi_1, \ldots, \phi_r\}$ be a set of mappings operating on the regions Ω, via

$$\phi_j(\mathbf{x}) = [A]_{\cdot,j,\cdot}\, \mathbf{x} + [B]_{j,\cdot}$$

for $\mathbf{x} \in \omega_j$ and $j = 1, \ldots, r$, where $A \in \mathbb{R}^{d \times r \times k}$ is a matrix of slopes and $B \in \mathbb{R}^{r \times k}$ is a vector of offsets.

Definition 2.3.1. *An affine spline operator $S[A, B, \Omega] : \mathbb{R}^d \to \mathbb{R}^k$ is given by*

$$S[A, B, \Omega](\mathbf{x}) = \sum_{j=1}^{r} \left([A]_{\cdot,j\cdot} \mathbf{x} + [B]_{j,\cdot} \right) \mathbf{1}_{\{\mathbf{x} \in \omega_j\}}.$$

Remark 2.3.2. *As remarked in Remark 2.2.4, the local mappings ϕ_j can be taken to be arbitrary functions, however we focus on the affine mappings. Moreover, if S is continuous along its boundaries we refer to it as a continous piecewise affine spline function.*

An affine spline operator can be seen as a natural extension of the affine spline function. An affine spline operator is constructed by stacking independent affine spline functions. More specifically, given k affine spline functions $s\left[A^{(i)}, B^{(i)}, \Omega^{(i)}\right] : \mathbb{R}^d \to \mathbb{R}$, the induced affine spline operator $S\left[\left(A^{(i)}, B^{(i)}, \Omega^{(i)}\right)_{i=1}^{k}\right] : \mathbb{R}^d \to \mathbb{R}^k$ is

$$S\left[\left(A^{(i)}, B^{(i)}, \Omega^{(i)}\right)_{i=1}^{k}\right](\mathbf{x}) = \begin{pmatrix} s\left[A^{(1)}, B^{(1)}, \Omega^{(1)}\right](\mathbf{x}) \\ \vdots \\ s\left[A^{(k)}, B^{(k)}, \Omega^{(k)}\right](\mathbf{x}) \end{pmatrix}.$$

The partition structure of S is not immediately clear in this construction, however it is still present, and it is explicitly given as

$$\Omega = \left(\bigcup_{\left(\omega^{(1)},\ldots,\omega^{(k)}\right) \in \Omega^{(1)} \times \Omega^{(k)}} \left(\bigcap_{j=1,\ldots,k} \omega^{(j)} \right) \right) \setminus \emptyset.$$

More specifically, the local mapping on $\omega_j = \left(\omega^{(1)}, \ldots, \omega^{(k)}\right) \in \Omega$ is given by

$$\phi_j(\mathbf{x}) = \begin{pmatrix} \phi^{(1)}_{\omega^{(1)}}(\mathbf{x}) \\ \vdots \\ \phi^{(k)}_{\omega^{(k)}}(\mathbf{x}) \end{pmatrix},$$

where $\phi^{(1)}_{\omega^{(1)}}$ denotes the local mapping of $s\left[A^{(1)}, B^{(1)}, \Omega^{(1)}\right]$ on the region $\omega^{(1)} \in \Omega^{(1)}$. Note that by construction we have $\omega_j \subseteq \omega^{(i)}$ for each $i = 1, \ldots, k$. In particular, if $\omega^{(i)} =\in \Omega^{(i)}$ has index q_i, then we also have $\omega_j \cap \omega^{(i)}_p = \emptyset$ for $p \in \{1, \ldots, r^{(i)}\} \setminus q_i$. In other words, for $r = |\Omega|$ we have that for all $(j, i) \in \{1, \ldots, r\} \times \{1, \ldots, k\}$ there exists a unique $q_i \in \left\{1, \ldots, r^{(i)}\right\}$ such that

$$\omega_j \cap \omega^{(i)}_p = \begin{cases} \omega_j & p = q_i \\ \emptyset & p \neq q_i. \end{cases}$$

Allowing us to write

$$\begin{aligned} \phi_j(\mathbf{x}) &= \begin{pmatrix} \phi^{(1)}_{q_1}(\mathbf{x}) \\ \vdots \\ \phi^{(k)}_{q_k}(\mathbf{x}) \end{pmatrix} \\ &= \begin{pmatrix} \left[A^{(1)}\right]^{\top}_{q_1,\cdot} \\ \vdots \\ \left[A^{(k)}\right]_{q_k,\cdot} \end{pmatrix} \mathbf{x} + \begin{pmatrix} \left[B^{(1)}\right]_{q_1} \\ \vdots \\ \left[B^{(k)}\right]_{q_k} \end{pmatrix} \\ &=: [A]_{\cdot,j,\cdot}\mathbf{x} + [B]_{j,\cdot}, \end{aligned}$$

allowing us to recover the construction given in Definition 2.3.1 as

$$S\left[A, B, \Omega\right] = \sum_{j=1}^{r} \phi_j(\mathbf{x}) \mathbf{1}_{\{\mathbf{x} \in \omega_j\}}.$$

2.4 *Max-Affine Spline Functions*

The specification of a partitioning Ω for affine spline functions is burdensome in practical settings. Therefore, it is of interest to eliminate it by imposing some additional conditional on its constructions.

However, this will inevitably reduce the expressivity of the resulting class of functions.

Definition 2.4.1. *A max-affine spline function* $s[A, B] : \mathbb{R}^d \to \mathbb{R}$ *is given by*

$$s[A, B] = \max_{j=1,\dots,r} \left(\langle [A]_{j,\cdot}, \mathbf{x} \rangle + [B]_j \right),$$

for $A \in \mathbb{R}^{r \times d}$ *and* $B \in \mathbb{R}^r$.

Remark 2.4.2. *The partition* Ω *is not specified in Definition 2.4.1 since it is implicitly constructed with the maximum operator. Indeed, a max-affine spline function is completely determined by the parameters* A *and* B.

As pre-cautioned, max affine spline functions express a restricted space of functions, however, as we will see in Proposition 2.4.3, this restriction is still meaningful.

Proposition 2.4.3. *Let* s *be a max-affine spline function.*

1. s *is piecewise affine.*
2. s *is globally convex.*
3. s *is continuous.*

Proof.

1. The partition Ω of induced by s contains finitely many regions. On each of these regions s is an affine function, namely $\langle [A]_{j,\cdot}, \mathbf{x} \rangle + [B]_j$ for some $j = 1, \dots, r$. Therefore, s is piecewise affine.
2. Let $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$. Then for $\lambda \in [0, 1]$ we have,

$$\begin{aligned} s\left(\lambda \mathbf{x} + (1-\lambda)\mathbf{x}'\right) &= \max_{j=1,\dots,r} \left(\langle [A]_{j,\cdot}, \lambda \mathbf{x} + (1-\lambda)\mathbf{x}' \rangle + [B]_j \right) \\ &= \max_{j=1,\dots,r} \Big(\langle [A]_{j,\cdot}, \lambda \mathbf{x} \rangle + \langle [A]_{j,\cdot}, (1-\lambda)\mathbf{x}' \rangle \\ &\quad + \lambda [B]_j + (1-\lambda)[B]_j \Big) \\ &\leq \lambda \max_{j=1,\dots,r} \left(\langle [A]_{j,\cdot}, \mathbf{x} \rangle + [B]_j \right) \\ &+ (1-\lambda) \max_{j=1,\dots,r} \left(\langle [A]_{j,\cdot}, \mathbf{x}' \rangle + [B]_j \right) \\ &= \lambda s(\mathbf{x}) + (1-\lambda) s\left(\mathbf{x}'\right). \end{aligned}$$

3. Using statements 1 and 2, we have that s is piecewise affine and globally convex. Continuity can only fail at knots of s. Suppose for contradiction that s is not continuous at $x \in \mathbb{R}$. In particular, suppose without loss of generality that for $z \nearrow x$ we have $s(z) \to y_1$, and for $z \searrow x$ we have $s(z) \to y_2$ with $y_1 < y_2$. Now choose

$\tilde{z} \in (x, x+\epsilon)$, for some $\epsilon > 0$, such that z is part of the same affine component as x_+ and $s(\tilde{z}) > y_1$. Now let

$$\begin{cases} \lambda = \frac{1}{2}\left(\frac{y_2 - y_1}{s(\tilde{z}) - y_1}\right) & s(\tilde{z}) > y_2 \\ \frac{1}{2} & \text{otherwise.} \end{cases}$$

The point $\lambda x + (1-\lambda)\tilde{z}$ is in the same affine component as x_+ and $\tilde{z}$. Thus,

$$\begin{aligned} s(\lambda x + (1-\lambda)\tilde{x}) &= \lambda s(x_+) + (1-\lambda) s(\tilde{z}) \\ &\in (\min(y_2, s(\tilde{z})), \max(y_2, s(\tilde{z}))). \end{aligned}$$

However,

$$\lambda s(x_-) + (1-\lambda) s(\tilde{z}) \in (y_1, \min(y_2, s(\tilde{z}))),$$

meaning s is not globally convex. Therefore, it must be the case that s is continuous at x, and thus s is continuous.

□

Due to statement 3 of Proposition 2.4.3 tells us that max-affine spline functions are continuous piecewise affine spline functions.

Proposition 2.4.4. *For any function h that is piecewise affine and globally convex, there exists parameters A and B such that*

$$h(\mathbf{x}) = s[A, B](\mathbf{x})$$

for every $\mathbf{x} \in \mathbb{R}^d$.

Remark 2.4.5. *Note how the combination of statements 1 and 2 from Proposition 2.4.3 and Proposition 2.4.4 provide an equivalence between max-affine spline functions and piecewise affine, globally convex functions.*

2.5 *Max-Affine Spline Operators*

As in the case of affine spline operators, max-affine spline operators are the natural multi-dimensional extension of max-affine spline functions, obtained by stacking independent max-affine spline functions.

Definition 2.5.1. *A max-affine spline operator $S[A, B] : \mathbb{R}^d \to \mathbb{R}^k$ is an affine spline operator, constructed by stacking k max-affine spline functions*

$$S[A, B](\mathbf{x}) = \begin{pmatrix} \max_{j=1,\ldots,r}\left(\langle [A]_{1,j,\cdot}, \mathbf{x}\rangle + [B]_{1,j}\right) \\ \vdots \\ \max_{j=1,\ldots,r}\left(\left\langle [A]_{k,j,\cdot}, \mathbf{x}\right\rangle + [B]_{k,j}\right) \end{pmatrix},$$

where $A \in \mathbb{R}^{k \times r \times d}$ and $B \in \mathbb{R}^{k \times r}$.

Proposition 2.5.2. *For any operator H that is piecewise affine and globally convex, there exists parameters A and B such that*

$$H(\mathbf{x}) = S[A, B](\mathbf{x})$$

for every $\mathbf{x} \in \mathbb{R}^d$.

Definition 2.5.3. *A Lipschitz constant of an operator $f : \mathbb{R}^d \to \mathbb{R}^k$ is a constant $\kappa \in \mathbb{R}$ such that*

$$\|f(\mathbf{x}_1) - f(\mathbf{x}_2)\|_2 \leq \kappa \|\mathbf{x}_1 - \mathbf{x}_2\|_2,$$

for every $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^d$.

Reference to *the* Lipschitz constant refers to smallest such constant.

Proposition 2.5.4. *A Lipschitz constant of a max-affine spline operator $S[A, B] : \mathbb{R}^d \to \mathbb{R}^k$ is*

$$\kappa = \sqrt{\sum_{i=1}^{d} \left(\max_{j=1,\ldots,r} \|[A]_{i,j,\cdot}\| \right)^2}.$$

Proof. Let $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^d$, then

$$\begin{aligned}
\|S[A, B](\mathbf{x}_1) - S[A, B](\mathbf{x}_2)\|^2 &= \sum_{i=1}^{d} |[[S[A, B](\mathbf{x}_1)]_i - [S[A, B](\mathbf{x}_2)]_i|^2 \\
&= \sum_{i=1}^{d} \left| \max_{j=1,\ldots,r} \left(\langle [A]_{i,j,\cdot}, \mathbf{x}_1 \rangle + [B]_j \right) - \max_{j=1,\ldots,r} \left(\langle [A]_{i,j,\cdot}, \mathbf{x}_2 \rangle + [B]_j \right) \right|^2 \\
&\leq \sum_{i=1}^{d} \left| \max_{j=1,\ldots,r} \left(\langle [A]_{i,j,\cdot}, \mathbf{x}_1 - \mathbf{x}_2 \rangle \right) \right|^2 \\
&\overset{(1)}{\leq} \left(\sum_{i=1}^{d} \left(\max_{j=1,\ldots,r} \|[A]_{i,j,\cdot}\| \right)^2 \right) \|\mathbf{x}_1 - \mathbf{x}_2\|^2,
\end{aligned}$$

where (1) is an application of the Cauchy-Schwarz inequality. Therefore, a Lipschitz constant of S is

$$\kappa = \sqrt{\sum_{i=1}^{d} \left(\max_{j=1,\ldots,r} \|[A]_{i,j,\cdot}\| \right)^2}.$$

□

2.6 *References*

Chapter 2 from Balestriero and Baraniuk (2018).

3 Deep Networks and Spline Operators

3.1 Introduction

Both deep networks and spline operators can be viewed as functional approximation methods. As we will show in this chapter, a large class of deep networks are equivalent to spline operators. Consequently, we can translate questions regarding deep networks into the language of spline theory, opening an array of research avenues, for which it will be the purpose of this book to explore. In this chapter we first recognise how certain deep network operators can be represented as max affine spline operators which we then utilise to demonstrate that layers–which are compositions of operators– are continuous affine spline operators. We then explore how these layers can be combined to represent common deep network architectures such as convolutional neural networks, residual neural networks and recurrent neural networks.

3.2 Deep Network Operators as Spline Operators

Proposition 3.2.1. *A fully connected operator* $f_{fc}^{(\ell)}$ *is a degenerate max-affine spline operator* $S\left[A_{fc}^{(\ell)}, B_{fc}^{(\ell)}\right]$ *with* $r = 1$, $\left[A_{fc}^{(\ell)}\right]_{j,1,\cdot} = \left[W_{fc}^{(\ell)}\right]_{j,\cdot}$ *and* $\left[B_{fc}^{(\ell)}\right]_{j,1} = \left[b_{fc}^{(\ell)}\right]_{j}$ *for* $j = 1, \ldots, d^{(\ell)}$.

Proof. With $A_{\text{fc}}^{(\ell)}$ and $B_{\text{fc}}^{(\ell)}$ as given in Proposition 3.2.1, it is clear that

$$\begin{aligned} f_{\text{fc}}^{(\ell)}(\mathbf{x}) &= W_{\text{fc}}^{(\ell)}\mathbf{x} + b_{\text{fc}}^{(\ell)} \\ &= \begin{pmatrix} \left\langle \left[W_{\text{fc}}^{(\ell)}\right]_{1,\cdot}, \mathbf{x} \right\rangle + \left[b_{\text{fc}}^{(\ell)}\right]_1 \\ \vdots \\ \left\langle \left[W_{\text{fc}}^{(\ell)}\right]_{d^{(\ell)},\cdot}, \mathbf{x} \right\rangle + \left[b_{\text{fc}}^{(\ell)}\right]_{d^{(\ell)}} \end{pmatrix} \\ &= \begin{pmatrix} \max_{j=1}\left(\left\langle \left[A_{\text{fc}}^{(\ell)}\right]_{1,j,\cdot}, \mathbf{x} \right\rangle + \left[B_{\text{fc}}^{(\ell)}\right]_{1,j} \right) \\ \vdots \\ \max_{j=1}\left(\left\langle \left[A_{\text{fc}}^{(\ell)}\right]_{d^{(\ell)},j,\cdot}, \mathbf{x} \right\rangle + \left[B_{\text{fc}}^{(\ell)}\right]_{d^{(\ell)},j} \right) \end{pmatrix} \\ &= S\left[A_{\text{fc}}^{(\ell)}, B_{\text{fc}}^{(\ell)}\right](\mathbf{x}). \end{aligned}$$

□

Remark 3.2.2. *Proposition 3.2.1 also applies to convolution operators by respectively replacing $W_{fc}^{(\ell)}, b_{fc}^{(\ell)}$ with $\mathbf{C}_{conv}^{(\ell)}, b_{conv}^{(\ell)}$.*

Proposition 3.2.3. *An activation operator $f_{\sigma}^{(\ell)}$, where σ is piecewise affine and convex, is a max-affine spline operator $S\left[A_{\sigma}^{(\ell)}, B_{\sigma}^{(\ell)}\right]$ with $r = 2$, $\left[B_{\sigma}^{(\ell)}\right]_{j,1} = \left[B_{\sigma}^{(\ell)}\right]_{j,2} = 0$ for every j and $A_{\sigma}^{(\ell)}$ being dependent on σ.*

Proof. We consider the case $\sigma = \sigma_{\text{ReLU}}$, with the understanding that the other cases follow analogously. It follows that

$$\begin{aligned} \sigma_{\text{ReLU}}(\mathbf{x}) &= \begin{pmatrix} \sigma_{\text{ReLU}}\left([\mathbf{x}]_1\right) \\ \vdots \\ \sigma_{\text{ReLU}}\left([\mathbf{x}]_{d^{(\ell)}}\right) \end{pmatrix} \\ &= \begin{pmatrix} \max\left(0, [\mathbf{x}]_1\right) \\ \vdots \\ \max\left(0, [\mathbf{x}]_{d^{(\ell)}}\right) \end{pmatrix} \\ &= \begin{pmatrix} \max\left(0, \langle \mathbf{e}_1, \mathbf{x}\rangle\right) \\ \vdots \\ \max\left(0, \langle \mathbf{e}_{d^{(\ell)}}, \mathbf{x}\rangle\right) \end{pmatrix}. \end{aligned}$$

Therefore, $\sigma_{\text{ReLU}}(\mathbf{x}) = S\left[A_{\sigma}^{(\ell)}, \mathbf{0}\right](\mathbf{x})$ where

$$\begin{cases} \left[A_{\sigma}^{(\ell)}\right]_{j,1,\cdot} = \mathbf{0} \\ \left[A_{\sigma}^{(\ell)}\right]_{j,2,\cdot} = \mathbf{e}_j \end{cases}$$

for $j = 1, \ldots, d^{(\ell)}$. □

Example 3.2.4.

1. *For ReLU we have*
$$\begin{cases} \left[A_\sigma^{(\ell)}\right]_{j,1,\cdot} = \mathbf{0} \\ \left[A_\sigma^{(\ell)}\right]_{j,2,\cdot} = \mathbf{e}_j \end{cases}$$
for $j = 1, \ldots, d^{(\ell)}$.

2. *For leaky ReLU we have*
$$\begin{cases} \left[A_\sigma^{(\ell)}\right]_{j,1,\cdot} = \eta \mathbf{e}_j \\ \left[A_\sigma^{(\ell)}\right]_{j,2,\cdot} = \mathbf{e}_j \end{cases}$$
for $j = 1, \ldots, d^{(\ell)}$.

3. *For the absolute value we have*
$$\begin{cases} \left[A_\sigma^{(\ell)}\right]_{j,1,\cdot} = -\mathbf{e}_j \\ \left[A_\sigma^{(\ell)}\right]_{j,2,\cdot} = \mathbf{e}_j \end{cases}$$
for $j = 1, \ldots, d^{(\ell)}$.

Proposition 3.2.5. *A pooling operator* $f_\rho^{(\ell)}$ *that is piecewise affine and convex is a max-affine spline operator* $S\left[A_\rho^{(\ell)}, B_\rho^{(\ell)}\right]$.

Proof. We consider the case $\rho = \rho_{\max}$, with the understanding that the other cases follow analogously. It follows that

$$\begin{aligned} f_{\rho_{\max}}^{(\ell)}(\mathbf{x}) &= \begin{pmatrix} \max_{t \in \mathcal{R}_1^{(\ell)}} \left([\mathbf{x}]_t\right) \\ \vdots \\ \max_{t \in \mathcal{R}_{d^{(\ell)}}^{(\ell)}} \left([\mathbf{x}]_t\right) \end{pmatrix} \\ &= \begin{pmatrix} \max_{t \in \mathcal{R}_1^{(\ell)}} \left(\langle \mathbf{e}_t, \mathbf{x} \rangle\right) \\ \vdots \\ \max_{t \in \mathcal{R}_{d^{(\ell)}}^{(\ell)}} \left(\langle \mathbf{e}_t, \mathbf{x} \rangle\right) \end{pmatrix}. \end{aligned}$$

Therefore, $f_{\rho_{\max}}^{(\ell)}(\mathbf{x}) = S\left[A_{\rho_{\max}}, B_{\rho_{\max}}^{(\ell)}\right](\mathbf{x})$ where

$$\begin{cases} \left[A_\rho^{(\ell)}\right]_{j,i,\cdot} = \mathbf{e}_{\left[\mathcal{R}_j^{(\ell)}\right]_i} \\ \left[B_\rho^{(\ell)}\right]_{j,i} = 0 \end{cases}$$

for every $j = 1, \ldots, d^{(\ell)}$ and $i = 1, \ldots, r$ with $r = \left|\mathcal{R}_1^{(\ell)}\right|$. □

Example 3.2.6.

1. *For max pooling we have* $r = \left|\mathcal{R}_1^{(\ell)}\right|$*, and*

$$\begin{cases} \left[A_\rho^{(\ell)}\right]_{j,i,\cdot} = \mathbf{e}_{\left[\mathcal{R}_j^{(\ell)}\right]_i} \\ \left[B_\rho^{(\ell)}\right]_{j,i} = 0 \end{cases}$$

for every $j = 1, \ldots, d^{(\ell)}$ *and* $i = 1, \ldots, r$*.*

2. *For mean pooling we have* $r = 1$*, and*

$$\begin{cases} \left[A_\rho^{(\ell)}\right]_{j,1,\cdot} = \frac{1}{|\mathcal{R}_j|} \sum_{t \in \mathcal{R}_j} \mathbf{e}_t \\ \left[B_\rho^{(\ell)}\right]_{j,1} = 0 \end{cases}$$

for every $j = 1, \ldots, d^{(\ell)}$*.*

Note that if the parameters $\mu^{(\ell)}$ and $\sigma^{(\ell)}$ for normalisation operators are fixed, then normalisation operators are just affine transformations and are thus max-affine splines. However, in the case of batch normalisation during and layer normalisation the parameter $\sigma^{(\ell)}$ depend non-linearly on the input and thus are not max-affine spline operators.

Proposition 3.2.7. *A skip connection operator* $f_{skip}^{(\ell)}$ *is a max-affine spline operator* $S\left[A_{skip}^{(\ell)}, B_{skip}^{(\ell)}\right]$*.*

Proof. This follows analogously to Proposition 3.2.1 after modifying $f_{\text{skip}}^{(\ell)}$ to take input $\left(\mathbf{x}, f_{\theta^{(\ell)}}^{(\ell)}(\mathbf{x})\right)^\top$, where we are concatenating the vectors. □

Proposition 3.2.8 (Wang et al. (2019))**.** *A recurrent connection operators* $f_{rec}^{(1,t)}$ *and* $f_{rec}^{(\ell,t)}(\mathbf{x})$ *are max-affine spline operators* $S\left[A_{rec}^{(1,t)}, B_{rec}^{(1,t)}\right]$ *and* $S\left[A_{rec}^{(\ell,t)}, B_{rec}^{(\ell,t)}\right]$ *respectively.*

Proof. This follows analogously to Proposition 3.2.1 after modifying $f_{\text{rec}}^{(1,t)}$ and $f_{\text{rec}}^{(\ell,t)}$ to take inputs $\left(\mathbf{x}, \mathbf{z}^{(1,t-1)}\right)^\top$ and $\left(\mathbf{x}, \mathbf{z}^{(\ell-1,t)}, \mathbf{z}^{(\ell,t-1)}\right)$ respectively, where we are concatenating the vectors in each case. □

Proposition 3.2.9. *Any deep network layer as per Section 1.4, is a max-affine spline operator.*

Proof. We consider a fully connected operator followed by an activation operation, with the understanding that the other cases follow analogously. It follows that

$$
\begin{aligned}
&S\left[A_\sigma^{(\ell)}, B_\sigma^{(\ell)}\right]\left(W_{\text{fc}}^{(\ell)}\mathbf{x} + b_{\text{fc}}^{(\ell)}\right) \\
&\quad = \begin{pmatrix} \max_{j=1,\dots,r^{(\sigma)}} \left(\left(\left[A_\sigma^{(\ell)}\right]_{1,j,\cdot}\right)^\top \left(W_{\text{fc}}^{(\ell)}\mathbf{x} + b_{\text{fc}}^{(\ell)}\right) + \left[B_\sigma^{(\ell)}\right]_{1,j}\right) \\ \vdots \\ \max_{j=1,\dots,r^{(\sigma)}} \left(\left(\left[A_\sigma^{(\ell)}\right]_{d^{(\ell)},j,\cdot}\right)^\top \left(W_{\text{fc}}^{(\ell)}\mathbf{x} + b_{\text{fc}}^{(\ell)}\right) + \left[B_\sigma^{(\ell)}\right]_{d^{(\ell)},j}\right) \end{pmatrix} \\
&\quad = \begin{pmatrix} \max_{j=1,\dots,r^{(\sigma)}} \left(\left(\left(W_{\text{fc}}^{(\ell)}\right)^\top \left[A_\sigma^{(\ell)}\right]_{1,j,\cdot}\right)^\top \mathbf{x} + \left(\left[A_\sigma^{(\ell)}\right]_{1,j,\cdot}\right)^\top b_{\text{fc}}^{(\ell)} + \left[B_\sigma^{(\ell)}\right]_{1,j}\right) \\ \vdots \\ \max_{j=1,\dots,r^{(\sigma)}} \left(\left(\left(W_{\text{fc}}^{(\ell)}\right)^\top \left[A_\sigma^{(\ell)}\right]_{d^{(\ell)},j,\cdot}\right)^\top \mathbf{x} + \left(\left[A_\sigma^{(\ell)}\right]_{d^{(\ell)},j,\cdot}\right)^\top b_{\text{fc}}^{(\ell)} + \left[B_\sigma^{(\ell)}\right]_{k,j}\right) \end{pmatrix}.
\end{aligned}
$$

Therefore, $\left(\sigma \circ f_{\text{fc}}^{(\ell)}\right)(\mathbf{x}) = S\left[A^{(\ell)}, B^{(\ell)}\right](\mathbf{x})$ where

$$
\begin{cases} \left[A^{(\ell)}\right]_{i,j,\cdot} = \left(W_{\text{fc}}^{(\ell)}\right)^\top \left[A_\sigma^{(\ell)}\right]_{i,j,\cdot} \\ \left[B^{(\ell)}\right]_{i,j} = \left[B_\sigma^{(\ell)}\right]_{i,j} + \left\langle b_{\text{fc}}^{(\ell)}, [A_\sigma]_{i,j,\cdot}\right\rangle, \end{cases}
$$

for $j = 1, \dots, r^{(\sigma)}$ and $i = 1, \dots, d^{(\ell)}$. □

Example 3.2.10. *For an affine transformation followed by an activation function we have*

$$
\begin{cases} \left[A^{(\ell)}\right]_{i,j,\cdot} = \left(W_{fc}^{(\ell)}\right)^\top \left[A_\sigma^{(\ell)}\right]_{i,j,\cdot} \\ \left[B^{(\ell)}\right]_{i,j} = \left[b_\sigma^{(\ell)}\right]_{i,j} + \left\langle b_{fc}^{(\ell)}, [A_\sigma]_{i,j,\cdot}\right\rangle, \end{cases}
$$

for $j = 1, \dots, r^{(\sigma)}$ and $i = 1, \dots, d^{(\ell)}$.

maybe add the other combinations as exercises

3.3 Deep Networks as Spline Operators

Proposition 3.3.1. *The composition of max-affine spline operators is a continuous piecewise affine spline operator.*

Proof. Without loss of generality we consider max-affine spline operators $s^{(1)}\left[A^{(1)}, B^{(1)}\right] : \mathbb{R}^d \to \mathbb{R}^{d^{(1)}}$ and $s^{(2)}\left[A^{(2)}, B^{(2)}\right] : \mathbb{R}^{d_1} \to \mathbb{R}$. In particular, suppose the induced partitioning of $s^{(1)}$ is $\left\{\omega_1^{(1)}, \dots, \omega_{r^{(1)}}^{(1)}\right\}$ and the induced partitioning of $s^{(2)}$ is $\left\{\omega_1^{(2)}, \dots, \omega_{r^{(2)}}^{(2)}\right\}$.

So that, $A^{(1)} \in \mathbb{R}^{r^{(1)} \times d}$, $B^{(1)} \in \mathbb{R}^{r^{(1)}}$, $A^{(2)} \in \mathbb{R}^{r^{(2)}}$ and $B^{(2)} \in \mathbb{R}$. Then,

$$\begin{aligned}(s_2 \circ s_1)(\mathbf{x}) &= s_2\left(\max_{j=1,\ldots,r^{(1)}}\left(\left\langle \left[A^{(1)}\right]_{j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_j\right)\right) \\ &= \max_{j'=1,\ldots,r^{(2)}}\left(\left[A^{(2)}\right]_{j'}\left(\max_{j=1,\ldots,r^{(1)}}\left(\left\langle \left[A^{(1)}\right]_{j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_j\right)\right) + B^{(2)}\right).\end{aligned}$$

In particular,

$$(s_2 \circ s_1)(\mathbf{x}) = \sum_{j'=1}^{r^{(2)}}\sum_{j=1}^{r^{(1)}}\left(\left\langle \left[A^{(2)}\right]_{j'}\left[A^{(1)}\right]_{j,\cdot}, \mathbf{x}\right\rangle + \left[A^{(2)}\right]_{j'}\left[B^{(1)}\right]_j + B^{(2)}\right)\mathbf{1}_{\left\{\mathbf{x}\in\omega_j^{(1)}\right\}\cap\left\{s_1(\mathbf{x})\in\omega_{j'}^{(2)}\right\}}.$$

Therefore, $s_1 \circ s_2$ is an affine spline function. Continuity follows as s_1 and s_2 are continuous from statement 3 of Proposition 2.4.3 and so their composition is also continuous. □

Corollary 3.3.2. *A deep network constructed from layers as per Section 1.4, is a continuous piecewise affine spline operator.*

Remark 3.3.3. *Since the deep network mapping is an affine spline operator, it follows that the prediction (or feature map) is signal dependent as the mapping is dependent on which partition the signal fall into.*

3.4 Constructions

Convolutional Neural Network

The input-output formula for a convolutional neural network utilising a convolution operator, ReLU activation operator, a max-pooling operator and a fully connected operator is,[1]

$$\begin{aligned}\mathbf{z}_{\text{CNN}}^{(L)}(\mathbf{x}) = W_{\text{fc}}^{(L)} &\underbrace{\left(\prod_{\ell=L-1}^{1} A_\rho^{(\ell)}[\mathbf{x}]A_\sigma^{(\ell)}[\mathbf{x}]\mathbf{C}_{\text{conv}}^{(\ell)}\right)}_{A_{\text{CNN}}[\mathbf{x}]}\mathbf{x} \\ &+ \underbrace{W_{\text{fc}}^{(L)}\sum_{\ell=1}^{L-1}\left(\prod_{j=L-1}^{\ell+1} A_\rho^{(j)}[\mathbf{x}]A_\sigma^{(j)}[\mathbf{x}]\mathbf{C}_{\text{conv}}^{(j)}\right)\left(A_\rho^{(\ell)}[\mathbf{x}]A_\sigma^{(\ell)}[\mathbf{x}]b_{\text{conv}}^{(\ell)}\right)}_{B_{\text{CNN}}[\mathbf{x}]} \\ &+ b_{\text{fc}}^{(L)}. \qquad (3.1)\end{aligned}$$

[1] Through this analysis we are streamlining notation by adopting the notation convention from statement 2 of Remark 2.2.2.

We can collapse (3.1) into

$$\mathbf{z}_{\text{CNN}}^{(L)}(\mathbf{x}) = W_{\text{fc}}^{(L)}\mathbf{z}_{\text{CNN}}^{(L-1)} + B_{\text{fc}}^{(L)}, \qquad (3.2)$$

where

$$\mathbf{z}_{\text{CNN}}^{(L-1)}(\mathbf{x}) = A_{\text{CNN}}[\mathbf{x}]\mathbf{x} + B_{\text{CNN}}[\mathbf{x}],$$

can be thought of as a signal-dependent affine formula for the featurization of the signal $\mathbf{x}$.

ResNet

A ResNet utilises skip-connections. The linear skip-connection involves a convolution operator followed by an activation operator in parallel with a second convolution operator,

$$\mathbf{z}^{(\ell)}(\mathbf{x}) = \mathbf{C}_{\text{skip}}^{(\ell)}\mathbf{z}^{(\ell-1)}(\mathbf{x}) + A_{\sigma}^{(\ell)}[\mathbf{x}]\left(\mathbf{C}_{\text{conv}}^{(\ell)}\mathbf{z}^{(\ell-1)}(\mathbf{x}) + b_{\text{conv}}^{(\ell)}\right) + b_{\text{skip}}^{(\ell)}.$$

Pooling in a ResNet is achieved by subsampling the rows of $\mathbf{C}_{\text{skip}}^{(\ell)}$, $A^{(\ell)}[\mathbf{x}]$ and $b_{\text{skip}}^{(\ell)}$. The input-output formula for a ResNet constructed from a convolution operator, ReLU activation operator, max-pooling operator in parallel with convolution operator and with a last fully connected operator is

$$\begin{aligned}\mathbf{z}_{\text{RES}}^{(L)}(\mathbf{x}) = {} & W_{\text{fc}}^{(L)} \underbrace{\prod_{\ell=L-1}^{1}\left(A_{\rho}^{(\ell)}[\mathbf{x}]A_{\sigma}^{(\ell)}[\mathbf{x}]\mathbf{C}_{\text{conv}}^{(\ell)} + \mathbf{C}_{\text{skip}}^{(\ell)}\right)}_{A_{\text{RES}}[\mathbf{x}]}\mathbf{x} \\ & + W_{\text{fc}}^{(L)} \underbrace{\sum_{\ell=L-1}^{1}\left(\prod_{i=L-1}^{\ell+1}\left(A_{\rho}^{(\ell)}[\mathbf{x}]A_{\sigma}^{(\ell)}[\mathbf{x}]\mathbf{C}_{\text{conv}}^{(\ell)} + \mathbf{C}_{\text{skip}}^{(\ell)}\right)\right)\left(A_{\rho}^{(\ell)}[\mathbf{x}]A_{\sigma}^{(\ell)}[\mathbf{x}]b_{\text{conv}}^{(\ell)} + b_{\text{skip}}^{(\ell)}\right)}_{B_{\text{RES}}[\mathbf{x}]} \\ & + b_{\text{fc}}^{(L)}. \end{aligned} \tag{3.3}$$

We see that (3.3) has a similar functional form to (3.1), namely, a signal dependent featurizer followed by a linear classifier. In this case, there is an additional linear path from the input to the output.

Recurrent Neural Networks

A recurrent neural network evolves hidden states through time, say up to time T, and through depth using layers. Namely, at the first layer, at time step t, the hidden state is updated using the hidden state of the previous time step and the input at the current time step as

$$\begin{aligned}\mathbf{z}_{\text{RNN}}^{(1,t)} = {} & A_{\sigma}^{(1,t)}\left(W_{\text{in}}^{(1)}x^{t} + W_{\text{rec}}^{(1)}\mathbf{z}_{\text{RNN}}^{(1,t-1)} + b^{(1)}\right) \\ & \cdot\left(W_{\text{in}}^{(1)}x^{t} + W_{\text{rec}}^{(1)}\mathbf{z}_{\text{RNN}}^{(1,t-1)} + b^{(1)}\right) \\ & + b_{\sigma}^{(1,t)}. \end{aligned} \tag{3.4}$$

At the $\ell^{\text{th}} > 1$ layer, the hidden state is updated using the hidden state of the previous time step at layer ℓ, the hidden state of the current time step at layer $\ell - 1$ and the input at the current time step

as

$$\begin{aligned}\mathbf{z}_{\text{RNN}}^{(\ell,t)} =& A_{\sigma}^{(\ell,t)}\left(W_{\text{in}}^{(\ell)}x^{t} + W_{\text{rec}}^{(\ell)}\mathbf{z}_{\text{RNN}}^{(\ell,t-1)} + W_{\text{up}}^{(\ell)}\mathbf{z}_{\text{RNN}}^{(\ell-1,t)} + b^{(\ell)}\right)\\ &\cdot\left(W_{\text{in}}^{(\ell)}x^{t} + W_{\text{rec}}^{(\ell)}\mathbf{z}_{\text{RNN}}^{(\ell,t-1)} + W_{\text{up}}^{(\ell)}\mathbf{z}_{\text{RNN}}^{(\ell-1,t)} + b^{(\ell)}\right)\\ &+ b_{\sigma}^{(\ell,t)}. \end{aligned} \tag{3.5}$$

One can unroll (3.4) and (3.5) to obtain the input-output formulation of the input sequence $\left(x^{1},\ldots,x^{T}\right)$.

Transformers

3.5 *References*

Section 3.2, Section 3.3, Section 3.4, Section 3.4 and Section 3.4 from Balestriero and Baraniuk (2018).

4 Properties of Continuous Piecewise Linear Deep Network

4.1 Introduction

With the understanding that deep networks are continuous piecewise linear affine operators, we can translate problems regarding deep networks into the language of spline theory. In this chapter we start to answer some of those questions. In particular, we consider the convexity of deep networks, and explore the necessary conditions for deep networks to be globally convex. We look at deriving bounds for the Lipschitz constant of deep networks. Then we use spline theory to explore the approximation capacity of deep networks, and thus we arrive at a universality result for two-layer deep networks. Furthermore, we interpret the spline formulation of deep networks as a form of template matching.

4.2 Convexity

Proposition 4.2.1. *A deep network layer is non-decreasing with respect to each of its output dimensions if and only if its slope parameters are non-negative.*

Proof. Without loss of generality consider a deep network layer $f^{(\ell)}$ with $d^{(\ell-1)} = 1$ and $d^{(\ell)}$. From Proposition 3.2.9, we can write $f^{(\ell)} = S\left[A^{(\ell)}, B^{(\ell)}\right]$ for $A^{(\ell)} \in \mathbb{R}^r$ and $B^{(\ell)} \in \mathbb{R}$.
($\Rightarrow$). Since $f^{(\ell)}$ is non-decreasing we have

$$\max_{j=1,\ldots,r}\left(\left[A^{(\ell)}\right]_j x + B^{(\ell)}\right) \geq \max_{j=1,\ldots,r}\left(\left[A^{(\ell)}\right]_j y + B^{(\ell)}\right)$$

for any $x \geq y$. Equivalently,

$$\max_{j=1,\ldots,r}\left(\left[A^{(\ell)}\right]_j x\right) \geq \max_{j=1,\ldots,r}\left(\left[A^{(\ell)}\right]_j y\right)$$

for any $x \geq y$. Choosing $x = 1$ and $y = 0$ it follows that

$$\max_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j \right) \geq 0. \tag{4.1}$$

Similarly, choosing $x = 0$ and $y = -1$, it follows that

$$0 \geq \max_{j=1,\ldots,r} \left(- \left[A^{(\ell)} \right]_j \right),$$

which is equivalent to

$$\min_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j \right) \geq 0. \tag{4.2}$$

Combining (4.1) and (4.2), it follows that $\left[A^{(\ell)} \right]_j \geq 0$ for every $j = 1, \ldots, r$.
($\Leftarrow$). Let $x \leq y$. Then since $\left[A^{(\ell)} \right]_j \geq 0$ for each $j = 1, \ldots, r$, we have

$$\left[A^{(\ell)} \right]_j x \geq \left[A^{(\ell)} \right]_j y$$

for every $j = 1, \ldots, r$. Therefore,

$$\begin{aligned}
S\left[A^{(\ell)}, B^{(\ell)}\right](x) - S\left[A^{(\ell)}, B^{(\ell)}\right](y) \\
&= \max_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j x + B^{(\ell)} \right) - \max_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j y + B^{(\ell)} \right) \\
&= \max_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j x \right) - \max_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j y \right) \\
&\geq \max_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j x \right) - \max_{j=1,\ldots,r} \left(\left[A^{(\ell)} \right]_j x \right) \\
&= 0.
\end{aligned}$$

That is,

$$S\left[A^{(\ell)}, B^{(\ell)}\right](x) \geq S\left[A^{(\ell)}, B^{(\ell)}\right],$$

meaning that S, and thus $f^{(\ell)}$, is non-decreasing. □

Theorem 4.2.2. *An L-layer deep network whose layers $\ell = 2, \ldots, L$ are piecewise affine, convex and non-decreasing with respect to each of their output dimensions is a max-affine spline operator, and hence convex.*

Proof. We proceed by induction on L.

- Consider the case $L = 2$. For simplicity, let $d^{(2)} = 1$. The case of $d^{(2)} > 1$ follows similarly since a max-affine spline operator is just the stacking of $d^{(2)}$ max-affine spline units. From Proposition 3.2.9, we have $f^{(1)}(\mathbf{x}) = S\left[A^{(1)}, B^{(1)}\right]$ for $A \in \mathbb{R}^{d^{(1)} \times r^{(1)} \times d}$,

$B^{(1)} \in \mathbb{R}^{r^{(1)} \times d}$ and $f^{(2)}(\mathbf{x}) = S\left[A^{(2)}, B^{(2)}\right]$ for $A \in \mathbb{R}^{1, r^{(2)}, d^{(1)}}$, $B^{(2)} \in \mathbb{R}^{r^{(2)}}$. Hence,

$$
\begin{aligned}
&\left(f^{(2)} \circ f^{(1)}\right)(\mathbf{x}) \\
&\quad = \left(S\left[A^{(2)}, B^{(2)}\right] \circ S\left[A^{(1)}, B^{(1)}\right]\right)(\mathbf{x}) \\
&\quad = S\left[A^{(2)}, B^{(2)}\right]\left(\begin{pmatrix} \max_{j=1,\dots,r^{(1)}}\left(\left\langle \left[A^{(1)}\right]_{1,j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_{1,j}\right) \\ \vdots \\ \max_{j=1,\dots,r^{(1)}}\left(\left\langle \left[A^{(1)}\right]_{d^{(1)},j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_{d^{(1)},j}\right) \end{pmatrix}\right) \\
&\quad = \max_{j'=1,\dots,r^{(2)}}\left(\left(\left[A^{(2)}\right]_{1,j',\cdot}\right)^{\top}\begin{pmatrix} \max_{j=1,\dots,r^{(1)}}\left(\left\langle \left[A^{(1)}\right]_{1,j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_{1,j}\right) \\ \vdots \\ \max_{j=1,\dots,r^{(1)}}\left(\left\langle \left[A^{(1)}\right]_{d^{(1)},j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_{d^{(1)},j}\right) \end{pmatrix} + \left[B^{(2)}\right]_{j'}\right) \\
&\quad = \max_{j'=1,\dots,r^{(2)}}\left(\sum_{i=1}^{d^{(1)}}\left[A^{(2)}\right]_{1,j',i}\max_{j=1,\dots,r^{(1)}}\left(\left\langle \left[A^{(1)}\right]_{i,j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_{i,j}\right) + \left[B^{(2)}\right]_{j'}\right) \\
&\quad \overset{(1)}{=} \max_{j'=1,\dots,r^{(2)}}\max_{j=1,\dots,r^{(1)}}\left(\sum_{i=1}^{d^{(1)}}\left[A^{(2)}\right]_{1,j',i}\left(\left\langle \left[A^{(1)}\right]_{i,j,\cdot}, \mathbf{x}\right\rangle + \left[B^{(1)}\right]_{i,j}\right) + \left[B^{(2)}\right]_{j'}\right) \\
&\quad = S\left[A^{(1,2)}, B^{(1,2)}\right],
\end{aligned}
$$

where $A^{(1,2)} \in \mathbb{R}^{1 \times r^{(2)} r^{(1)} \times d}$ and $B^{(1,2)} \in \mathbb{R}^{r^{(2)} r^{(1)}}$ with

$$
\begin{cases}
\left[A^{(1,2)}\right]_{1,j,\cdot} = \sum_{i=1}^{d^{(\ell)}}\left[A^{(2)}\right]_{1,p+1,i}\left[A^{(1)}\right]_{i,q,\cdot} & j = pr^{(1)} + q \text{ for } p = 0, \dots, r^{(2)} - 1 \text{ and } q \in \left\{1, \dots, r^{(1)}\right\} \\
\left[B^{(1,2)}\right] = \left[B^{(2)}\right]_{q} + \sum_{i=1}^{d^{(\ell)}}\left[B^{(1)}\right]_{i,p} & j = pr^{(1)} + q \text{ for } p = 0, \dots, r^{(2)} - 1 \text{ and } q \in \left\{1, \dots, r^{(1)}\right\}.
\end{cases}
$$

In (1) we have used the non-decreasing property of the second layer to infer $\left[A^{(2)}\right]_{1,j',i} \geq 0$ by Proposition 4.2.1, allowing us to move it into the maximum operation.

- The inductive step follows the same steps as the base case, with layers one and two replaced with layers $\ell - 1$ and ℓ, where we are assuming the result for layer $\ell - 1$.

□

4.3 *Lipschitz*

the results in the paper may be wrong, so have updated here

Proposition 4.3.1. *Let $f^{(\ell)} : \mathbb{R}^{d^{(\ell-1)}} \to \mathbb{R}^{d^{(\ell)}}$ be a deep network operator.*

1. *If $f^{(\ell)}$ is a fully connected operator, then its Lipschitz constant $\kappa_{fc}^{(\ell)}$ is such that $\kappa_{fc}^{(\ell)} \leq \left\| W_{fc}^{(\ell)} \right\|$.*

2. *If $f^{(\ell)}$ is a convolutional operator, then its Lipschitz constant $\kappa_{conv}^{(\ell)}$ is such that $\kappa_{conv}^{(\ell)} \leq \left\| \mathbf{C}_{conv}^{(\ell)} \right\|$.*

3. *If $f^{(\ell)}$ is the ReLU, and the absolute value activation operator, then its Lipschitz constant $\kappa_{\sigma}^{(\ell)} \leq \sqrt{d^{(\ell)}}$.*

4. *If $f^{(\ell)}$ is the leaky ReLU activation operator with parameter $\eta > 0$, then its Lipschitz constant $\kappa_{\sigma}^{(\ell)} \leq \sqrt{d^{(\ell)} \max\left(|\eta|, 1\right)}$.*

5. *If $f^{(\ell)}$ is the max-pooling or mean-pooling operator, then its Lipschitz constant $\kappa_{\rho_{\max}}^{(\ell)}$ is such that $\kappa_{\rho_{\max}}^{(\ell)} \leq \sqrt{d^{(\ell)}}$.*

Proof.

1. This follows directly from

$$\begin{aligned}\left\| f^{(\ell)}\left(\mathbf{z}_1\right) - f^{(\ell)}\left(\mathbf{z}_2\right) \right\|^2 &= \left\| W_{\text{fc}}^{(\ell)}\left(\mathbf{z}_1 - \mathbf{z}_2\right) \right\|^2 \\ &\leq \left\| W_{\text{fc}}^{(\ell)} \right\|^2 \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 .\end{aligned}$$

2. This follows directly from

$$\begin{aligned}\left\| f^{(\ell)}\left(\mathbf{z}_1\right) - f^{(\ell)}\left(\mathbf{z}_2\right) \right\|^2 &= \left\| \mathbf{C}_{\text{fc}}^{(\ell)}\left(\mathbf{z}_1 - \mathbf{z}_2\right) \right\|^2 \\ &\leq \left\| \mathbf{C}_{\text{fc}}^{(\ell)} \right\|^2 \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 .\end{aligned}$$

3. This follows from utilising statements 1 and 3 of Example 3.2.4 and Proposition 2.5.4 since

$$\begin{aligned}\left\| f^{(\ell)}\left(\mathbf{z}_1\right) - f^{(\ell)}\left(\mathbf{z}_2\right) \right\|^2 &\leq \left(\sum_{i=1}^{d^{(\ell)}} \left(\max_{j=1,2} \left\| \left[A_\sigma\right]_{i,j,\cdot} \right\| \right)^2 \right) \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 \\ &= \left(\sum_{j=1}^{d^{(\ell)}} \left\| \mathbf{e}_i \right\|^2 \right) \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 \\ &\leq d^{(\ell)} \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 .\end{aligned}$$

4. This follows from utilising statement 2 of Example 3.2.4 and Proposition 2.5.4 since

$$\begin{aligned}\left\| f^{(\ell)}\left(\mathbf{z}_1\right) - f^{(\ell)}\left(\mathbf{z}_2\right) \right\|^2 &\leq \left(\sum_{i=1}^{d^{(\ell)}} \left(\max_{j=1,2} \left\| \left[A_\sigma\right]_{i,j,\cdot} \right\| \right)^2 \right) \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 \\ &= \left(\sum_{j=1}^{d^{(\ell)}} \max\left(|\eta| \left\| \mathbf{e}_i \right\|, \left\| \mathbf{e}_i \right\|\right)^2 \right) \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 \\ &\leq d^{(\ell)} \max\left(|\eta|, 1\right) \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 .\end{aligned}$$

5. This follows from utilising Example 3.2.6 and Proposition 2.5.4 since

$$\left\| f^{(\ell)}\left(\mathbf{z}_1\right) - f^{(\ell)}\left(\mathbf{z}_2\right) \right\|^2 \leq \left(\sum_{i=1}^{d^{(\ell)}} \left(\max_{j=1,\ldots,r} \left\| \left[A_\sigma\right]_{i,j,\cdot} \right\| \right)^2 \right) \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2$$
$$\leq d^{(\ell)} \left\| \mathbf{z}_1 - \mathbf{z}_2 \right\|^2 .$$

□

Proposition 4.3.2. *Let $f_\theta : \mathbb{R}^d \to \mathbb{R}^{d^{(L)}}$ be a deep network constructed from the composition of L max-affine spline operators. Then a Lipschitz constant κ is given by*

$$\kappa = \prod_{\ell=1}^{L} \kappa^{(\ell)},$$

where each $\kappa^{(\ell)}$ is as given by Proposition 4.3.1 depending on the topology of the ℓth layer.

Proof. This follows from the more general statement given in Lemma 4.3.3.

Lemma 4.3.3. *Let $f_1 : \mathbb{R}^d \to \mathbb{R}^{(1)}$ and $f_2 : \mathbb{R}^{d^{(1)}} \to \mathbb{R}^{d^{(2)}}$ be functions with Lipschitz constants of κ_1 and κ_2 respectively. Then $\kappa_2\kappa_1$ is a Lipschitz constant of $f_1 \circ f_2$.*

Proof. For $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^d$ observe that

$$\left\| \left(f_2 \circ f_1\right)\left(\mathbf{x}_1\right) - \left(f_2 \circ f_1\right)\left(\mathbf{x}_2\right) \right\|^2 \leq \kappa_2^2 \left\| f_1\left(\mathbf{x}_1\right) - f_1\left(\mathbf{x}_2\right) \right\|^2$$
$$\leq \kappa_2^2 \kappa_1^2 \left\| \mathbf{x}_1 - \mathbf{x}_2 \right\|^2 .$$

□

□

4.4 Universality

Although we have been considering deep networks with piecewise affine and convex operators, these are sufficient from an approximation perspective.

Theorem 4.4.1. *Let $f : \mathbb{R}^d \to \mathbb{R}^c$ be a continuous operator, and let f_θ be a deep network with parameters θ constructed from the composition of two max-affine spline operators with $r^{(1)} = 2$ and $r^{(2)} = 1$. Then there exists parameters θ such that*

$$\left\| f(\mathbf{x}) - f_\theta(\mathbf{x}) \right\|_2^2 \leq \frac{\sum_{j=1}^{c} \eta_j}{d^{(1)}},$$

where η_j characterises the regularity of the jth output of f.

Proof. Due to the independence the max-affine spline units constituting the layer operators, we can consider the approximation in each dimension independently. Observe that

$$\begin{aligned}[f_\theta(\mathbf{x})]_j &= \left[W_{\text{fc}}^{(2)} f_\theta^{(1)}(\mathbf{x}) + b_{\text{fc}}^{(2)}\right]_j \\ &= \sum_{i=1}^{d^{(1)}} \left[W_{\text{fc}}^{(2)}\right]_{j,i} \left[f_\theta^{(1)}(\mathbf{x})\right]_i + \left[b_{\text{fc}}^{(2)}\right]_j .\end{aligned}$$

Since $r^{(1)} = 2$, it follows that $[f_\theta(\mathbf{x})]_j$ is the sum of $d^{(1)}$ hinge functions, where we are assuming the terminology of Breiman (1993). Using Theorem 3 from Breiman (1993), there exists parameters such that

$$\left|[f(\mathbf{x})]_j - [f_\theta(\mathbf{x})]_i\right|^2 \leq \frac{\eta_i}{d^{(1)}}.$$

Therefore,

$$\begin{aligned}\|f(\mathbf{x}) - f_\theta(\mathbf{x})\|_2^2 &= \sum_{j=1}^{c} \left|[f(\mathbf{x})]_j - [f_\theta(\mathbf{x})]_j\right|^2 \\ &\leq \frac{\sum_{j=1}^{c} \eta_j}{d^{(1)}}.\end{aligned}$$

□

generalise this to deeper networks

Corollary 4.4.2. *A deep network containing non-piecewise affine and/or non-convex operators can be approximated arbitrarily closely by a max-affine spline operator based deep network.*

4.5 Template Matchers

Filter Template Matching

Recall (3.2), where we collapsed the convolutional neural network operation, allowing for the interpretation that the operation is a signal-dependent affine featurization of a signal $\mathbf{x}$ into linear extractable features. More generally, we consider[1]

$$\mathbf{z}^{(L)}(\mathbf{x}) = \left(W_{\text{fc}}^{(L)} A[\mathbf{x}]\right)\mathbf{x} + \left(W^{(L)} B[\mathbf{x}] + b_{\text{fc}}^{(L)}\right). \tag{4.3}$$

[1] Recall the notation convention from statement 2 of Remark 2.2.2.

From here we gain the alternative interpretation that the output of a deep network is a bank of linear matched filters with a set of biases. More specifically, given an input $\mathbf{x}$, the chosen filter, or class, is given by the index of the largest element of $\mathbf{z}^{(L)}(\mathbf{x})$.

Accounting for the intermediate layer computations of the deep networks, leads to the more fine-grained interpretation that deep

networks are performing hierarchical template matching,

$$\mathbf{z}^{(L)}(\mathbf{x}) = W_{\text{fc}}^{(L)} \max_{j_{(L-1)}=1,\dots,r^{(L-1)}} \left(A^{(L-1)}_{\cdot,r^{(L-1)},\cdot} \dots A^{(2)}_{\cdot,j_{(2)},\cdot} \right.$$
$$\left. \cdot \max_{j=1,\dots,r^{(1)}} \left(\left[A^{(1)}\right]_{\cdot,j_{(1)},\cdot} \mathbf{x} + \left[B^{(1)}\right]_{j_(1),\cdot} \right) + \dots + \left[B^{(L-1)}\right]_{j_{(L-1)},\cdot} \right) + b_{\text{fc}}^{(L)}. \tag{4.4}$$

More specifically, we observe that the template matching is greedy, which is a computationally efficient yet sub-optimal template matching technique. It is only optimal when the deep network is convex.

Corollary 4.5.1. *For a deep network satisfying the conditions of Theorem 4.2.2, the computation (4.4) collapses to*

$$\mathbf{z}^{(L)}(\mathbf{x}) = W_{fc}^{(L)} \max_{j_{(L-1)}=1,\dots,r^{(L-1)};\dots;j_{(1)}=1,\dots,r^{(1)}} \left(A^{(L-1)} \left(\dots A^{(2)}_{j_{(2)}} \left(A^{(1)}_{j_{(1)}} \mathbf{x} + B^{(1)}_{j_{(1)}} \right) + B^{(2)}_{j_{(2)}} \right) + \dots + B^{(L-1)}_{j_{(L-1)}} \right) + b_{fc}^{(L)},$$

which is globally optimal template matching.

Filter Template versus Bias

A large positive or negative bias – at any intermediate layer in (4.4) – can dominate the filter match computations. This can either be induced by poor initialisation schemes or be as a consequence of the difference in batch norm implementation during training and during inference.

Collinear Templates

In the matched filterbank interpretation (4.3), the optimal template for input $\mathbf{x}$ of class c is a scaled version of $\mathbf{x}$. In a particular setting, the optimal templates for the other incorrect classes is provided by Proposition 4.5.2.

Proposition 4.5.2. *Let $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{i=1}^n$ be a training dataset with c classes and $\|\mathbf{x}_j\|_2 = 1$ for $j = 1, \dots, n$. Consider a deep network f_θ with sufficient approximation power to span arbitrary slope matrix in (4.3) for any input $\mathbf{x}_j$ with $j = 1, \dots, n$. Suppose the deep network is trained using the cross-entropy loss $\mathcal{L}_{CE} : \mathbb{R} \times \mathbb{R} \to \mathbb{R}_{\geq 0}$ with a regularisation constrain of $\sum_{i=1}^c \left\| A\left[\mathbf{x}_n\right]_{i,\cdot} \right\|_2 < \alpha$ for some $\alpha > 0$. At the global minimum of the optimization problem $A^\star$, the optimal templates take the form*

$$\left[A^\star\left[\mathbf{x}_j\right]\right]_{i,\cdot} = \begin{cases} \sqrt{\frac{(c-1)\alpha}{c}}\mathbf{x}_j & i = y_j \\ -\sqrt{\frac{\alpha}{c(c-1)}}\mathbf{x}_j & i \neq y_j. \end{cases}$$

Proof. The loss function is convex on a convex set, thus it is sufficient to find an extremum point to solve the optimization problem. The augmented loss with the Lagrange multiple is

$$l\left(\left[A[\mathbf{x}_j]\right]_{1,\cdot},\cdot,\left[A[\mathbf{x}_j]\right]_{c,\cdot},\lambda\right) = -\left\langle \left[A[\mathbf{x}_j]\right]_{y_j,\cdot},\mathbf{x}_j\right\rangle + \log\left(\sum_{i=1}^{c}\exp\left(\left\langle\left[A[\mathbf{x}_j]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)\right) - \lambda\left(\sum_{i=1}^{c}\left\|\left[A[\mathbf{x}_j]\right]_{i,\cdot}\right\|^2-\alpha\right).$$

The sufficient Karush–Kuhn–Tucker conditions are thus

$$\begin{cases} \frac{\partial l}{\partial\left[A[\mathbf{x}_j]\right]_{1,\cdot}} = -\mathbf{1}_{\{y_j=1\}}\mathbf{x}_j + \frac{\exp\left(\left\langle\left[A[\mathbf{x}_j]\right]_{1,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{i=1}^{c}\exp\left(\left\langle\left[A[\mathbf{x}_j]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)} - 2\lambda\left[A\left[\mathbf{x}_j\right]\right]_{1,\cdot} = \mathbf{0} \\ \vdots \\ \frac{\partial l}{\partial\left[A[\mathbf{x}_j]\right]_{c,\cdot}} = -\mathbf{1}_{\{y_j=c\}}\mathbf{x}_j + \frac{\exp\left(\left\langle\left[A[\mathbf{x}_j]\right]_{c,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{i=1}^{c}\left\langle\exp\left(\left[A[\mathbf{x}_j]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)} - 2\lambda\left[A\left[\mathbf{x}_j\right]\right]_{1,\cdot} = \mathbf{0} \\ \frac{\partial l}{\partial\lambda} = \alpha - \sum_{i=1}^{c}\left\|\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}\right\|^2 = 0. \end{cases}$$

From the first c conditions it follows that

$$\begin{aligned} 0 &= \sum_{i=1}^{c}\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}^{\top}\frac{\partial l}{\partial\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}} \\ &= -\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{y_j,\cdot},\mathbf{x}_j\right\rangle + \sum_{i=1}^{c}\frac{\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{t=1}^{c}\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot},\mathbf{x}_j\right\rangle\right)}\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle - 2\lambda\sum_{i=1}^{c}\left\|\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}\right\|^2. \end{aligned}$$

Therefore, using the $(c+1)^{\text{th}}$ condition it follows that

$$\lambda = \frac{1}{2\alpha}\left(\sum_{i=1}^{c}\frac{\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{t=1}^{c}\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot},\mathbf{x}_j\right\rangle\right)}\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle - \left\langle\left[A\left[\mathbf{x}_j\right]\right]_{y_j,\cdot},\mathbf{x}_j\right\rangle\right).$$

Substituting this into the kth condition for $k\in\{1,\ldots,c\}$ we obtain

$$\begin{aligned} \frac{\partial l}{\partial\left[A\left[\mathbf{x}_j\right]\right]_{k,\cdot}} &= -\mathbf{1}_{\{y_j=k\}}\mathbf{x}_j + \frac{\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{k,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{i=1}^{c}\left\langle\exp\left(\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)} \\ &\quad - \frac{1}{\alpha}\sum_{i=1}^{c}\frac{\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{t=1}^{c}\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot},\mathbf{x}_j\right\rangle\right)}\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\left[A\left[\mathbf{x}_j\right]\right]_{k,\cdot}. \end{aligned}$$

Noting that $\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot} = \left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot}$ for $i,t\neq y_j$, we can simplify this to

$$\begin{aligned} \frac{\partial l}{\partial\left[A\left[\mathbf{x}_j\right]\right]_{k,\cdot}} &= \left(\frac{\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{k,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{t=1}^{c}\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot},\mathbf{x}_j\right\rangle\right)} - \mathbf{1}_{\{k=y_j\}}\right)\mathbf{x}_j \\ &\quad + \frac{c-1}{\alpha}\frac{\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\right)}{\sum_{t=1}^{c}\exp\left(\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot},\mathbf{x}_j\right\rangle\right)}\left\langle\left[A\left[\mathbf{x}_j\right]\right]_{y_j,\cdot}-\left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot},\mathbf{x}_j\right\rangle\left[A\left[\mathbf{x}_j\right]\right]_{k,\cdot}, \end{aligned}$$

where $i \neq y_j$. We can now substitute in the proposed global optimum $A^\star$ to confirm that it is the unique extremum point which is the global optimum. Maintaining that $i \neq y_j$, in the case $k = y_j$ we have

$$\begin{aligned}
\frac{\partial l}{\partial \left[A\left[\mathbf{x}_j\right]\right]_{y_j,\cdot}} &= \frac{c-1}{\alpha} \frac{\exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}, \mathbf{x}_j \right\rangle\right)}{\sum_{t=1}^{c} \exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot}, \mathbf{x}_j \right\rangle\right)} \left(-\alpha \mathbf{x}_j + \left\langle \left[A\left[\mathbf{x}_j\right]\right]_{y_j,\cdot} - \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}, \mathbf{x}_j \right\rangle \left[A\left[\mathbf{x}_j\right]\right]_{y_j,\cdot}\right) \\
&= \frac{c-1}{\alpha} \frac{\exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}, \mathbf{x}_j \right\rangle\right)}{\sum_{t=1}^{c} \exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot}, \mathbf{x}_j \right\rangle\right)} \left(-\alpha \mathbf{x}_j + \alpha \left\|\mathbf{x}_j\right\|^2 \mathbf{x}_j\right) \\
&= 0.
\end{aligned}$$

In the case $k \neq y_j$ we have

$$\begin{aligned}
\frac{\partial l}{\partial \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}} &= \frac{\exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}, \mathbf{x}_j \right\rangle\right)}{\sum_{t=1}^{c} \exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot}, \mathbf{x}_j \right\rangle\right)} \left(\mathbf{x}_j + \frac{c-1}{\alpha} \left\langle \left[A\left[\mathbf{x}_j\right]\right]_{y_j,\cdot} - \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}, \mathbf{x}_j \right\rangle \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}\right) \\
&= \frac{\exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{i,\cdot}, \mathbf{x}_j \right\rangle\right)}{\sum_{t=1}^{c} \exp\left(\left\langle \left[A\left[\mathbf{x}_j\right]\right]_{t,\cdot}, \mathbf{x}_j \right\rangle\right)} \left(\mathbf{x}_j - \left\|\mathbf{x}_j\right\|^2 \mathbf{x}_j\right) \\
&= 0.
\end{aligned}$$

Even though we have only considered $\mathbf{x}_j$ in the above analysis, minimising the per-example loss also minimises the dataset loss. □

Remark 4.5.3. *Proposition 4.5.2 shows that an idealized deep network will memorize a set of collinear templates from the training set.*

Orthogonal Templates

The signal-dependent matched filterbank computation of a deep network (4.3), is optimal for classifying signals immersed in additive white Gaussian noise. However, in practice, noise will be introduced from nuanced sources, affecting the optimality of this approach. Using orthogonal templates is a method of addressing this issue.

For the templates of a deep network with max-affine spline operator layers to be orthogonal, it is necessary that the rows of the $W_{\text{fc}}^{(L)}$ are orthogonal. To promote this property during training one can add a regularisation term to the cross-entropy loss function $\mathcal{L}_{\text{CE}}$. More specifically, during training one can use the loss function

$$\mathcal{L}_{\text{CE}} + \lambda \sum_{i_1 \neq i_2} \left| \left\langle \left[W_{\text{fc}}^{(L)}\right]_{i_1,\cdot}, \left[W_{\text{fc}}^{(L)}\right]_{i_2,\cdot} \right\rangle \right|^2.$$

4.6 Input Space Partitioning

Max-Affine Spline Partition

The max-affine spline operator at layer ℓ of a deep network directly partitions its input $\mathbb{R}^{d^{(\ell-1)}}$ as $\Omega^{(\ell)}$ constituting $r^{(\ell)}$ regions. More specifically, each unit of the operator induces a partition on $\mathbb{R}^{d^{(\ell)}}$. Let $\omega_{i,j}^{(\ell)}$ denote the jth region, out of $r_i^{(\ell)}$, induced by the ith unit of the max-affine spline operator at layer ℓ of the deep network. Then

$$\Omega^{(\ell)} = \bigcup_{j_1=1}^{r_1^{(\ell)}} \cdots \bigcup_{j_{d^{(\ell)}}}^{r_{d^{(\ell)}}^{(\ell)}} \bigcap_{i=1}^{d^{(\ell)}} \omega_{i,j_i}^{(\ell)}.$$

Example 4.6.1.

1. *For the leaky ReLU activation operator, recall that* $r_i^{(\ell)} = 2$ *for each* $i = 1, \ldots, d^{(\ell)}$. *In particular, we have*

$$\begin{cases} \omega_{i,1}^{(\ell)} = \left\{ \mathbf{z}^{(\ell-1)}(\mathbf{x}) \in \mathbb{R}^{d^{(\ell-1)}} : \left[\sigma_{LReLU}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x}) \right) \right]_i \leq 0 \right\} \\ \omega_{i,2}^{(\ell)} = \left\{ \mathbf{z}^{(\ell-1)}(\mathbf{x}) \in \mathbb{R}^{d^{(\ell-1)}} : \left[\sigma_{LReLU}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x}) \right) \right]_i > 0 \right\} \end{cases}$$

   for $i = 1, \ldots, d^{(\ell)}$.

2. *For the max-pooling operator, we have*

$$\begin{cases} \omega_{i,1}^{(\ell)} = \left\{ \mathbf{z}^{(\ell-1)}(\mathbf{x}) \in \mathbb{R}^{d^{(\ell-1)}} : \operatorname{argmax}_{t \in \mathcal{R}_i^{(\ell)}} \left[\mathbf{z}^{(\ell-1)}(\mathbf{x}) \right]_t = 1 \right\} \\ \vdots \\ \omega_{i,\left|\mathcal{R}_i^{(\ell)}\right|} = \left\{ \mathbf{z}^{(\ell-1)}(\mathbf{x}) \in \mathbb{R}^{d^{(\ell)}} : \operatorname{argmax}_{t \in \mathcal{R}_i^{(\ell)}} \left[\mathbf{z}^{(\ell-1}(\mathbf{x}) \right]_t = \left|\mathcal{R}_i^{(\ell)}\right| \right\} \end{cases}$$

   for $i = 1, \ldots, d^{(\ell)}$.

Generally speaking, for the max-affine spline operator $S\left[A^{(\ell)}, B^{(\ell)}\right] : \mathbb{R}^{d^{(\ell-1)}} \to \mathbb{R}^{d^{(\ell)}}$ we have

$$\omega_{i,j}^{(\ell)} = \left\{ \mathbf{z}^{(\ell-1)} \in \mathbb{R}^{d^{(\ell-1)}} : \operatorname*{argmax}_{t=1,\ldots,r_i^{(\ell)}} \left[S\left[\mathbf{z}^{(\ell-1)} \right] \right]_i = j \right\}.$$

Let $T_H^{(\ell)} \in \mathbb{R}^{d^{(\ell)} \times r^{(\ell)}}$ be given by

$$\left[T_H^{(\ell)} \mathbf{z}^{(\ell-1)}(\mathbf{x}) \right]_{i,j} = \frac{\partial}{\partial \left(\left\langle \left[A^{(\ell)} \right]_{i,j}, \mathbf{z}^{(\ell-1)}(\mathbf{x}) \right\rangle + \left[B^{(\ell)} \right]_{i,j} \right)} \left(\max_{t=1,\ldots,r^{(\ell)}} \left(\left\langle \left[A^{(\ell)} \right]_{i,t}, \mathbf{z}^{(\ell-1)}(\mathbf{x}) \right\rangle + \left[B^{(\ell)} \right]_{i,t} \right) \right).$$

Proposition 4.6.2. *We have*

$$\omega_{i,j}^{(\ell)} = \left\{ \mathbf{z}^{(\ell-1)}(\mathbf{x}) \in \mathbb{R}^{d^{(\ell-1)}} : \left[T_H^{(\ell)} \mathbf{z}^{(\ell-1)}(\mathbf{x}) \right]_{i,j} = 1 \right\}.$$

Deep Network Partition

The partition on the input domain $\mathbb{R}^d$, due to the ℓth is formed by pulling back the partition $\Omega^{(\ell)}$ onto $\mathbb{R}^d$. In particular,

$$\Omega^{(1\leftarrow\ell)} = \bigcup_{j_1=1}^{r_1^{(\ell)}} \cdots \bigcup_{j_{d^{(\ell)}}}^{r_{d^{(\ell)}}} \bigcap_{i=1}^{d^{(\ell)}} \omega_{i,j_i}^{(1\leftarrow\ell)},$$

where

$$\omega_{i,j_i}^{(1\leftarrow\ell)} = \left\{ \mathbf{x} \in \mathbb{R}^d : \left[T_H^{(\ell)} \left(\mathbf{z}^{(\ell-1)}(\mathbf{x}) \right) \right]_{i,j_i} = 1 \right\}.$$

Therefore, the partition induced by the entire deep network on the input space Ω is given by

$$\Omega = \bigcap_{\ell'=1}^{L} \Omega^{(1\leftarrow\ell')}.$$

This characterisation of regions in the underlying partition hints at a simplified notation. More specifically, there is a one-to-one correspondence between regions in $\Omega^{(\ell)}$ and vectors

$$\mathbf{t}^{(\ell)} \in \left\{1, \ldots, r_1^{(\ell)}\right\} \times \cdots \times \left\{1, \ldots, r_{d^{(\ell)}}^{(\ell)}\right\},$$

so that

$$\begin{aligned} \Omega^{(\ell)} &= \bigcup_{\mathbf{t}^{(\ell)}} \bigcap_{j=1}^{d^{(\ell)}} \omega_{i,\left[\mathbf{t}^{(\ell)}\right]_i}^{(\ell)} \\ &=: \bigcup_{\mathbf{t}^{(\ell)}} \omega_{\mathbf{t}^{(\ell)}}^{(\ell)} \end{aligned}$$

Consequently, it follows that $\Omega^{(\ell)}$ can have at most $\prod_{i=1}^{d^{(\ell)}} r_i^{(\ell)}$ regions. We can extend this naturally to the deep network partition as

$$\Omega = \bigcup_{\mathbf{t}=\left[\mathbf{t}^{(1)},\ldots,\mathbf{t}^{(L)}\right]} \omega_{\mathbf{t}},$$

where

$$\omega_{\mathbf{t}} = \bigcap_{\ell=1}^{L} \omega_{\mathbf{t}^{(\ell)}}^{(1\leftarrow\ell)}.$$

For deep networks implementing exclusively ReLU, leaky ReLU or absolute value activations operators, we adopt a slightly altered convention to streamline future computational implementations. Namely, we characterise regions with $\mathbf{s} = \left[\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)}\right]$ where $\mathbf{s}^{(\ell)} \in \{-1, 1\}^{d^{(\ell)}}$ so that

let

$$\Omega^{(\ell)} = \bigcup_{\mathbf{s}^{(\ell)} \in \{-1,1\}^{d^{(\ell)}}} \bigcap_{j=1}^{d^{(\ell)}} \left\{ \mathbf{z}^{(\ell-1)} \in \mathbb{R}^{d^{(\ell-1)}} : \left(\left\langle \left[W^{(\ell)} \right]_{j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[b^{(\ell)} \right]_j \right) \mathbf{s}_j^{d^{(\ell)}} \leq 0 \right\}$$
$$=: \bigcup_{\mathbf{s}^{(\ell)} \in \{-1,1\}^{d^{(\ell)}}} \omega^{(\ell)}_{\mathbf{s}^{(\ell)}}, \tag{4.5}$$

and

$$\Omega = \bigcup_{\mathbf{s} = \left[\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)}\right]} \omega_{\mathbf{s}}$$

as before.

Remark 4.6.3. *For* $\mathbf{z}^{(\ell-1)} \in \mathbb{R}^{d^{(\ell)}}$*, we'll let* $\mathbf{t}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}\right)$ *be such that* $\mathbf{z}^{(\ell-1)} \in \omega^{(\ell)}_{\mathbf{t}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}\right)}$*. Similarly, for* $\mathbf{x} \in \mathbb{R}^d$ *we'll let* $\mathbf{t}(\mathbf{x}) = \left[\mathbf{t}^{(1)}(\mathbf{x}), \ldots, \mathbf{t}^{(L)}\left(\mathbf{z}^{(L-1)}\right)\right]$.

Functional Decomposition

By construction a deep network is an affine transformation on the regions of Ω, thus we can use these regions to decompose the function of a deep network into a series of affine transformation.

Let f be a deep network. The ℓth layer of the network can be written as a max-affine spline operator $S\left[A^{(\ell)}, B^{(\ell)}\right]$. In particular,

$$\begin{aligned} f^{(\ell)}\left(\mathbf{z}^{(\ell-1)}\right) &= S\left[A^{(\ell)}, B^{(\ell)}\right]\left(\mathbf{z}^{(\ell-1)}\right) \\ &= \sum_{\omega^{(\ell)} \in \Omega^{(\ell)}} \left(A^{(\ell)}_{\omega^{(\ell)}} \mathbf{z}^{(\ell-1)} + b^{(\ell)}_{\omega^{(\ell)}} \right) \mathbf{1}_{\left\{\mathbf{z}^{(\ell-1)} \in \omega^{(\ell)}\right\}} \\ &= \sum_{\mathbf{t}^{(\ell)}} \left(A^{(\ell)}_{\omega^{(\ell)}_{\mathbf{t}^{(\ell)}}} \mathbf{z}^{(\ell-1)} + b^{(\ell)}_{\omega^{(\ell)}_{\mathbf{t}^{(\ell)}}} \right) \mathbf{1}_{\left\{\mathbf{z}^{(\ell-1)} \in \omega^{(\ell)}_{\mathbf{t}^{(\ell)}}\right\}}, \end{aligned} \tag{4.6}$$

so that $f^{(\ell)}_{\omega^{(\ell)}}\left(\mathbf{z}^{(\ell-1)}\right) := A^{(\ell)}_{\omega^{(\ell)}} \mathbf{z}^{(\ell-1)} + b^{(\ell)}_{\omega^{(\ell)}}$ is the affine transformation acting on $\omega^{(\ell)} \in \Omega^{(\ell)}$.

Therefore,

$$\begin{aligned} f(\mathbf{x}) &= \sum_{\mathbf{t} = \left[\mathbf{t}^{(1)}, \ldots, \mathbf{t}^{(L)}\right]} \left(f^{(L)}_{\omega^{(L)}_{\mathbf{t}^{(L)}}} \circ \cdots \circ f^{(1)}_{\omega^{(1)}_{\mathbf{t}^{(1)}}} \right)(\mathbf{x}) \mathbf{1}_{\{\mathbf{x} \in \omega_{\mathbf{t}}\}} \\ &=: \sum_{\mathbf{t} = \left[\mathbf{t}^{(1)}, \ldots, \mathbf{t}^{(L)}\right]} \left(A_{\omega_{\mathbf{t}}} \mathbf{x} + b_{\omega_{\mathbf{t}}} \right) \mathbf{1}_{\{\mathbf{x} \in \omega_{\mathbf{t}}\}} \\ &=: \sum_{\mathbf{t} = \left[\mathbf{t}^{(1)}, \ldots, \mathbf{t}^{(L)}\right]} f_{\omega_{\mathbf{t}}}(\mathbf{x}) \mathbf{1}_{\{\mathbf{x} \in \omega_{\mathbf{t}}\}}. \end{aligned} \tag{4.7}$$

Proposition 4.6.4. *The parameters $A_{\omega_{\mathbf{t}}}$ and $b_{\omega_{\mathbf{t}}}$ from (4.7) are given by*

$$A_{\omega_{\mathbf{t}}} = \prod_{\ell=L}^{1} A^{(\ell)}_{\omega_{\mathbf{t}^{(\ell)}}}$$

and

$$b_{\omega_{\mathbf{t}}} = b^{(L)}_{\omega_{\mathbf{t}^{(L)}}} + \sum_{i=1}^{L-1} \left(\prod_{j=i+1}^{L} A^{(j)}_{\omega_{\mathbf{t}^{(j)}}} \right) b^{(i)}_{\omega_{\mathbf{t}^{(i)}}}.$$

Proof. We proceed by induction on L.

- The case for $L = 1$ is clear.
- Assume the expressions hold for $L - 1$. Then,

$$\begin{aligned}
f^{(L)}(\mathbf{x}) &= \sum_{\mathbf{t}^{(L)}} \left(A^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \mathbf{z}^{(L-1)} + b^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \right) \mathbf{1}_{\left\{ \mathbf{z}^{(L-1)} \in \omega_{\mathbf{t}^{(L)}} \right\}} \\
&= \sum_{\mathbf{t}^{(L)}, \mathbf{t}^{(L-1)}} \left(A^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \left(\left(A^{(L-1)}_{\omega_{\mathbf{t}^{(L-1)}}} \mathbf{z}^{(L-2)} + b^{(L-1)}_{\omega_{\mathbf{t}^{(L-1)}}} \right) \mathbf{1}_{\left\{ \mathbf{z}^{(L-1)} \in \omega_{\mathbf{t}^{(L-2)}} \right\}} \right) + b^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \right) \mathbf{1}_{\left\{ \mathbf{z}^{(L-1)} \in \omega_{\mathbf{t}^{(L)}} \right\}} \\
&= \sum_{\mathbf{t}^{(L)}, \mathbf{t}^{(L-1)}} \left(A^{(L)}_{\omega_{\mathbf{t}^{(L)}}} A^{(L-1)}_{\omega_{\mathbf{t}^{(L-1)}}} \mathbf{z}^{(L-2)} + A^{(L)}_{\omega_{\mathbf{t}^{(L)}}} b^{(L-1)}_{\omega_{\mathbf{t}^{(L-1)}}} + b^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \right) \mathbf{1}_{\left\{ \mathbf{z}^{(L-1)} \in \omega_{\mathbf{t}^{(L-2)}} \right\}} \mathbf{1}_{\left\{ \mathbf{z}^{(L-1)} \in \omega_{\mathbf{t}^{(L)}} \right\}} \\
&= \sum_{\mathbf{t}} \left(A^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \left(\prod_{\ell=L-1}^{1} A^{(\ell)}_{\omega_{\mathbf{t}^{(\ell)}}} \right) \mathbf{x} + A^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \left(b^{(L-1)}_{\omega_{\mathbf{t}^{(L-1)}}} + \sum_{i=1}^{L-2} \left(\prod_{j=i+1}^{L} A^{(j)}_{\omega_{\mathbf{t}^{(j)}}} \right) b^{(i)}_{\omega_{\mathbf{t}^{(i)}}} \right) + b^{(L)}_{\omega_{\mathbf{t}^{(L)}}} \right) \mathbf{1}_{\{\mathbf{x} \in \omega_{\mathbf{t}}\}} \\
&= \sum_{\mathbf{t}} \left(\left(\prod_{\ell=L}^{1} A^{(\ell)}_{\omega_{\mathbf{t}^{(\ell)}}} \right) \mathbf{x} + b^{(L)}_{\omega_{\mathbf{t}^{(L)}}} + \sum_{i=1}^{L-1} \left(\prod_{j=i+1}^{L} A^{(j)}_{\omega_{\mathbf{t}^{(j)}}} \right) b^{(i)}_{\omega_{\mathbf{t}^{(i)}}} \right) \mathbf{1}_{\{\mathbf{x} \in \omega_{\mathbf{t}}\}}.
\end{aligned}$$

□

Proposition 4.6.5. *Let f be a deep network implementing ReLU, leaky ReLU or absolute value activation operators. Let*

$$f^{(\ell)}\left(\mathbf{z}^{(\ell-1)}\right) = \sigma\left(W^{(\ell)}\mathbf{z}^{(\ell-1)} + b^{(\ell)}\right).$$

If $\mathbf{z}^{(\ell-1)} \in \omega^{(\ell)} \in \Omega^{(\ell)}$, then

$$\begin{cases} A^{(\ell)}_{\omega^{(\ell)}} = \operatorname{diag}\left(q^{(\ell)}_{\omega^{(\ell)}}\right) W^{(\ell)} \\ b^{(\ell)}_{\omega^{(\ell)}} = \operatorname{diag}\left(q^{(\ell)}_{\omega^{(\ell)}}\right) b^{(\ell)}, \end{cases}$$

where $q^{(\ell)}_{\omega^{(\ell)}}$ is the point-wise derivative of σ at the pre-activation $W^{(\ell)}\mathbf{z}^{(\ell-1)} + b^{(\ell)}$.

Proof. The follows from the fact that the operator σ is a locally affine map around the pre-activation which passes through the origin with slope $\operatorname{diag}\left(q^{(\ell)}_{\omega^{(\ell)}}\right)$. □

Remark 4.6.6.

1. *Note that* $q^{(\ell)}_{\omega^{(\ell)}}$ *is well-defined as it is constant over* $\omega^{(\ell)}$.

2. *In these instances, note that one does not actually need to compute point-wise derivatives to obtain* $q^{(\ell)}_{\omega^{(\ell)}}$. *In fact,*

$$\left[q^{(\ell)}_{\omega^{(\ell)}}\right]_i = \begin{cases} \eta & \left[W^{(\ell)}\mathbf{z}^{(\ell-1)} + b^{(\ell)}\right]_i \leq 0 \\ 1 & \text{otherwise,} \end{cases}$$

   where $\eta = 0$ *for ReLU activation operators,* η *is the parameter for leaky ReLU operators, or* $\eta = -1$ *for the absolute value activation operator.*

3. *Similar to the notational convention given in statement 2 of Remark 2.2.2, we will let*

$$Q^{(\ell)}[\mathbf{x}] = \operatorname{diag}\left(q^{(\ell)}_{\omega^{(\ell)}_{\mathbf{s}^{(\ell)}(\mathbf{x})}}\right),$$

   and

$$Q^{(\ell)}[\omega] = \operatorname{diag}\left(q^{(\ell)}_{\omega}\right),$$

   where here $q^{(\ell)}_{\omega}$ *is the point-wise derivative of* σ *at* $W^{(\ell)}\mathbf{z}^{(\ell-1)}(\mathbf{x}) + b^{(\ell)}$ *for* $\mathbf{x} \in \omega$, *where* $\omega \in \Omega$.

4.7 Fourier Analysis

The serial structure of deep networks hints that it may be conducive to a Fourier analysis. In this section we consider that step for ReLU deep networks. could we potentially extend this to leaky relu and absolute value. In particular, we'll focus on ReLU deep networks where $d^{(L)} = 1$.

The Fourier representation of a deep network is

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{d}{2}}} \int_{\mathbb{R}^d} \tilde{f}(\mathbf{u}) e^{i\mathbf{u}\cdot\mathbf{x}} \, \mathrm{d}\mathbf{u},$$

where

$$\tilde{f}(\mathbf{u}) = \int f(\mathbf{x}) e^{-i\mathbf{u}\cdot\mathbf{x}} \, \mathrm{d}\mathbf{x}$$

is the Fourier transform

Lemma 4.7.1. *The Fourier transform of a ReLU deep network decomposes as*

$$\tilde{f}(\mathbf{u}) = i \sum_{\mathbf{s}} \frac{A_{\omega_{\mathbf{s}}}\mathbf{u}}{\|\mathbf{u}\|^2} \tilde{\mathbf{1}}_{\omega_{\mathbf{s}}}(\mathbf{u}),$$

where

$$\tilde{\mathbf{1}}_{\omega_{\mathbf{s}}} = \int_{\omega_{\mathbf{s}}} e^{-i\mathbf{u}\cdot\mathbf{x}} \, \mathrm{d}\mathbf{x}.$$

Lemma 4.7.2. *Let ω be a full-dimensional polytope in $\mathbb{R}^d$. Its Fourier spectrum takes the form*

$$\tilde{\mathbf{1}}_{\omega}(\mathbf{u}) = \sum_{n=0}^{d} \frac{D_n(\mathbf{u})\mathbf{1}_{\bar{\omega}_n}(\mathbf{u})}{\|\mathbf{u}\|^n},$$

where $\bar{\omega}_n$ is the union of n-dimensional subspaces that are orthogonal to some n-codimensional face of ω, and $D_n : \mathbb{R}^n \to \mathbb{C}$ is a term of order $\Theta(1)$ as $\|u\| \to \infty$.

Theorem 4.7.3. *The Fourier components of a ReLU deep network f_θ is given by*

$$\tilde{f}_\theta(\mathbf{u}) = \sum_{n=0}^{d} \frac{C_n(\theta, \mathbf{u})\mathbf{1}_{\bar{\omega}_n^\theta}(\mathbf{u})}{\|u\|^{n+1}},$$

where $\bar{\omega}_n^\theta$ is the union of n-dimensional subspaces that are orthogonal to some n-codimensional faces of some polytope ω, and $C_n : \mathbb{R}^d \to \mathbb{C}$ is a term of order $\Theta(1)$ as $\|u\| \to \infty$.

4.8 References

Section 4.2, Section 4.3, Section 4.4, Section 4.5 and Section 4.6 from Balestriero and Baraniuk (2018) with Proposition 4.6.5 from Humayun et al. (2023a). Section 4.7 from Rahaman et al. (2018).

5 Locally Non-Linear Deep Networks

5.1 Introduction

Our exploration thus far has been limited to deep networks whose layers are affine transformations followed by a piecewise linear activation or pooling operator. Although Theorem 4.4.1 reassures us that this is sufficient for deep networks approximating arbitrary continuous operators, in this chapter we explore how one can extend our current ideas to deep networks employing locally non-linear activation or pooling operators. This is done by introducing a probabilistic perspective on the current framework by noting its connection to vector quantization. We first develop the vector quantization theory, and then show how it can be used to derive a spectrum of deep network architectures through so-called β-VQ inference.

5.2 Vector Quantization

For a deep network we can formulate its ℓth intermediary operator as a max-affine spline operator $S\left[A^{(\ell)}, B^{(\ell)}\right]$ such that

$$\left[\mathbf{z}^{(\ell)}\right]_i = \max_{j=1,\ldots,r^{(\ell)}} \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,j}\right), \tag{5.1}$$

where $\mathbf{z}^{(\ell)} \in \mathbb{R}^{d^{(\ell)}}$, $\mathbf{z}^{(\ell-1)} \in \mathbb{R}^{d^{(\ell-1)}}$, and $r^{(\ell)}$ is the number of regions in the partition of $\mathbb{R}^{d^{(\ell-1)}}$. Importantly, this max-affine spline is completely determined by the slopes $A \in \mathbb{R}^{d^{(\ell)} \times r^{(\ell)} \times d^{(\ell-1)}}$ and offsets $B \in \mathbb{R}^{d^{(\ell)} \times r^{(\ell)}}$. It computes an optimized partition of of $\mathbb{R}^{d^{(\ell-1)}}$ which is equivalent to a vector quantization.

Consider the matrix $T_H^{(\ell)} \in \mathbb{R}^{d^{(\ell)} \times r^{(\ell)}}$ whose i^{th} row is the one-hot vector with index

$$\left[\mathbf{t}^{(\ell)}\right]_i = \text{argmax}_{j=1,\ldots,r^{(\ell)}} \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,j}\right).$$

This is the hard-VQ matrix which allows us to explicitly highlight the

vector quantization aspect of (5.1) as

$$\left[\mathbf{z}^{(\ell)}\right]_i = \sum_{j=1}^{r^{(\ell)}} \left[T_H^{(\ell)}\right]_{i,j} \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,j}\right), \quad (5.2)$$

5.3 *Soft Max-Affine Spline Operators*

Consider a single unit i from the ℓth layer, which has $d^{(\ell)}$ total units, of a max-affine spline operator based deep network.

Proposition 5.3.1. *Given* $-\frac{1}{2}\left\|\left[A_{i,j,\cdot}^{(\ell)}\right]\right\|^2 = \left[B^{(\ell)}\right]_{i,j}$*, the max-affine spline operator vector quantization partition corresponds to* $r^{(\ell)}$*-means clustering with centroids* $[A]_{i,j,\cdot}$ *computed through*

$$\left[\widehat{\mathbf{t}^{(\ell)}}\right]_i = \underset{j=1,\ldots,r^{(\ell)}}{\operatorname{argmin}} \left\|\left[A^{(\ell)}\right]_{i,j,\cdot} - \mathbf{z}^{(\ell-1)}\right\|^2.$$

Proof. Observe that,

$$\begin{aligned}
\left\|\left[A^{(\ell)}\right]_{i,j,\cdot} - \mathbf{z}^{(\ell-1)}\right\|^2 &= \left\langle \left[A^{(\ell)}\right]_{i,j,\cdot} - \mathbf{z}^{(\ell-1)}, \left[A^{(\ell)}\right]_{i,j,\cdot} - \mathbf{z}^{(\ell-1)} \right\rangle \\
&= \left\|\left[A^{(\ell)}\right]_{i,j,\cdot}\right\|^2 - 2\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle \\
&+ \left\|\mathbf{z}^{(\ell-1)}\right\|^2 \\
&= -2\left[B^{(\ell)}\right]_{i,j} - 2\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle \\
&+ \left\|\mathbf{z}^{(\ell-1)}\right\|^2.
\end{aligned}$$

Therefore,

$$\begin{aligned}
&\underset{j=1,\ldots,r^{(\ell)}}{\operatorname{argmin}} \left\|\left[A^{(\ell)}\right]_{i,j,\cdot} - \mathbf{z}^{(\ell-1)}\right\|^2 \\
&= \underset{j=1,\ldots,r^{(\ell)}}{\operatorname{argmax}} \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,j}\right).
\end{aligned}$$

□

Proposition 5.3.1, determines the selection variable $\left[\mathbf{t}^{(\ell)}\right]_i$ deterministically. Instead, one can consider it as an unobserved categorical variable $\left[\mathbf{t}^{(\ell)}\right]_i \sim \text{Cat}\left(\left[\pi^{(\ell)}\right]_{i,\cdot}\right)$, where $\left[\pi^{(\ell)}\right]_{i,\cdot} \in \Delta_{r^{(\ell)}}$.[1] With this, we can construct a generative model for $\mathbf{z}^{(\ell-1)}$ as a mixture of $r^{(\ell)}$ isotropic Gaussians with mean $\left[A^{(\ell)}\right]_{i,j,\cdot} \in \mathbb{R}^{d^{(\ell-1)}}$ and covariance τ^2 as

$$\mathbf{z}^{(\ell-1)} = \sum_{j=1}^{r^{(\ell)}} \mathbf{1}_{\left\{\left[\mathbf{t}^{(\ell)}\right]_i = j\right\}} \left[A^{(\ell)}\right]_{i,j,\cdot} + \epsilon, \quad (5.3)$$

[1] $\Delta_{r^{(\ell)}}$ denotes the simplex of dimension $r^{(\ell)}$. That is, the subset of $\mathbb{R}^{r^{(\ell)}}$ whose coordinate sum is equal to one.

where $\epsilon \sim \mathrm{N}\left(0, I\tau^2\right)$.

5.4 *Vector Quantization Inference*

Hard-VQ Inference

In hard inference, the selection variable $\left[\mathbf{t}^{(\ell)}\right]_i$ is computed via the maximum a posteriori principle as

$$\left[\widehat{\mathbf{t}^{(\ell)}}\right]_i = \underset{t=1,\ldots,r^{(\ell)}}{\operatorname{argmax}}\, p\left(t|\mathbf{z}^{(\ell-1)}\right).$$

Theorem 5.4.1. *The maximum a posteriori inference of the latent selection variable* $\left[\widehat{\mathbf{t}^{(\ell)}}\right]_i$ *can be computed through the max-affine spline operator hard-VQ*

$$\left[\widehat{\mathbf{t}^{(\ell)}}\right]_i = \underset{t=1,\ldots,r^{(\ell)}}{\operatorname{argmax}} \left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,t}$$

for all $A^{(\ell)}$ *and* $B^{(\ell)}$*, when* $\tau^2 = 1$ *and*

$$\left[\pi^{(\ell)}\right]_{i,t} = \frac{\exp\left(\left[B^{(\ell)}\right]_{i,t} + \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\|^2\right)}{\sum_{j=1}^{r^{(\ell)}} \exp\left(\left[B^{(\ell)}\right]_{i,t} + \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,j,\cdot}\right\|^2\right)}$$

for $t = 1, \ldots, r^{(\ell)}$*. The optimal hard-VQ selection matrix is then given by* $\left[\widehat{T}_H^{(\ell)}\right]_{i,t} = \mathbf{1}_{\left\{t = \left[\widehat{\mathbf{t}^{(\ell)}}\right]_i\right\}}$.

Proof. Observe that

$$\begin{aligned}
\underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(p\left(t|\mathbf{z}^{(\ell-1)}\right)\right) &= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(p\left(t|\mathbf{z}^{(\ell-1)}\right) p\left(\mathbf{z}^{(\ell-1)}\right)\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(p\left(\mathbf{z}^{(\ell-1)}|t\right) p\left(t\right)\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(\log\left(p\left(\mathbf{z}^{(\ell-1)}|t\right) p\left(t\right)\right) - \frac{1}{2}\left\|\mathbf{z}^{(\ell-1)}\right\|^2\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(\log\left(p\left(\mathbf{z}^{(\ell-1)}|t\right) p\left(t\right)\right) + \left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle - \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\|^2\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(\log\left(p\left(\mathbf{z}^{(\ell-1)}|t\right)\right) + \log\left(p(t)\right) + \left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle - \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\|^2\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(\log(p(t)) + \left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle - \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\|^2\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(\log\left(\frac{\exp\left(\left[B^{(\ell)}\right]_{i,t} + \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\|^2\right)}{\sum_{j=1}^{r^{(\ell)}} \exp\left(\left[B^{(\ell)}\right]_{i,t} + \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,j,\cdot}\right\|^2\right)}\right) + \left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle - \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\|^2\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(\left[B^{(\ell)}\right]_{i,t} + \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\| + \left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle - \frac{1}{2}\left\|\left[A^{(\ell)}\right]_{i,t,\cdot}\right\|^2\right) \\
&= \underset{t=1,\dots,r^{(\ell)}}{\operatorname{argmax}} \left(\left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle + \left[B^{(\ell)}\right]_{i,t}\right).
\end{aligned}$$

□

Soft-VQ Inference

One can replace the one-hot entries of the hard-VQ matrix with the probability that the layer input belongs to a given region,

$$\begin{aligned}
\left[\widehat{T_S^{(\ell)}}\right]_{i,t} &= p\left(\left[\mathbf{t}^{(\ell)}\right]_i = t|\mathbf{z}^{(\ell-1)}\right) \\
&= \frac{\exp\left(\left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle + \left[B^{(\ell)}\right]_{i,t,\cdot}\right)}{\sum_{j=1}^{r_i^{(\ell)}} \exp\left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle + \left[B^{(\ell)}\right]_{i,j,\cdot}\right)}. \qquad (5.4)
\end{aligned}$$

Leading to a soft inference of the categorical variable $\left[\mathbf{t}^{(\ell)}\right]_j$. Note that $T_S^{(\ell)} \to T_H^{(\ell)}$ as $\tau^2 \to 0$ in (5.3).

Theorem 5.4.2. *The entries of the soft-VQ matrix from* (5.4) *solve the*

entropy penalised maximization

$$\left[\widehat{T_S^{(\ell)}}\right]_{i,\cdot} = \underset{\mathbf{t}\in\Delta_{r_i^{(\ell)}}}{\operatorname{argmax}} \left(\sum_{j=1}^{r_i^{(\ell)}} [\mathbf{t}]_j \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,j} \right) + H(\mathbf{t}) \right),$$

where $H(\cdot)$ is the Shannon entropy.

Proof. The optimization under consideration is

$$\begin{aligned}\left[T_S^{(\ell)}\right]_i &= \underset{\mathbf{u}\sim q_i^{(\ell)}}{\operatorname{argmax}} \left(\mathbb{E}_q \left(\log \left(p\left(\mathbf{z}^{(\ell-1)}|\mathbf{u}\right) p(\mathbf{u}) \right) \right) + H\left(\mathbf{u}\right) \right) \\ &= \underset{\mathbf{t}\in\Delta_{r_i^{(\ell)}}}{\operatorname{argmax}} \left(\sum_{j=1}^{r_i^{(\ell)}} [\mathbf{t}]_j \left(-\frac{\left\|\mathbf{z}^{(\ell-1)} - \mu_j\right\|^2}{2\tau^2} + \log\left(\pi_j\right) \right) - \sum_{j=1}^{r_i^{(\ell)}} [\mathbf{t}]_j \log\left([\mathbf{t}]_j\right) \right).\end{aligned}$$

Let

$$\begin{aligned}\mathcal{L}(\mathbf{t}) = &\sum_{j=1}^{r_i^{(\ell)}} [\mathbf{t}]_j \left(-\frac{\left\|\mathbf{z}^{(\ell-1)} - \mu_j\right\|^2}{2\tau^2} + \log\left(\pi_j\right) \right) \\ &- \sum_{j=1}^{r_i^{(\ell)}} [\mathbf{t}]_j \log\left([\mathbf{t}]_j\right) + \lambda \left(\sum_{j=1}^{r_j^{(\ell)}} [\mathbf{t}]_j - 1 \right).\end{aligned}$$

Then

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial [\mathbf{t}]_p} = -\frac{\left\|\mathbf{z}^{(\ell-1)} - \mu_p\right\|^2}{2\tau^2} + \log\left(\pi_p\right) - \log\left([\mathbf{t}]_p\right) - 1 + \lambda \\ \frac{\partial \mathcal{L}}{\partial \lambda} = \sum_{j=1}^{r_i^{(\ell)}} [\mathbf{t}]_j - 1. \end{cases}$$

Setting the derivatives to zero, it follows that

$$[\mathbf{t}]_p = \exp\left(-\frac{\left\|\mathbf{z}^{(\ell-1)} - \mu_p\right\|^2}{2\tau^2} + \log\left(\pi_p\right) - 1 + \lambda \right)$$

for $p = 1, \ldots, r_i^{(\ell)}$. In particular,

$$\lambda = -\log\left(\sum_{p=1}^{r_i^{(\ell)}} \exp\left(-\frac{\left\|\mathbf{z}^{(\ell-1)} - \mu_p\right\|^2}{2\tau^2} + \log\left(\pi_p\right) - 1 \right) \right).$$

Therefore,

$$[\mathbf{t}]_p = \frac{\exp\left(-\frac{\left\|\mathbf{z}^{(\ell-1)} - \mu_p\right\|^2}{2\tau^2} + \log\left(\pi_p\right) \right)}{\sum_{j=1}^{r_i^{(\ell)}} \exp\left(-\frac{\left\|\mathbf{z}^{(\ell-1)} - \mu_j\right\|^2}{2\tau^2} + \log\left(\pi_j\right) \right)}$$

for $p = 1, \ldots, r_i^{(\ell)}$. □

β-VQ Inference

A variable combination of hard-VQ inference and soft-VQ inference leads to β-VQ inference, which recovers hard, soft and linear VQ inference. More specifically,

$$\left[\widehat{T_\beta^{(\ell)}}\right]_i = \operatorname{argmax}_{\mathbf{t}\in\Delta_{r_i^{(\ell)}}} \left[\beta^{(\ell)}\right]_i \sum_{j=1}^{r_i^{(\ell)}} [\mathbf{t}]_j \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle + [B]_{i,j}\right) + \left(1 - \left[\beta^{(\ell)}\right]_i\right) H(\mathbf{t}).$$

Theorem 5.4.3. *The unique global optimum of β-VQ inference is*

$$\left[\widehat{T_\beta^{(\ell)}}\right]_{i,j} = \frac{\exp\left(\frac{[\beta^{(\ell)}]_i}{1-[\beta^{(\ell)}]_i}\left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle + \left[B^{(\ell)}\right]_{i,j}\right)\right)}{\sum_{t=1}^{r^{(\ell)}} \exp\left(\frac{[\beta^{(\ell)}]_i}{1-[\beta^{(\ell)}]_i}\left(\left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle + \left[B^{(\ell)}\right]_{i,t}\right)\right)}.$$

Proof. Proceeds in the same way as Theorem 5.4.2, except with the convex combination provided by β. □

Remark 5.4.4. *From Theorem 5.4.3 we observe that β-VQ recovers hard-VQ inference with $\beta = 1$, soft-VQ inference with $\beta = \frac{1}{2}$ and linear max-affine spline operators with $\beta = 0$.*

5.5 Constructions

Using hard-VQ inference and soft-VQ inference, one can generate distinct deep networks given a set of parameters $A^{(\ell)}$ and $B^{(\ell)}$. More generally, one can generate a spectrum of different deep networks using β-VQ inference.

Proposition 5.5.1. *The max-affine spline operator parameters $A^{(\ell)}$ and $B^{(\ell)}$ that induce the ReLU activation operator under hard-VQ inference induce the swish non-linearity under β-VQ inference with*

$$\left[\eta^{(\ell)}\right]_i = \frac{\left[\beta^{(\ell)}\right]_i}{1 - [\beta^{(\ell)}]_i}.$$

Proof. Recall statement 1 Example 3.2.4 for the parameters of the

ReLU activation operator. Using (5.2) and Theorem 5.4.3, we have

$$\begin{aligned}
\left[\mathbf{z}^{(\ell)}\right]_i &= \left[T_\beta^{(\ell)}\right]_{i,2} \left[\mathbf{z}^{(\ell-1)}\right]_i \\
&= \frac{\exp\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}\left[\mathbf{z}^{(\ell-1)}\right]_i\right)}{1+\exp\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}\left[\mathbf{z}^{(\ell-1)}\right]_i\right)}\left[\mathbf{z}^{(\ell-1)}\right]_i \\
&= \frac{1}{1+\exp\left(-\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}\left[\mathbf{z}^{(\ell-1)}\right]_i\right)}\left[\mathbf{z}^{(\ell-1)}\right]_i \\
&= \sigma_{\text{sig}}\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}\left[\mathbf{z}^{(\ell-1)}\right]_i\right)\left[\mathbf{z}^{(\ell-1)}\right]_i \\
&= \sigma_{\text{swish}}\left(\left[\mathbf{z}^{(\ell-1)}\right]_i ; \frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}\right)
\end{aligned}$$

□

Remark 5.5.2. *With $\beta = \frac{1}{2}$, namely soft-VQ inference, we recover the sigmoid gated linear activation operator in Proposition 5.5.1.*

Proposition 5.5.3. *The max-affine spline operator parameters $A^{(\ell)}$ and $B^{(\ell)}$ that induce the max-pooling operators under hard-VQ inference induce local-importance-pooling with $g(x) = \exp\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}x\right)$ under β-VQ inference.*

Proof. Recall statement 1 of Example 3.2.6 for the parameters of the max-pooling operator. Using (5.2) and Theorem 5.4.3, we have

$$\begin{aligned}
\left[\mathbf{z}^{(\ell)}\right]_i &= \sum_{j=1}^{r}\left[T_\beta^{(\ell)}\right]_{i,j}\left[\mathbf{z}^{(\ell-1)}\right]_{\left[\mathcal{R}_i^{(\ell)}\right]_j} \\
&= \sum_{j=1}^{r}\frac{\exp\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}\left[\mathbf{z}^{(\ell-1)}\right]_{\left[\mathcal{R}_i^{(\ell)}\right]_j}\right)}{\sum_{t=1}^{r}\exp\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i}\left[\mathbf{z}^{(\ell-1)}\right]_{\left[\mathcal{R}_i^{(\ell)}\right]_t}\right)}\left[\mathbf{z}^{(\ell-1)}\right]_{\left[\mathcal{R}_i^{(\ell)}\right]_j}
\end{aligned}$$

□

Remark 5.5.4. *With $\beta = \frac{1}{2}$, namely soft-VQ inference, we recover the softmax-pooling activation operator in Proposition 5.5.1.*

5.6 References

Section 5.2, Section 5.3, Section 5.4 and Section 5.5.

6 Partition Geometry

6.1 Introduction

In Section 4.6 we were able to characterise the input space partitioning induced by a deep network. However, an analytic closed-form formula for the regions alluded us. In this chapter we aim to rectify that by using power diagrams.

6.2 Power Diagrams

Definition 6.2.1. *A power diagram partitions a space $\mathcal{X}$ into r disjoint regions $\Omega = \{\omega_1, \ldots, \omega_r\}$ such that $\bigcup_{j=1}^{r} \omega_j = \mathcal{X}$, with*

$$\omega_j = \{\mathbf{x} \in \mathcal{X} : \nu(\mathbf{x}) = j\}$$

for $j = 1, \ldots, r$ where $\nu(\mathbf{x}) = \operatorname{argmin}_{i=1,\ldots,r} \left(\|\mathbf{x} - [\mu]_{i,\cdot}\|^2 - [\tau]_i \right)$, for centroids $[\mu]_{i,\cdot}$ and radius $[\tau]_i$.

Observe how a power diagram is invariant under global shifting of the radii,

$$\operatorname*{argmin}_{i=1,\ldots,r} \left(\|\mathbf{x} - [\mu]_{i,\cdot}\|^2 - [\tau]_{i,\cdot} \right) = \operatorname*{argmin}_{i=1,\ldots,r} \left(\|\mathbf{x} - [\mu]_{i,\cdot}\|^2 - ([\tau]_{i,\cdot} + q) \right).$$

6.3 Max-Affine Splines

A max-affine spline operator with k output dimensions and input space $\mathcal{X}$, is constituted from k max-affine spline units, with the ith having the computation

$$[\mathbf{z}(\mathbf{x})]_i = \max_{j=1,\ldots,r} \langle [A]_{i,j,\cdot}, \mathbf{x} \rangle + [B]_{i,j} = \max_{j=1,\ldots,r} \mathcal{E}_{i,j}(\mathbf{x}), \tag{6.1}$$

where $\mathcal{E}_{i,j}(\mathbf{x})$ is the projection of $\mathbf{x}$ onto the hyperplane parameterized by $[A]_{i,j,\cdot}$ and $[B]_{i,j}$.

Let

$$\mathcal{E}_{i,j}^{+} = \left\{ (\mathbf{x}, y) \in \mathcal{X} \times \mathbb{R} : y \geq \mathcal{E}_{i,j}(\mathbf{x}) \right\}.$$

Proposition 6.3.1. *The ith max-affine unit maps its input space onto the boundary of the convex polytope $\mathcal{P}_i = \bigcap_{j=1}^{r} \mathcal{E}_{i,j}^{+}$, leading to*

$$\mathcal{X} \times [\mathbf{z}]_i(\mathcal{X}) = \partial \mathcal{P}_i,$$

where

$$[\mathbf{z}]_i(\mathcal{X}) = \operatorname{Im}([\mathbf{z}]_i) = \{[\mathbf{z}(\mathbf{x})]_i : \mathbf{x} \in \mathcal{X}\}.$$

Proof. $(\subseteq)$
Let $(\mathbf{x}, y) \in \mathcal{X} \times [\mathbf{z}]_i(\mathcal{X})$. Then $y = \mathcal{E}_{i,j'}(\mathbf{x})$ for some $j' \in \{1, \ldots, r\}$ and $y \geq \mathcal{E}_{i,j}(\mathbf{x})$ for every $j = 1, \ldots, r$. Therefore,

$$\begin{aligned}(\mathbf{x}, y) &\in \mathcal{P}_i \setminus \operatorname{int}(\mathcal{P}_i) \\ &= \overline{\mathcal{P}_i} \setminus \operatorname{int}(\mathcal{P}_i) \\ &\subseteq \partial \mathcal{P}_i.\end{aligned}$$

$(\supseteq)$
We proceed by induction on r.

- For $r = 1$ we have

$$\begin{aligned}\partial \mathcal{P}_i &= \partial \mathcal{E}_{i,1}^{+} \\ &= \{(\mathbf{x}, y) : y = \mathcal{E}_{i,1}(\mathbf{x})\} \\ &\subseteq \mathcal{X} \times [\mathbf{z}]_i(\mathcal{X}).\end{aligned}$$

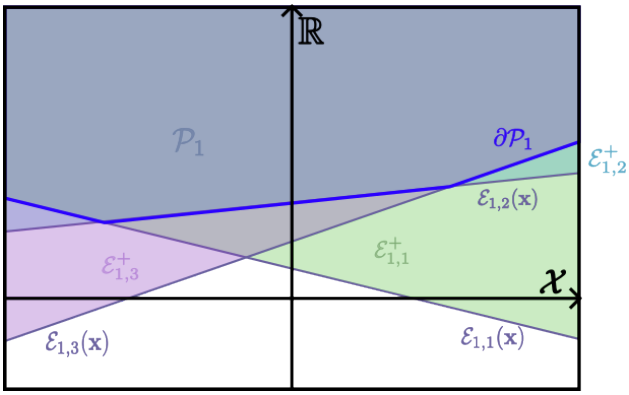
- Assume that

$$\mathcal{X} \times [\mathbf{z}]_i(\mathcal{X}) \subseteq \partial \mathcal{P}_i = \partial \left(\bigcap_{j=1}^{n-1} \mathcal{E}_{i,j}^{+} \right).$$

- For $r = n$ we have

$$\begin{aligned}\partial \mathcal{P}_i &= \partial \left(\bigcap_{j=1}^{n-1} \mathcal{E}_{i,j}^{+} \cap \mathcal{E}_{i,j}^{+} \right) \\ &\subseteq \left(\overline{\bigcap_{j=1}^{n-1} \mathcal{E}_{i,j}^{+}} \cap \partial \mathcal{E}_{i,n}^{+} \right) \cup \left(\partial \left(\bigcap_{j=1}^{n-1} \mathcal{E}_{i,j}^{+} \right) \cap \overline{\mathcal{E}_{i,n}^{+}} \right) \\ &= \left(\bigcap_{j=1}^{n-1} \mathcal{E}_{i,j}^{+} \cap \left\{ (\mathbf{x}, y) : \mathcal{E}_{i,n}^{+} = y \right\} \right) \\ &\quad \cup \left(\partial \left(\bigcap_{j=1}^{n-1} \mathcal{E}_{i,j}^{+} \right) \cap \mathcal{E}_{i,n}^{+} \right) \\ &\subseteq \mathcal{X} \times [\mathbf{z}]_i(\mathcal{X}).\end{aligned}$$

□



Lemma 6.3.2. *The vertical projection on the input space $\mathcal{X}$ of the faces of $\mathcal{P}_i$ from Proposition 6.3.1 construct regions of a power diagram through*

$$v_i(\mathbf{x}) = \operatorname{argmax}_{j=1,\dots,r} \mathcal{E}_{i,j}(\mathbf{x}).$$

Proof. The faces of $\mathcal{P}_i$ constitute the boundary $\partial\mathcal{P}_i$, which from Proposition 6.3.1 are points $(\mathbf{x}, y)$ where $y \geq \mathcal{E}_{i,j}(\mathbf{x})$ for every $j \in \{1, \dots, r\}$ and $y = \mathcal{E}_{i,j'}(\mathbf{x})$ for $j' \in \mathcal{J} \subseteq \{1, \dots, r\}$, where $\mathcal{J} \neq \emptyset$. When $\mathcal{J} = \{j'\}$, the point $(\mathbf{x}, y)$ is an interior point to a face and is vertically projected, through (6.1), to an interior point of

$$\omega_{j'} := \left\{ \mathbf{x} \in \mathcal{X} : \operatorname*{argmax}_{j=1,\dots,r} \mathcal{E}_{i,j}(\mathbf{x}) = j' \right\}.$$

Clearly, the sets $\{\omega_1, \dots, \omega_r\}$ have disjoint interior, with boundaries between them corresponding to boundaries between faces. Moreover, the set $\{\omega_1, \dots, \omega_r\}$ cover $\mathcal{X}$ and thus form a power diagram. □

Theorem 6.3.3. *The ith max-affine unit partitions its input space according to a power diagram with r centroids given by $[\mu]_{i,j} = [A]_{i,j}$ and $[\tau]_{i,j} = 2[B]_{i,j} + \left\|[A]_{i,j,\cdot}\right\|^2$ for all $j \in \{1, \dots, r\}$.*

Proof. Observe that

$$\begin{aligned}
\operatorname*{argmax}_{j=1,\dots,r} \left(\mathcal{E}_{i,j}(\mathbf{x})\right) &= \operatorname*{argmax}_{j=1,\dots,r} \left(\left\langle [A]_{i,j,\cdot}, \mathbf{x} \right\rangle + [B]_{i,j}\right) \\
&= \operatorname*{argmax}_{j=1,\dots,r} \left(2\left\langle [A]_{i,j,\cdot}, \mathbf{x} \right\rangle + 2[B]_{i,j}\right) \\
&= \operatorname*{argmax}_{j=1,\dots,r} \left(\left\|[A]_{i,j,\cdot}\right\|^2 - \left\|[A]_{i,j,\cdot}\right\|^2 + 2\left\langle [A]_{i,j,\cdot}, \mathbf{x} \right\rangle + \|\mathbf{x}\|^2 + 2[B]_{i,j}\right) \\
&= \operatorname*{argmax}_{j=1,\dots,r} \left(-\left\|[A]_{i,j,\cdot} - \mathbf{x}\right\|^2 + \left\|[A]_{i,j,\cdot}\right\|^2 + 2[B]_{i,j}\right) \\
&= \operatorname*{argmin}_{j=1,\dots,r} \left(\left\|[A]_{i,j,\cdot} - \mathbf{x}\right\|^2 - \left(\left\|[A]_{i,j,\cdot}\right\|^2 + 2[B]_{i,j}\right)\right).
\end{aligned}$$

Therefore, we conclude using Lemma 6.4.2. □

Corollary 6.3.4. *The input space partitioning of a deep network unit is composed of convex polytopes.*

6.4 Max-Affine Spline Operator

A max-affine spline operator is made by concatenating k max-affine spline units. Let

$$\boldsymbol{\nu}(\mathbf{x}) = (\nu_1(\mathbf{x}), \ldots, \nu_k(\mathbf{x}))$$

and

$$E^+_{\boldsymbol{\nu}} = \left\{ (\mathbf{x}, \mathbf{y}) \in \mathcal{X} \times \mathbb{R}^k : [\mathbf{y}]_1 \geq \mathcal{E}_{1,\nu_1}(\mathbf{x}), \ldots, [\mathbf{y}]_k \geq \mathcal{E}_{1,\nu_k}(\mathbf{x}) \right\}.$$

Proposition 6.4.1. *The layer operator* $\mathbf{z}$ *maps its input space into the boundary of the* $\dim(\mathcal{X}) + k$ *dimensional convex polytope* $\mathbf{P} = \bigcap_{\boldsymbol{\nu} \in \{1,\ldots,r\}^k} E^+_{\boldsymbol{\nu}}$ *as*

$$\mathcal{X} \times \mathbf{z}(\mathcal{X}) = \mathcal{X} \times [\mathbf{z}]_1(\mathcal{X}) \times \cdots \times [\mathbf{z}]_k(\mathcal{X}) = \partial \mathbf{P}.$$

Lemma 6.4.2. *The vertical projection on the input space* $\mathcal{X}$ *of the faces of the polytope of* $\mathbf{P}$ *from Proposition 6.4.1 construct regions of a power diagram through* $\boldsymbol{\nu}$.

Theorem 6.4.3. *A deep network layer partitions its input space according to a power diagram with* $\{1, \ldots, r\}^k$ *cells, centroids* $\mu_{\boldsymbol{\nu}} = \sum_{j=1}^k [A]_{j,\nu_j,\cdot}$ *and radii* $\tau_{\boldsymbol{\nu}} = 2 \langle \mathbf{1}, B_{\boldsymbol{\nu}} \rangle + \|\mu_{\boldsymbol{\nu}}\|^2$.

Corollary 6.4.4. *The input space partitioning of a deep network layer is composed of convex polytopes.*

6.5 Deep Network

The power diagram partitioning of an L-layer deep network can be constructed recursively; each per layer polytope subdivides the previous layer's partitioning.

INITIALIZATION identifies the part of the input space of interest $\mathcal{X}^{(0)} \subseteq \mathbb{R}^d$.

LAYER ONE recursion subdivides $\mathcal{X}^{(0)} \subseteq \mathbb{R}^d$ into a power diagram from Theorem 6.4.3 with parameters $A^{(1)}$ and $B^{(1)}$ to obtain $\Omega^{(1)}$.

LAYER TWO recursion subdivides each cell of $\Omega^{(1)}$. Consider the cell $\omega^{(1)}_{\boldsymbol{\nu}^{(1)}} \subseteq \mathbb{R}^d$. Inputs of this cell a projected to $\mathcal{X}^{(1)} \subseteq \mathbb{R}^{d^{(1)}}$ as

$$\text{aff}_{\boldsymbol{\nu}^{(1)}} = \left\{ A^{(1)}_{\boldsymbol{\nu}^{(1)}} \mathbf{x} + B^{(1)}_{\boldsymbol{\nu}^{(1)}} : \mathbf{x} \in \omega^{(1)}_{\boldsymbol{\nu}^{(1)}} \right\} \subseteq \mathcal{X}^{(1)},$$

which is also convex.

Lemma 6.5.1. *The domain restricted polytope*

$$\mathbf{P}^{(2)}_{\boldsymbol{\nu}^{(1)}} = \mathbf{P}^{(2)} \cap \left(\mathrm{aff}_{\boldsymbol{\nu}^{(1)}} \times \mathbb{R}^{d^{(2)}}\right),$$

where $\mathbf{P}^{(2)}$ *is as in Proposition 6.4.1, can be expressed in the input space* $\omega^{(1)}_{\boldsymbol{\nu}^{(1)}} \subseteq \mathcal{X}^{(0)}$ *as*

$$\mathbf{P}^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(1)}} = \bigcap_{\boldsymbol{\nu}^{(2)}} \left\{ (\mathbf{x}, \mathbf{y}) \in \omega^{(1)}_{\boldsymbol{\nu}^{(1)}} \times \mathbb{R}^{d^{(1)}} : [\mathbf{y}]_1 \geq \mathcal{E}^{(1\leftarrow 2)}_{1,\nu^{(2)}_1}(\mathbf{x}), \ldots, [\mathbf{y}]_{d^{(1)}} \geq \mathcal{E}^{(1\leftarrow 2)}_{d^{(1)},\nu^{(2)}_{d^{(1)}}}(\mathbf{x}) \right\},$$

with $\mathcal{E}^{(1\leftarrow 2)}_{i,[\boldsymbol{\nu}^{(1)}]_i}$ *the hyperplane with slope* $\left(A^{(1)}_{\boldsymbol{\nu}^{(1)}}\right)^\top A^{(2)}_{\mathbf{r}^{(2)}}$ *and bias* $\left\langle \left[A^{(2)}_{\boldsymbol{\nu}^{(2)}}\right]_{i,j}, B_{\boldsymbol{\nu}^{(1)}} \right\rangle + B^{(2)}_{\boldsymbol{\nu}^{(2)}}$ *for* $j \in \left\{1, \ldots, d^{(1)}\right\}$.

From Lemma 6.4.2, $\mathbf{P}^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(1)}}$ induces a power diagram on $\omega^{(1)}_{\boldsymbol{\nu}^{(1)}}$, say $\mathrm{PD}^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(1)}}$ with centroids

$$\mu^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(1)},\boldsymbol{\nu}^{(2)}} = \left(A^{(1)}_{\boldsymbol{\nu}^{(1)}}\right)^\top \mu^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(2)}}$$

and radii

$$\tau^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(1)},\boldsymbol{\nu}^{(2)}} = \left\|\mu^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(1)},\boldsymbol{\nu}^{(2)}}\right\|^2 + 2\left\langle \mu^{(2)}_{\boldsymbol{\nu}^{(2)}}, B^{(1)}_{\boldsymbol{\nu}^{(1)}} \right\rangle + 2\left\langle \mathbf{1}, B^{(2)}_{\boldsymbol{\nu}^{(2)}} \right\rangle$$

for every $\boldsymbol{\nu}^{(2)} \in \{1, \ldots, r\}^{d^{(2)}}$. Repeating this for every cell of $\Omega^{(1)}$ leads to

$$\Omega^{(1,2)} = \bigcup_{\boldsymbol{\nu}^{(1)}} \mathrm{PD}^{(1\leftarrow 2)}_{\boldsymbol{\nu}^{(1)}}.$$

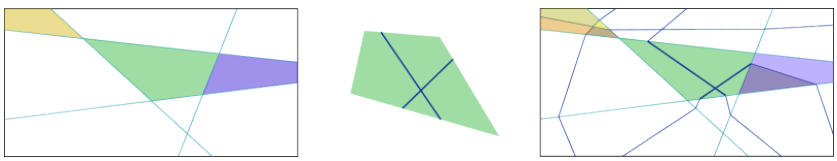


Figure 6.1: **Left:** The bounding box represents $\mathcal{X}^{(0)}$, with each line representing the partition boundary induced by a unit of layer one. A few regions of $\Omega^{(1)}$ are highlighted, suppose that the green region corresponds to $\omega^{(1)}_{\boldsymbol{v}^{(1)}}$. **Centre:** Represents $\mathrm{aff}_{\boldsymbol{v}^{(1)}}$ with the blue lines representing the power diagram induces by the vertical projection of $\mathbf{P}^{(2)}_{\boldsymbol{v}^{(1)}}$. **Right:** Represents the power diagram $\mathrm{PD}^{(1\leftarrow 2)}_{\boldsymbol{v}^{(1)}}$ along with the other power diagrams for the other cells of $\Omega^{(1)}$. Altogether they constitute $\Omega^{(1,2)}$.

Layer ℓ recursion subdivides each cell of $\Omega^{(1,\ldots,\ell-1)}$ from the previous recursion steps, forming the power diagram

$$\Omega^{(1,\ldots,\ell)} = \bigcup_{\boldsymbol{\nu}^{(1\leftarrow \ell-1)}} \mathrm{PD}_{\boldsymbol{\nu}^{(1\leftarrow \ell-1)}}.$$

Theorem 6.5.2. *Each cell* $\omega^{(1\leftarrow \ell-1)}_{\boldsymbol{\nu}^{(1\leftarrow \ell-1}} \in \Omega^{(1\leftarrow \ell-1)}$ *is subdivided into* $\mathrm{PD}^{(1\leftarrow \ell)}_{\boldsymbol{\nu}^{(1\leftarrow \ell-1)}}$, *a power diagram with domain* $\omega^{(1\leftarrow \ell-1)}_{\boldsymbol{\nu}^{(1\leftarrow \ell-1)}} \in \Omega^{(1\leftarrow \ell-1)}$, *centroids*

$$\mu^{(1\leftarrow \ell)}_{\boldsymbol{\nu}^{(1\leftarrow \ell)}} = \left(A^{(1\leftarrow \ell-1)}_{\boldsymbol{\nu}^{(1\leftarrow \ell-1)}}\right)^\top \mu^{(\ell)}_{\boldsymbol{\nu}^{(\ell)}}$$

and radii

$$\tau^{(1\leftarrow\ell)}_{\boldsymbol{\nu}^{(1\leftarrow\ell)}} = -\left\|\mu^{(1\leftarrow\ell)}_{\boldsymbol{\nu}^{(1\leftarrow\ell)}}\right\|^2 - 2\left\langle \mu^{(\ell)}_{\boldsymbol{\nu}^{(\ell)}}, B^{(1\leftarrow\ell-1)}_{\boldsymbol{\nu}^{(1\leftarrow\ell-1)}}\right\rangle - 2\left\langle \mathbf{1}, B^{(\ell)}_{\boldsymbol{\nu}^{(\ell)}}\right\rangle,$$

for every $\boldsymbol{\nu}^{(i)} \in \{1,\dots,r\}^{d^{(i)}}$ *with*

$$B^{(1\leftarrow\ell-1)} = \sum_{\ell'=1}^{\ell-1}\left(\prod_{i=\ell-1}^{\ell'} A^{(i)}_{\boldsymbol{\nu}^{(i)}}\right) B^{(\ell')}_{\boldsymbol{\nu}^{(\ell')}}$$

forming $\Omega^{(1\leftarrow\ell)}$.

Corollary 6.5.3. *The input space partitioning of a deep network is composed of convex polygons.*

6.6 Decision Boundary

Consider the case of $r = 2$ nonlinear activation operators.

Definition 6.6.1. *The edges of a polytope* $\mathcal{P}^{(\ell)}_i$ *in a space* $\mathcal{X}^{(\ell')}$*, for* $\ell' < \ell$*, are*

$$\text{edge}_{\mathcal{X}^{(\ell')}}(i,\ell) = \left\{\mathbf{x} \in \mathcal{X}^{(\ell')} : \mathcal{E}^{(\ell-1)}_{i,2}\left(\mathbf{z}^{(\ell'\to\ell-1)}(\mathbf{x})\right) = 0\right\},$$

where $\mathbf{z}^{(\ell'\to\ell-1)} = \mathbf{z}^{(\ell-1)} \circ \cdots \circ \mathbf{z}^{(\ell')}$.[1]

[1] Note how it is sufficient to only consider $\mathcal{E}^{(\ell-1)}_{i,2}$, due our construction Example 3.2.4, the piecewise continuity of the operators and the fact that $r = 2$.

That is, edges are the level set curve of the unit in $\mathcal{X}^{(\ell')}$. Let

$$\text{Pol}^{(\ell)}(\mathbf{x}) = \prod_{i=1}^{d^{(\ell)}}\left(\left[\mathbf{z}^{(\ell)}\right]_i \circ \mathbf{z}^{(\ell-1)} \circ \cdots \circ \mathbf{z}^{(1)}\right)(\mathbf{x}).$$

Theorem 6.6.2. *The polynomial* $\text{Pol}^{(\ell)}$ *is of order* $\prod_{\ell'=1}^{\ell} d^{(\ell')}$*, with roots corresponding to the partitioning boundaries*

$$\partial\Omega^{(1\leftarrow\ell)} = \bigcup_{\ell'=1}^{\ell}\bigcup_{i=1}^{d^{(\ell')}} \text{edge}_{\mathcal{X}^{(0)}}(i,\ell') = \left\{\mathbf{x} \in \mathcal{X}^{(0)} : \prod_{\ell'=1}^{\ell}\text{Pol}^{(\ell')}(\mathbf{x}) = 0\right\}.$$

In particular, the root order gives the dimension of the boundary.

Corollary 6.6.3. *The decision boundary of an L layer deep network with* $d^{(L)} = 1$ *is the edge of the last layer polytope* $\mathbf{P}^{(L)}$ *expressed in* $\mathcal{X}^{(0)}$*. Namely,*

$$\text{DB} = \left\{\mathbf{x} \in \mathcal{X}^0 : f(\mathbf{x}) = 0\right\} = \text{edge}_{\mathcal{X}^{(0)}}(1,L) \subseteq \partial\Omega^{(1,\dots,L)}.$$

6.7 References

Results in Chapter 6 from Balestriero et al. (2019).

7 Initialisation and Regularisation

7.1 Introduction

Having developed an understanding of how the parameters of a deep network effect the underlying geometry of the deep network, we can start investigating the precise effects of operators on the functional geometry of a deep network and how this translates into properties of the deep network. Specifically, we focus on the normalisation operations and how they initialise and regularise deep networks.

7.2 Batch Normalisation

For a batch normalised layer, let

$$\left[\mathbf{h}^{(\ell)}\right]_i = \frac{\left\langle \left[W^{(\ell)}\right]_{i,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle - \left[\mu^{(\ell)}\right]_i}{\left[\sigma^{(\ell)}\right]_i},$$

so that $\mathbf{h}^{(\ell)}$ is the pre-activation[1] of layer ℓ and $\mathbf{z}^{(\ell)} = \sigma_{\text{act}}\left(\mathbf{h}^{(\ell)}\right)$.

The separation of regimes induced by the affine components of σ_{act}[2] is governed by the hyperplane arrangement

$$\partial\Omega^{(\ell)} = \bigcup_{i=1}^{d^{(\ell)}} \mathcal{H}_i^{(\ell)}$$

where

$$\mathcal{H}_i^{(\ell)} = \left\{ \mathbf{z}^{(\ell-1)} \in \mathbb{R}^{d^{(\ell-1)}} : \left[\mathbf{h}^{(\ell)}\right]_i = 0 \right\}$$

is a $d^{(\ell)}$-dimensional hyperplane.[3]

[1] This is well-defined provided $\left\| \left[W^{(\ell)}\right]_{i,\cdot} \right\|_2^2 > 0$ and the inputs of the batch are not orthogonal to $\left[W^{(\ell)}\right]_{i,\cdot}$.

[2] In this analysis we are assuming that σ_{act} consists of two affined components joining at the origin.

[3] Note that $\mathcal{H}_i^{(\ell)}$ is independent of $\left[\sigma^{(\ell)}\right]_i$.

Single Layer Analysis

Lemma 7.2.1. *The Euclidean distance from a point* $\mathbf{v}$ *in layer* ℓ*'s input space and* $\mathcal{H}_i^{(\ell)}$ *is*

$$d\left(\mathbf{v}, \mathcal{H}_i^{(\ell)}\right) = \frac{\left| \left\langle \left[W^{(\ell)}\right]_{i,\cdot}, \mathbf{v} \right\rangle - \left[\mu^{(\ell)}\right]_i \right|}{\left\| \left[W^{(\ell)}\right]_{i,\cdot} \right\|_2},$$

when $\left\| \left[W^{(\ell)} \right]_{i,\cdot} \right\|_2 > 0.$[4]

[4] For a proof refer to Alexander Schrijver (1998).

Let $\mathcal{V}$ be a collection of points in ℓ's input space. From Lemma 7.2.1, it follows that the unit least squares distance is

$$\mathcal{L}_i \left(\left[\mu^{(\ell)} \right]_i ; \mathcal{V} \right) = \frac{1}{|\mathcal{V}|} \sum_{\mathbf{v} \in \mathcal{V}} d \left(\mathbf{v}, \mathcal{H}_i^{(\ell)} \right)^2 := \frac{\left[\sigma^{(\ell)} \right]_{\ell,i}}{\left\| \left[W^{(\ell)} \right]_{i,\cdot} \right\|_2^2} \tag{7.1}$$

for $i = 1, \ldots, d^{(\ell)}$ where the total least squares distance is

$$\mathcal{L} \left(\mu^{(\ell)} ; \mathcal{V} \right) = \sum_{i=1}^{d^{(\ell)}} \mathcal{L}_i \left(\left[\mu^{(\ell)} \right]_i ; \mathcal{V} \right).$$

Theorem 7.2.2. *Consider a deep network layer ℓ equipped with batch normalisation. Let $\mathcal{B}^{(\ell)} \subseteq \mathcal{X}^{(\ell)}$ be a mini-batch. Then $\mu^{(\ell)}$ as given by (1.1) is the unique minimiser of $\mathcal{L} \left(\mu^{(\ell)} ; \mathcal{B}^{(\ell)} \right)$.*

Proof. Consider the optimisation problem

$$\min_{\mu \in \mathbb{R}^{d^{(\ell)}}} \mathcal{L} \left(\mu ; \mathcal{Z} \right).$$

Since the problem is unconstrained and the sum is separable, we can focus on $[\mu]_i$, yielding the optimisation problem

$$\min_{[\mu]_i \in \mathbb{R}} \sum_{\mathbf{z} \in \mathcal{Z}} \frac{\left| \left\langle \left[W^{(\ell)} \right]_{i,\cdot}, \mathbf{z} \right\rangle - [\mu]_i \right|^2}{\left\| \left[W^{(\ell)} \right]_{i,\cdot} \right\|_2^2}.$$

Observe that,

$$\partial \left(\sum_{\mathbf{z} \in \mathcal{Z}} \frac{\left| \left\langle \left[W^{(\ell)} \right]_{i,\cdot}, \mathbf{z} \right\rangle - [\mu]_i \right|^2}{\left\| \left[W^{(\ell)} \right]_{i,\cdot} \right\|_2^2} \right) = -2 \sum_{\mathbf{z} \in \mathcal{Z}} \frac{\left\langle \left[W^{(\ell)} \right]_{i,\cdot}, \mathbf{z} \right\rangle}{\left\| \left[W^{(\ell)} \right]_{i,\cdot} \right\|_2^2} + 2|\mathcal{Z}| \frac{[\mu]_i}{\left\| \left[W^{(\ell)} \right]_{i,\cdot} \right\|_2^2}.$$

It follows that the first derivative is zero if and only if

$$[\mu]_i = \frac{1}{|\mathcal{Z}|} \sum_{\mathbf{z} \in \mathcal{Z}} \left\langle \left[W^{(\ell)} \right]_{i,\cdot}, \mathbf{z} \right\rangle.$$

Second derivative calculations confirm that this turning point is a minimum. By aggregating these across each of the $d^{(\ell)}$ dimensions, we deduce the desired result. □

Remark 7.2.3. *For concreteness we note that by taking $\mathcal{B}^{(\ell)} = \mathcal{X}^{(\ell)}$ in Theorem 7.2.2 we deduce that $\bar{\mu}^{(\ell)}$ is the unique minimiser of $\mathcal{L}\left(\mu^{(\ell)}; \mathcal{X}^{(\ell)}\right)$.*

Definition 7.2.4. *A collection of hyperplanes $\mathcal{H}$ is central if*

$$\bigcap_{H \in \mathcal{H}} H \neq \emptyset.$$

Corollary 7.2.5. *On $\mathcal{Z} \subseteq \mathcal{X}$, the induced arrangement of hyperplanes $\partial\Omega^{(\ell)}$ is central.*

Proof. Let $\bar{\mathbf{z}}$ be the centroid of $\mathcal{Z}$. For $i \in \left\{1, \ldots, d^{(\ell)}\right\}$ observe that

$$\begin{aligned}
\left\langle \left[W^{(\ell)}\right]_{i,\cdot}, \bar{\mathbf{z}} \right\rangle &= \left\langle \left[W^{(\ell)}\right]_{i,\cdot}, \frac{1}{|\mathcal{Z}|} \sum_{\mathbf{z} \in \mathcal{Z}} \mathbf{z} \right\rangle \\
&= \frac{1}{|\mathcal{Z}|} \sum_{\mathbf{z} \in \mathcal{Z}} \left\langle \left[W^{(\ell)}\right]_{i,\cdot}, \mathbf{z} \right\rangle \\
&\overset{(1.1)}{=} \left[\mu^{(\ell)}\right]_i.
\end{aligned}$$

Therefore, $\bar{\mathbf{z}} \in \mathcal{H}_i^{(\ell)}$. Since, i was arbitrary it follows that

$$\bar{\mathbf{z}} \in \bigcap_{i=1}^{d^{(\ell)}} \mathcal{H}_i^{(\ell)},$$

meaning the arrangement $\partial\Omega^{(\ell)}$ is central. □

In this analysis, we see that $\mu^{(\ell)}$ has a direct influence on the partitioning of a deep network layer's input space. The effect of the parameter $\sigma^{(\ell)}$ are introduced when composing deep network layers.

Multilayer Analysis

Similarly to Section 7.2, we have

$$\partial\Omega^{(1,\ldots,\ell)} = \bigcup_{j=1}^{\ell} \bigcup_{i=1}^{d^{(j)}} \left\{ \mathbf{x} \in \mathbb{R}^{d^{(1)}} : \left[\mathbf{h}^{(j)}\right]_i = 0 \right\} =: \bigcup_{j=1}^{\ell} \bigcup_{i=1}^{d^{(j)}} \mathcal{F}_{i,\omega}^{(j)}.$$

In this scenario, the hyperplanes $\mathcal{H}_i^{(j)}$ are folded through the preceding layers to form facets

$$\mathcal{F}_{i,\omega}^{(j)} = \left\{ \mathbf{x} \in \omega : \left[\mathbf{h}^{(j)}\right]_i = 0 \right\}$$

for $\omega \in \Omega^{(1,\ldots,j)}$.[5] With $\mathcal{F}_i^{(j)} := \bigcup_{\omega \in \Omega^{(1,\ldots,j)}} \mathcal{F}_{i,\omega}^{(j)}$ we can write

$$\partial\Omega^{(1,\ldots,\ell)} = \bigcup_{j=1}^{\ell} \bigcup_{i=1}^{d^{(j)}} \mathcal{F}_i^{(j)}.$$

[5] We can characterise the facets using the regions of $\Omega^{(1,\ldots,j)}$, since the folds only happen at the boundaries of these regions as within the regions the transformation is affine.

The Effect of $\mu^{(\ell)}$.

Theorem 7.2.6. *Consider a deep network layer $\ell > 1$ equipped with batch normalisation. Suppose that layers $1, \ldots, \ell - 1$ have fixed batch normalisation parameters. Let $\mu^{(\ell)}$ and $\tilde{\mu}^{(\ell)}$ yield hyperplanes $\mathcal{H}_i^{(\ell)}$ and $\tilde{\mathcal{H}}_i^{(\ell)}$ which are folded into $\mathcal{F}_i^{(\ell)}$ and $\tilde{\mathcal{F}}_i^{(\ell)}$ respectively. Then, $d\left(\mathbf{z}^{(\ell)}(\mathbf{x}), \mathcal{H}_i^{(\ell)}\right) < d\left(\mathbf{z}^{(\ell)}(\mathbf{x}), \tilde{\mathcal{H}}_i^{(\ell)}\right)$ implies that $d\left(\mathbf{x}, \mathcal{F}_i^{(\ell)}\right) < d\left(\mathbf{x}, \tilde{\mathcal{F}}_i^{(\ell)}\right)$.*[6]

[6] Here, $d\left(\mathbf{x}, \mathcal{F}_i^{(\ell)}\right) = \min_{\mathbf{x}' \in \mathcal{F}_i^{(\ell)}} \|\mathbf{x} - \mathbf{x}'\|^2$.

Proof. Let

$$\mathbf{x}^* = \operatorname*{argmin}_{\mathbf{u} \in \mathcal{F}_{\ell,i}} \|\mathbf{x} - \mathbf{u}\|_2 .$$

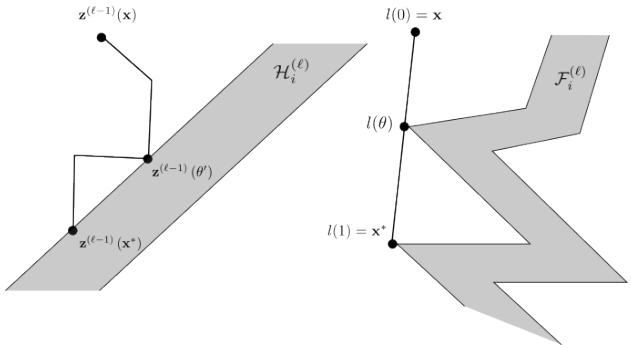
Let $l : [0, 1] \to \mathbb{R}^d$ be given by

$$l(\theta) = \mathbf{x}^* \theta + (1 - \theta)\mathbf{x}.$$

In the input space of layer ℓ, the line l becomes the piecewise affine parametric line

$$\mathbf{z}^{(\ell-1)}(\theta) = (f_{\ell-1} \circ \cdots \circ f_1)\,(l(\theta)).$$

Now if there exists $\theta' < 1$ such that $\mathbf{z}^{(\ell-1)}(\theta') \in \mathcal{H}_i^{(\ell)}$ then there exists $\theta < 1$ such that $l(\theta) \in \mathcal{F}_i^{(\ell)}$, Figure 7.2. In other words, if the



hyperplane $\mathcal{H}_i^{(\ell)}$ is bought closer to $\mathbf{z}^{(\ell-1)}(\mathbf{x})$ within the input space of layer ℓ, then $\mathcal{F}_i^{(\ell)}$ is bought closer to $\mathbf{x}$. □

That is, translating hyperplanes closer to $\mathbf{z}^{(\ell)}$ in the input space of layer ℓ, moves the folded hyperplane closer to $\mathbf{x}$ in the input space of the deep network.

Corollary 7.2.7. *Consider the setting of Theorem 7.2.6. Then $\mathbf{z}^{(\ell)}(\mathbf{x})$ lies on the hyperplane $\mathcal{H}_i^{(\ell)}$ for some i if and only if $\mathbf{x}$ lies on the corresponding folder hyperplane $\mathcal{F}_i^{(\ell)}$ in the input space of the deep network.*

The Effect of $\sigma^{(\ell)}$. The parameter $\sigma^{(\ell)}$ optimises the dihedral angles between adjacent facets of the folded hyperplanes, to direct them toward the training data. For an input $\mathbf{z}$, let $Q^{(\ell)}$ be the square diagonal matrix with entries

$$\left[Q^{(\ell)}\right]_{i,i} = \begin{cases} \frac{\alpha}{\left[\sigma^{(\ell)}\right]_i} & \left[\mathbf{h}^{(\ell)}\right]_i < 0 \\ \frac{1}{\left[\sigma^{(\ell)}\right]_i} & \left[\mathbf{h}^{(\ell)}\right]_i \geq 0, \end{cases}$$

where $\mathbf{h}^{(\ell)}$ is the pre-activation of $\mathbf{z}$, and α is a parameter dependent on the activation function operating on layer ℓ.[7]

[7] For ReLU we set $\alpha = 0$, for leaky ReLU we set $\alpha = -1$, for absolute value we set $\alpha = -1$.

The matrix $Q^{(\ell)}$ is constant across each region $\omega \in \Omega^{(\ell)}$. We will note this dependency with $Q^{(\ell)}(\omega)$.

Theorem 7.2.8. *Consider a two-layer batch normalised equipped deep network employing a leaky-ReLU or absolute value activation function. Let $\omega, \omega' \in \Omega$ be adjacent regions whose boundaries contain the facets $\mathcal{F}_{2,i,\omega}$ and $\mathcal{F}_{2,i,\omega'}$, which are created from folding across the shared hyperplane boundary $\mathcal{H}_i^{(1)}$. The dihedral angles between these hyperplanes are*

$$\begin{cases} \theta\left(\mathcal{F}_{i,\omega}^{(2)}, \mathcal{H}_j^{(1)}\right) = \arccos\left(\frac{\left|\left[W^{(2)}\right]_{i,\cdot}^\top Q^{(1)}(\omega) W^{(1)} \left[W^{(1)}\right]_{j,\cdot}\right|}{\left\|\left[W^{(2)}\right]_{i,\cdot}^\top Q^{(1)}(\omega) W^{(1)}\right\| \left\|\left[W^{(1)}\right]_{j,\cdot}\right\|}\right) \\ \theta\left(\mathcal{F}_{i,\omega'}^{(2)}, \mathcal{H}_j^{(1)}\right) = \arccos\left(\frac{\left|\left[W^{(2)}\right]_{i,\cdot}^\top Q^{(1)}(\omega') W^{(1)} \left[W^{(1)}\right]_{j,\cdot}\right|}{\left\|\left[W^{(2)}\right]_{i,\cdot}^\top Q^{(1)}(\omega') W^{(1)}\right\| \left\|\left[W^{(1)}\right]_{j,\cdot}\right\|}\right) \\ \theta\left(\mathcal{F}_{i,\omega}^{(2)}, \mathcal{F}_{i,\omega'}^{(2)}\right) = \arccos\left(\frac{\left|\left[W^{(2)}\right]_{i,\cdot}^\top Q^{(1)}(\omega) W^{(1)} \left(W^{(1)}\right)^\top Q^{(1)}(\omega') \left[W^{(2)}\right]_{i,\cdot}\right|}{\left\|\left[W^{(2)}\right]_{i,\cdot}^\top Q^{(1)}(\omega) W^{(1)}\right\| \left\|\left[W^{(2)}\right]_{i,\cdot}^\top Q^{(1)}(\omega') W^{(1)}\right\|}\right). \end{cases}$$

Remark 7.2.9. *Note that $Q^{(1)}(\omega)$ and $Q^{(1)}(\omega')$ differ only in the (i,i) entry where one takes the value $\frac{1}{\left[\sigma^{(1)}\right]}$ and the other $\frac{\alpha}{\left[\sigma^{(1)}\right]}$. Furthermore, recall that from (7.1) we have $\left[\sigma^{(\ell)}\right]_i^2 \propto \min_{\mu^{(\ell)},i} \mathcal{L}\left(\left[\mu^{(\ell)}\right]_i; \mathcal{B}^{(\ell)}\right)$. Therefore, using Theorem 7.2.8 we yield the following insights.*

- *If $\mathcal{H}_i^{(1)}$ is well-aligned with the training data $\mathcal{B}^{(1)}$, then the total least squares error, and hence $\left[\sigma^{(1)}\right]_i^2$, will be small. For the absolute value activation function it follows that $\theta\left(\mathcal{F}_{i,\omega}^{(2)}, \mathcal{H}_i^{(1)}\right) \approx 0$ and $\theta\left(\mathcal{F}_{i,\omega'}^{(2)}, \mathcal{H}_i^{(1)}\right) \approx 0$, meaning $\mathcal{F}_{i,\omega}^{(2)}$ and $\mathcal{F}_{i,\omega'}^{(2)}$ are folded to closely align with $\mathcal{H}_i^{(1)}$ and thus the data.*
- *If $\mathcal{H}_i^{(1)}$ is not well-aligned with the training data $\mathcal{B}^{(1)}$, then the total least squares error, and hence $\left[\sigma^{(1)}\right]_i^2$, will be large. Forcing $Q^{(1)}(\omega) \approx Q^{(1)}(\omega')$, making $\theta\left(\mathcal{F}_{i,\omega}^{(2)}, \mathcal{F}_{i,\omega'}^{(2)}\right) \approx \pi$, meaning that $\mathcal{H}_i^{(1)}$ will not appreciably fold intersection the layer two facets.*

Initialization Scheme

In the case of binary classification, we can concretely intuit on the effect of batch normalisation.

Proposition 7.2.10. *Consider a L-layer deep network configured as a binary classifier on training data $\mathcal{X}$ using a leaky ReLU activation function at each layer, arbitrary weights $W^{(\ell)}$ at each layer, batch normalisation at layers $\ell = 1, \ldots, L-1$, and layer L an affine layer with zero bias. Then for any training mini-batch from $\mathcal{X}$, there will be at least one data point on either side of the decision boundary.*

Proof. Each mini-batch input to the last layer will have one input whose entries contain positive and negative values. Hence each dimension will have at least one negative value with the other being positive, or vice-versa. Since the last layer has zero bias, the decision boundary is the hyperplanes of zero level sets of each output unit. Since being on one side of the decision boundary in the deep network's input space is equivalent to being on one side of the linear decision boundary in the last layer's input space, there has to be at least one sample on one side of the decision boundary compared to the other samples. □

In the setting of Proposition 7.2.10, batch normalisation primes learning through stochastic gradient descent by ensuring the decision boundary passes through every mini-batch.

Jitter

The parameters $\mu^{(\ell)}$ and $\sigma^{(\ell)}$ can be thought of as stochastic estimators to $\bar{\mu}^{(\ell)}$ and $\bar{\sigma}^{(\ell)}$ respectively.

Proposition 7.2.11. *Consider layer $\ell > 1$ of a batch normalised deep network. Assume that inputs $\mathbf{z}^{(\ell-1)}$ are independent and identically distributed, with the distribution having zero mean and covariance matrix $\mathrm{diag}(\mathbf{m}\rho)$. Then*

$$\mathrm{var}\left(\left[\mu^{(\ell)}\right]_i\right) = \frac{\left\langle \left[W^{(\ell)}\right]^2_{i,\cdot}, \mathbf{m}\rho \right\rangle}{\left|\mathcal{B}^{(\ell)}\right|}$$

and

$$\mathrm{var}\left(\left[\sigma^{(\ell)}\right]^2_i\right) = \frac{1}{\left|\mathcal{B}^{(\ell)}\right|}\left(\left[\phi^{(\ell)}\right]_i - \frac{\left\langle \left[W^{(\ell)}\right]^2_i, \mathbf{m}\rho \right\rangle \left(\left|\mathcal{B}^{(\ell)}\right| - 3\right)}{\left|\mathcal{B}^{(\ell)}\right| - 1}\right),$$

where $\left[W^{(\ell)}\right]^2_i$ is coordinate-wise and $\left[\phi^{(\ell)}\right]_i$ is the fourth-order central moment of $\left\langle \left[W^{(\ell)}\right]_i, \mathbf{z}^{(\ell)} \right\rangle$.

Proposition 7.2.11 formalises the intuition that batch normalisation introduces noise into the learning process.

7.3 *Layer Normalisation*

7.4 *References*

Section 7.2 from Balestriero and Baraniuk (2022).

8 Optimization

8.1 Introduction

In this chapter we seek to understand how the learning process of deep networks effects the functional geometry of the deep network.

9 Data Distribution

9.1 Introduction

From previous chapters, we have developed a sense of how the training data of a deep network can influence its underlying functional geometry, either through normalisation operators or the optimization procedure. In this chapter we aim to develop this question further by exploring what can be inferred about the data distribution of a training set from the geometry of the deep network. More specifically, we investigate the geometries necessary for capturing certain distributional features in a training set.

Part II

The Practical Implications

10 Visualisation

10.1 Introduction

In this chapter we develop the necessary results to construct methods for exactly computing visualisations of a deep network's geometry. Ultimately, we arrive at a computational algorithm for exactly depicting the induced partition of a deep network on a two-dimensional slice of its input domain.

10.2 Exact Computation of Partition Boundaries

Throughout these sections we are assuming that the piecewise affine non-linearity of the deep network is either ReLU, leaky ReLU or absolute value.

Recall Proposition 4.6.4, Proposition 4.6.5, and the notation outlined in statement 3 of Remark 4.6.6, so that for a deep network f we have $A_\omega^{(\ell)}$ and $b^{(\ell)}$ as

$$\begin{cases} A_\omega^{(1\leftarrow\ell)} = \prod_{i=1}^{\ell} Q^{(i)}[\omega]W^{(i)} \\ b_\omega^{(1\leftarrow\ell)} = Q^{(\ell)}[\omega]b^{(\ell)} + \sum_{i=1}^{\ell-1}\left(\prod_{j=i+1}^{\ell} Q^{(j)}[\omega]W^{(j)}\right)Q^{(i)}[\omega]b^{(i)}, \end{cases} \tag{10.1}$$

where $q_\omega^{(\ell)}$, in this setting, is the point-wise derivative of σ at the pre-activation $W^{(\ell)}\mathbf{z}^{(\ell-1)} + b^{(\ell)}$ for $\omega \in \Omega$.

Furthermore, recall that at layer ℓ of a deep network, we let

$$\mathcal{H}_i^{(\ell)} = \left\{\mathbf{z}^{(\ell-1)} \in \mathbb{R}^{d^{(\ell-1)}} : \left\langle [W^{(\ell)}]_{i,\cdot}, \mathbf{z}^{(\ell-1)}\right\rangle + \left[b^{(\ell)}\right]_i = 0\right\}.$$

Lemma 10.2.1. *The hyperplane $\mathcal{H}_i^{(\ell)} \subseteq \mathbb{R}^{d^{(\ell-1)}}$ can be projected onto the tangent space of a region $\omega \in \Omega \subseteq \mathbb{R}^d$ as*

$$\mathrm{proj}_\omega\left(\mathcal{H}_i^{(\ell)}\right) = \left\{\mathbf{x} \in \mathbb{R}^d : \left\langle \left[W^{(\ell)}\right]_{i,\cdot}, A_\omega^{(\ell-1)}\mathbf{x} + b_\omega^{(\ell-1)}\right\rangle + \left[b^{(\ell)}\right]_i = 0\right\}$$

Proof. This follows as

$$\mathbf{z}^{(\ell-1)} = A_\omega^{(\ell-1)}\mathbf{x} + b_\omega^{(\ell-1)}$$

for $\mathbf{x} \in \omega$. □

Theorem 10.2.2. *Consider a binary classification deep network, that is $d^{(L)} = 1$. In $\mathbb{R}^{d^{(L-1)}}$, the decision boundary is the hyperplane $\mathcal{H}_i^{(L)}$. Therefore, in $\mathbb{R}^d$ the decision boundary is*

$$\bigcup_{\omega \in \Omega} \text{proj}_\omega \left(\mathcal{H}_i^{(L)} \right) \cap \omega.$$

Proof. This follows by applying Lemma 10.2.1 to each $\omega \in \Omega$. □

10.3 SplineCam

SplineCam is an algorithm to perform that exact computation of the partition boundary of a deep network on two-dimensional subspaces of the input space.

Recall that $\Omega^{(1 \leftarrow \ell)}$ is the partition of the input space induced by the layers $1, \ldots, \ell$ of a deep network. The SplineCam constitutes Algorithm 1 and Algorithm 2, and can be summarised as follows.

- Determine a two-polytope subspace $P \subseteq \mathbb{R}^d$.
- Using Algorithm 1, partition P into $\Omega^{(1)}$ using the hyperplanes $\mathcal{H}_i^{(1)}$.
- For each $\omega \in \Omega^{(1)}$, use Lemma 10.1 and then Lemma 10.2.1 to obtain $\text{proj}_\omega \left(\mathcal{H}_i^{(2)} \right)$.
- Using Algorithm 1, partition each $\omega \in \Omega^{(1)}$ through the hyperplanes $\text{proj}_\omega \left(\mathcal{H}_i^{(2)} \right)$ to arrive at $\Omega^{(1 \leftarrow 2)}$.
- Repeat for all layers up to $\ell = L$ to obtain the final partition.

Algorithm 1: Find the partitions of a two-polytope induced by the hyperplanes $\mathcal{H}$.

Require: P, $\mathcal{H}$.

Solve for

$$N = \left\{ H_i \cap \left(H_j \cup P_e \right) : H_i, H_j \in \mathcal{H} \text{ with } i \neq j \right\},$$

where P_e are the edges of P.

For each H_i, sort N and connect in a sequence to obtain edges E.

Obtain set of cycles C from the undirected graph $G = (E, N)$ using Algorithm 2.

Proposition 10.3.1. *Given a set of n intersecting hyperplanes and a two-polytope P, Algorithm 2 has an average case complexity of $O\left(n^2\right)$.*

Proof. The number of intersections and cycles induces by intersecting a polytope and hyperplanes is on the order less than or equal to $O\left(n^2\right)$. Therefore, the average case complexity of Algorithm 2 is also of the order $O\left(n^2\right)$. □

Require: $G = (E, N)$, $P_e \subseteq E$, $e_s \subseteq P_e$.
  Initialise $C = \emptyset$ and $E_q = \emptyset$.
  $G' \leftarrow$ bidirectional(G).
  Append e_s to E_q.
  Remove e_s from G'.
  while $E_q \neq \emptyset$ **do**
    Pop e from E_q.
    Remove e' from G'. ▷ e' is e with reversed direction.
    $E_c \leftarrow$ bfs (G', v_e, v_s). ▷ v_s, v_e are start, end vertices of e.
    Append e'_c to E_q for all $e_c \in E_c$ if $e_c \notin P_e$.
    Remove e_c from G' for all $e_c \in E_c$.
    Remove e'_c from G' for all $e_c \in E_c$ such that $e_c \subseteq P_e$.
    Append e' to E_c
    Append E_c to C
  end while

Algorithm 2: Compute the cycles in an undirected graph $G = (E, N)$.

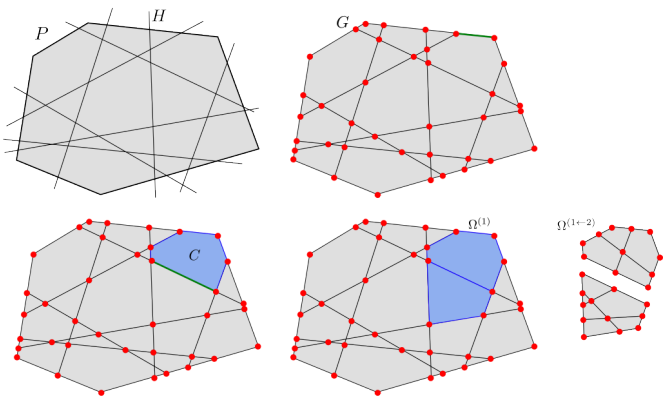


Figure 10.1: The SplineCam visualisation procedure starts by identifying a two-dimensional region of interest P and intersecting it with hyperplanes H, **upper left**. Then a graph G is constructed using the points of intersection as nodes and the hyperplanes and region of interest as edges; an edge is then selected, **upper right**. A cycle from the selected edge is generated using a breadth first search to identify edges which are then connected in an order; one of the identified edges is then selected, lower left. With the selected edge, another cycled is formed as before to continue the formation of $\Omega^{(1)}$, **lower middle**. Once all cycles are identified, the regions are considered individually and intersected with the hyperplanes from the second layer, and the process repeats, **lower right**.

10.4 *References*

Chapter 10 from Humayun et al. (2023a).

11 Complexity Measures

11.1 Introduction

Deep networks can become highly over-parameterized, in the classical sense, and thus can approximate even randomly generated training data. Despite this capacity, they are typically arrive at solutions that generalise to unseen data in a way that cannot be accounted for by memorising the training. The development of complexity measures aims at investigating this phenomena in computationally tractable ways; with the intention on illuminating deep network training beyond loss curves, or providing tools the regularise the training process. In this chapter we explore how the tools of Part I can be utilised to construct complexity measures for deep networks.

11.2 Local Complexity

Let $V = \left(\mathbf{v}_1, \ldots, \mathbf{v}_p\right)^\top$ be a matrix of vertices in a deep network's input space. Let $\mathcal{V} = \operatorname{conv}(V)$. Recall that the partition of the input space of layer ℓ can be expressed as the set of hyperplanes

$$\partial\Omega = \bigcup_{i=1}^{d^{(\ell)}} \mathcal{H}_i^{(\ell)},$$

where

$$\mathcal{H}_i^{(\ell)} = \left\{\mathbf{z} \in \mathbb{R}^{d^{(\ell-1)}} : \left\langle \left[W^{(\ell)}\right]_i, \mathbf{z}\right\rangle + \left[b^{(\ell)}\right]_i = 0\right\}.$$

One gauge the local complexity of $\mathcal{V}$ induced by layer ℓ by counting the number of regions in

$$\mathcal{V}^{(\ell)} \cap \partial\Omega = \bigcup_{i=1}^{d^{(\ell)}} \mathcal{V}^{(\ell)} \cap \mathcal{H}_i^{(\ell)},$$

where $\mathcal{V}^{(\ell)}$ is the embedded representation of $\mathcal{V}$ after layer $\ell - 1$ of the deep network. In particular, this will be on the order of $\mathcal{N}^{d^{(\ell-1)}}$, where

$$\mathcal{N} = \left|\left\{i : i = 1, \ldots, d^{(\ell)} \text{ and } \mathcal{H}_i^{(\ell)} \cap \mathcal{V}^{(\ell)} \neq \emptyset\right\}\right|,$$

which can thus be used as a proxy for the local complexity of $\mathcal{V}^{(\ell-1)}$. For practical tractability it is convenient to work with the convex hull of $V^{(\ell)}$[1] which we will denote $\hat{\mathcal{V}}^{(\ell)}$.

[1] The matrix $\left(\mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(p)}\right)^\top$, where $\mathbf{v}_i^{(\ell)}$ is the embedded representation of $\mathbf{v}_i$ after layer $\ell - 1$ of the deep network.

11.3 *Absolute Deviation*

Adaptive Linear Region Discovery

For a hyperplane $\mathcal{H}_i^{(\ell)} \subseteq \mathbb{R}^{d^{(\ell-1)}}$, suppose that $\mathbf{z}^{(\ell-1)}(\mathbf{x})$ lies on its negative side.[2]Then the smallest displacement $\lambda_j^{(\ell)}$ along a direction $\mathbf{d} \in \mathbb{R}^d$, that is non-zero and not lying in $\mathcal{H}_j^{(\ell)}$, required to cross $\mathcal{H}_j^{(\ell)}$ is

[2] If $\mathbf{z}^{(\ell-1)}(\mathbf{x})$ is on the positive side, flip the inequality and sign of $\lambda_j^{(\ell)}$ in (11.1).

$$\begin{aligned}\lambda_j^{(\ell)} &= \min_{\lambda \in \mathbb{R}} \left(\left[W^{(\ell)}\right]_{j,\cdot}^\top \left(\mathbf{z}^{(\ell-1)} + \lambda \mathbf{d}^{(\ell)}\right) + \left[b^{(\ell)}\right]_j > 0 \right) \\ &= \min_{\lambda \in \mathbb{R}} \left(\lambda > \frac{-\left[W^{(\ell)}\right]_{j,\cdot}^\top \mathbf{z}^{(\ell-1)} - \left[b^{(\ell)}\right]_j}{\left[W^{(\ell)}\right]_{j,\cdot}^\top \mathbf{d}^{(\ell)}} \right), \end{aligned} \tag{11.1}$$

where $\mathbf{d}^{(\ell)} \in \mathbb{R}^{d^{(\ell-1)}}$ is the direction $\mathbf{d}$ in the input space of layer ℓ, namely $\mathbf{d}^{(\ell)} = A^{(1\leftarrow\ell)}_{\omega_{\mathbf{s}^{(1\leftarrow\ell)}(\mathbf{x})}} \mathbf{d}$.

In practice, the condition $|\lambda| > \tau$ for a sensitivity parameter $0 < \tau \ll 1$ is added to (11.1), to ensure that a sufficient step is taken along $\mathbf{d}$ to traverse the boundary.

Algorithm 3: Minimum displacement finder.

Require: $\mathbf{x}$, $\mathbf{d}$
  for $\ell = 1, \ldots, L$ **do**
    $\lambda \leftarrow \infty$.
    lambdas $\leftarrow \emptyset$.
    lambdas $\leftarrow \frac{-b^{(\ell)} - W^{(\ell)}\mathbf{x}}{W^{(\ell)}\mathbf{d}}$. ▷ Division is element-wise.
    if min(lambdas) $< \lambda$ **then**
      $\lambda \leftarrow$ min(lambdas).
    end if
    $\mathbf{x} \leftarrow A^{(\ell)}_{\omega_{\mathbf{s}^{(\ell)}(\mathbf{x})}} \mathbf{x} + b^{(\ell)}_{\omega_{\mathbf{s}^{(\ell)}(\mathbf{x})}}$.
    $\mathbf{d} \leftarrow A^{(\ell)}_{\omega_{\mathbf{s}^{(\ell)}(\mathbf{x})}} \mathbf{d}$.
  end for
  return lambdas.

Given a $\mathbf{x} \in \mathbb{R}^d$ and a direction $\mathbf{d}$, the smallest displacement λ in direction $\mathbf{d}$ to a region is given by iteratively solving (11.1) for each layer, namely

$$\lambda = \min_{1 \leq \ell \leq L, 1 \leq j \leq d^{(\ell-1)}} \lambda_j^{(\ell)},$$

as summarised in Algorithm 3.

Iterating this procedure, it is possible to discover all regions along a path $\boldsymbol{\pi} : [0,1] \to \mathbb{R}^d$ given by $\boldsymbol{\pi}(t) = \mathbf{x} + t\mathbf{d}$. Since by the convexity of the regions, they induce a partition

$$\mathcal{P} = \{t \in [0,1] : 0 = t_0 < \cdots < t_n = 1\}. \tag{11.2}$$

Letting $\mathcal{J}_i = [t_i, t_{i+1}]$, it follows that with $\tau < \min_{i=1,\ldots,n} |\mathcal{J}_i|$ Algorithm 4 recovers all the regions, otherwise it recovers the regions with size larger than τ.

Algorithm 4: Adaptive linear region discovery on compact one-dimensional domains.

Require: $\mathbf{x}_0, \mathbf{x}_1, \tau$
  $\mathbf{c} \leftarrow \mathbf{s}(f(\mathbf{x}_0))$. ▷ Computed element-wise.
  $\mathbf{c}_1 \leftarrow \mathbf{s}(f(\mathbf{x}_1))$.
  $\mathbf{d} \leftarrow \frac{\mathbf{x}_1 - \mathbf{x}_0}{\|\mathbf{x}_1 - \mathbf{x}_0\|_2}$.
  $\mathcal{P} \leftarrow \emptyset$.
  while $\mathbf{c} \neq \mathbf{c}_1$ **do**
    $\lambda \leftarrow$ Algorithm 3$(\mathbf{x}, \mathbf{d})$.
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{\lambda\}$.
    if $\lambda = \infty$ **then**
      break
    end if
    $\mathbf{x} \leftarrow \mathbf{x} + \lambda \mathbf{d}$.
    if $\|\mathbf{x}_0 - \mathbf{x}_1\|_2 < \|\mathbf{x} - \mathbf{x}_0\|_2$ **then**
      break
    end if
    $\mathbf{c} \leftarrow \mathbf{s}(f(\mathbf{x}))$.
  end while
  return $\mathcal{P}$.

Quantifying Nonlinearity

For distinct points $\mathbf{x}_0, \mathbf{x}_1 \in \mathbb{R}^d$ let g be an interpolation of $f_{\omega_{\mathbf{s}(\mathbf{x}_0)}}$ and $f_{\omega_{\mathbf{s}(\mathbf{x}_1)}}$ such that

$$g(\mathbf{x}_0 + t(\mathbf{x}_1 - \mathbf{x}_0)) = f(\mathbf{x}_0) + t(f(\mathbf{x}_1) - f(\mathbf{x}_0)).$$

Definition 11.3.1. *The absolute deviation between distinct points* $\mathbf{x}_0, \mathbf{x}_1 \in \mathbb{R}^d$ *is*

$$\mathbf{a}(\mathbf{x}_0, \mathbf{x}_1) = \int_{\pi} |f(\mathbf{x}) - g(\mathbf{x})| \, \mathrm{d}\mathbf{x},$$

where $\boldsymbol{\pi} : [0,1] \to \mathbb{R}^d$ *is given by* $\boldsymbol{\pi}(t) = \mathbf{x}_0 + t(\mathbf{x}_1 - \mathbf{x}_0)$.[3]

[3] In practice, often the ℓ_2 norm of $\mathbf{a}(\mathbf{x}_0, \mathbf{x}_1)$ is considered instead as a scalar score.

Proposition 11.3.2. *Recall that a deep network's regions induces a partition*

Figure 11.1: An illustration of what our notation for absolute deviation means in the context deep networks with a one-dimensional input domain. The region shaded in green represents the absolute deviation.

on $[0,1]$ (11.2). *For* $j = 1, \ldots, d^{(L)}$ *we have*

$$\begin{aligned}\int_{\pi} \left|[f(\mathbf{x})]_j - [\mathbf{a}(\mathbf{x})]_j\right| \, \mathrm{d}\mathbf{x} = \|\mathbf{d}\| \Bigg(& \sum_{i=0}^{n-1} (t_\star - t_i) \left|[f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_0}(\mathbf{x}_0)]_j\right| \\ & + (t_{i+1} - t_\star) \left|[f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_0}(\mathbf{x}_0)]_j\right| \\ & + \frac{t_\star^2 - t_i^2}{2} \left|[f_{\omega_i}(\mathbf{x}_1)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_1}(\mathbf{x}_1)]_j + [f_{\omega_0}(\mathbf{x}_0)]_j\right| \\ & + \frac{t_{i+1}^2 - t_\star^2}{2} \left|[f_{\omega_i}(\mathbf{x}_1)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_1}(\mathbf{x}_1)]_j + [f_{\omega_0}(\mathbf{x}_0)]_j\right| \Bigg),\end{aligned}$$

where $t_\star = \max(t_i, \min(z_i, t_{i+1}))$ *for*

$$z_i = \frac{[f_{\omega_0}(\mathbf{x}_0)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j}{[f_{\omega_i}(\mathbf{x}_1)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_1}(\mathbf{x}_1)]_j - [f_{\omega_0}(\mathbf{x}_0)]_j},$$

and where f_{ω_i} *denotes the affine function of the deep network operating on the region corresponding to* t_i *in* $\mathcal{P}$.

Proof. Initially, we obtain that

$$\begin{aligned}&\int_{\pi} \left|[f(\mathbf{x})]_j - [\mathbf{a}(\mathbf{x})]_j\right| \, \mathrm{d}\mathbf{x} \\ &= \sum_{i=0}^{n-1} \int_{t_i}^{t_{i+1}} \left|[f(\boldsymbol{\pi}(t))]_j - [\mathbf{a}(\boldsymbol{\pi}(t))]_j\right| \|\dot{\boldsymbol{\pi}}(t)\| \, \mathrm{d}t \\ &= \sum_{i=0}^{n-1} \int_{t_i}^{t_{i+1}} \left|[f(\mathbf{x}_0 + t\mathbf{d})]_j - [\mathbf{a}(\mathbf{x}_0 + t\mathbf{d})]_j\right| \|\mathbf{d}\| \, \mathrm{d}t \\ &= \|\mathbf{d}\| \sum_{i=0}^{n-1} \int_{t_i}^{t_{i+1}} \left|[f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_0}(\mathbf{x}_0)]_j + t\left([f_{\omega_i}(\mathbf{x}_1)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_1}(\mathbf{x}_1)]_j + [f_{\omega_0}(\mathbf{x}_0)]_j\right)\right| \, \mathrm{d}t.\end{aligned}$$

Now the integrand is not necessarily monotonic, in particular, on ω_i the functions $[f]_j$ and $[\mathbf{a}]$ may intersect at

$$z_i = \frac{[f_{\omega_0}(\mathbf{x}_0)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j}{[f_{\omega_i}(\mathbf{x}_1)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_1}(\mathbf{x}_1)]_j - [f_{\omega_0}(\mathbf{x}_0)]_j}.$$

If $z_i \notin (t_i, t_{i+1})$, then we need not worry about this intersection for the computation of the integral, otherwise we can separate the integral across the domains $[t_i, z_i]$ and $[z_i, t_{i+1}]$. In any case, we can take $t_\star = \max(t_i, \min(z_i, t_{i+1}))$ and consider the integral over the domains $[t_i, t_\star]$ and $[t_\star, t_{i+1}]$ on which our integrand will be affine.

Therefore,

$$\begin{aligned}
&\int_{\pi} \left| [f(\mathbf{x})]_j - [\mathbf{a}(\mathbf{x})]_j \right| \mathrm{d}\mathbf{x} \\
&\quad = \|\mathbf{d}\| \sum_{i=0}^{n-1} \int_{t_i}^{t_\star} \left| [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_0}(\mathbf{x}_0)]_j + t \left([f_{\omega_i}(\mathbf{x}_1)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_1}(\mathbf{x}_1)]_j + [f_{\omega_0}(\mathbf{x}_0)]_j \right) \right| \mathrm{d}t \\
&\quad + \|\mathbf{d}\| \sum_{i=0}^{n-1} \int_{t_\star}^{t_{i+1}} \left| [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_0}(\mathbf{x}_0)]_j + t \left([f_{\omega_i}(\mathbf{x}_1)]_j - [f_{\omega_i}(\mathbf{x}_0)]_j - [f_{\omega_1}(\mathbf{x}_1)]_j + [f_{\omega_0}(\mathbf{x}_0)]_j \right) \right| \mathrm{d}t,
\end{aligned}$$

which simplifies to the desired expression. □

11.4 Hessian Based Rugosity Measure

Throughout this section we'll be consider a deep network $f : \mathbb{R}^d \to \mathbb{R}$ and a set of training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$ for $i = 1, \ldots, n$. In particular, we assume that the points $\{\mathbf{x}_i\}_{i=1}^n$ lie close to an m-dimensional manifold $\mathcal{M} \subseteq \mathbb{R}^d$.

Smooth Activation Deep Networks

Suppose that f employs smooth activation operators. For $p \geq 1$ let

$$C_p(f) := \left(\int_{\mathcal{M}} \left\| \nabla^2_{\tan} f(\mathbf{x}) \right\|_F^p \mathrm{d}\mathbf{x} \right)^{\frac{1}{p}},$$

where $\nabla^2_{\tan} f(\mathbf{x})$ is the Hessian of f at $\mathbf{x}$ in the coordinates of the m-dimensional affine space tangent to the manifold at $\mathbf{x} \in \mathcal{M}$.

The quantity $C_p(f)$ measure the average *curviness* of f over $\mathcal{M}$. More specifically, it is zero if and only if f is an affine function on $\mathcal{M}$.

As f is smooth, we have

$$\nabla^2 f(\mathbf{x})\mathbf{u} = \lim_{\epsilon \to 0} \left(\frac{\nabla f(\mathbf{x} + \epsilon \mathbf{u}) - \nabla f(\mathbf{x})}{\epsilon} \right).$$

Moreover, for $\mathbf{u} \in \mathbb{R}^d$ chosen uniformly at random on the unit sphere, we have

$$\mathbb{E}\left(\|A\mathbf{u}\|_2^2 \right) = \mathbb{E}\left(\sum_{i,j=1,\ldots,d} [\mathbf{u}]_i [\mathbf{u}]_j \left[A^\top A \right]_{ij} \right) = \frac{\|A\|_F^2}{d},$$

for any $A \in \mathbb{R}^{d \times d}$. Therefore, we can obtain a Monte Carlo approximation for $C_p(f)$ on the training data as

$$\tilde{C}_{p,\epsilon}(f) = \left(\frac{d^{\frac{p}{2}}}{n \epsilon^p \hat{n}^{\frac{p}{2}}} \sum_{i=1}^n \left(\sum_{j=1}^{\hat{n}} \left\| \nabla f\left(\mathbf{x}_i + \epsilon \mathbf{u}_j\right) - \nabla f\left(\mathbf{x}_i\right) \right\|_2^2 \right) \right)^{\frac{1}{p}},$$

where $\epsilon > 0$ is small and the $\mathbf{u}_j$ are uniform samples of unit vectors on the m-dimensional subspace tangent to $\mathcal{M}$ at $\mathbf{x}$. This just follows from the observation that

Continuous Piecewise Affine Deep Networks

For f employing piecewise affine activation operators, it can be characterised as a continuous piecewise affine operator. In particular, recall the notation convention from statement 2 of Remark 2.2.2, we have that $f(\mathbf{x}) = A[\mathbf{x}]\mathbf{x} + b[\mathbf{x}]$, where $A[\mathbf{x}]$ and $b[\mathbf{x}]$ are the parameters for the affine function on the region encompassing $\mathbf{x}$.

The Hessian for f is not well-defined, however, we can extend a definition of the Hessian to encompass this case. Let $\epsilon > 0$. Then for $\mathbf{x} \in \mathbb{R}^d$ not on the boundary of a partition and a unit vector $\mathbf{u}$, let

$$H^{\epsilon}_{\mathbf{u}}(\mathbf{x}) = \begin{cases} \frac{A[\mathbf{x}+\epsilon\mathbf{u}]-A[\mathbf{x}]}{\epsilon} & \mathbf{u} \text{ in tangent manifold of } \mathcal{M} \text{ at } \mathbf{x} \\ \mathbf{0} & \text{otherwise.} \end{cases}$$

With this, for $p \geq 1$, let

$$C_{p,\epsilon} := \sqrt{d}\left(\int_{\mathcal{M}}\left(\mathbb{E}_{\mathbf{u}}\left(\|H^{\epsilon}_{\mathbf{u}}(\mathbf{x})\|_2^2\right)\right)^{\frac{p}{2}}\right)^{\frac{1}{p}},$$

where $\mathbf{u}$ is uniform over the unit sphere.

A Monte Carlo approximation for $C_{p,\epsilon}(f)$ on the training data can be obtained as

$$\tilde{C}_{p,\epsilon}(f) = \left(\frac{d^{\frac{p}{2}}}{n\epsilon^p \hat{n}^{\frac{p}{2}}}\sum_{i=1}^{n}\left(\sum_{j=1}^{\hat{n}}\left\|A\left[\mathbf{x}_i + \epsilon\mathbf{u}_j\right] - A\left[\mathbf{x}_i\right]\right\|_2^2\right)\right)^{\frac{1}{p}}, \quad (11.3)$$

where the $\mathbf{u}_j$ are uniform samples of unit vectors on the m-dimensional subspace tangent to $\mathcal{M}$ at $\mathbf{x}$.

Connection to Data Augmentation

The process of data augmentation takes the original dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ and generates the augmented dataset $\left\{\left\{\left(\mathbf{x}_i + \mathbf{u}_{ij}, y_i\right)\right\}_{j=0}^{\hat{n}}\right\}_{i=1}^{n}$ by applying transformations to the points $\mathbf{x}_i$ such that they continue to lie on $\mathcal{M}$. In this case, we are using $\mathbf{u}_{i0}$ to represent the identity transformation on $\mathbf{x}_i$. Training f on the original dataset involves minimizing a loss

$$\mathcal{L} := \sum_{i=1}^{\ell}\left(f\left(\mathbf{x}_i\right), y_i\right),$$

where we are assuming ℓ is convex. On the augmented dataset the corresponding loss is

$$\mathcal{L}_{\text{aug}} := \frac{1}{\hat{n}+1} \sum_{i=1}^{n} \left(\ell\left(f\left(\mathbf{x}_i\right), y_i\right) + \sum_{j=1}^{\hat{n}} \ell\left(f\left(\mathbf{x}_i + \mathbf{u}_{ij}\right), y_i\right) \right).$$

Theorem 11.4.1. *Let f be a deep network that is a continuous piecewise affine operator. Assume that $\|\mathbf{x}_i\|_2 \leq R$, that f has Lipschitz constant K_1, and that ℓ has Lipschitz constant K_2. Then for $\left\|\mathbf{u}_{ij}\right\|_2 \leq \epsilon$ we have*

$$\begin{aligned}
\mathcal{L}_{aug} \leq \mathcal{L} &+ \frac{RK_2}{\hat{n}+1} \underbrace{\sum_{i=1}^{n} \sum_{j=1}^{\hat{n}} \left\|A\left[\mathbf{x}_i + u_{ij}\right] - A\left[\mathbf{x}_i\right]\right\|_2}_{\hat{\mathcal{C}}(f)} \\
&+ \frac{K_2}{\hat{n}+1} \sum_{i=1}^{n} \sum_{j=1}^{\hat{n}} \left|b\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - b\left[\mathbf{x}_i\right]\right| \\
&+ \frac{K_1 K_2 \hat{n} n \epsilon}{\hat{n}+1} + o(\epsilon). \qquad (11.4)
\end{aligned}$$

Proof. Observe that

$$\begin{aligned}
\ell\left(f\left(\mathbf{x}_i + \mathbf{u}_{ij}\right), y_i\right) &= \ell\left(A\left[\mathbf{x}_i + \mathbf{u}_{ij}\right]\left(\mathbf{x}_i + \mathbf{u}_{ij}\right) + b\left[\mathbf{x}_i + \mathbf{u}_{ij}\right], y_i\right) \\
&= \ell\big(A\left[\mathbf{x}_i\right]\mathbf{x}_i + b\left[\mathbf{x}_i\right] + \left(A\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - A\left[\mathbf{x}_i\right]\right)\mathbf{x}_i \\
&\quad + \left(b\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - b\left[\mathbf{x}_i\right]\right) + A\left[\mathbf{x}_i + \mathbf{u}_{ij}\right]\mathbf{u}_{ij}, y_i\big).
\end{aligned}$$

Therefore, the first-order approximation of $\ell\left(f\left(\mathbf{x}_i + \mathbf{u}_{ij}\right), y_i\right)$ around $\ell\left(f\left(\mathbf{x}_i r\right), y_i\right)$ is

$$\begin{aligned}
\tilde{\ell}\left(f\left(\mathbf{x}_i + \mathbf{u}_{ij}\right), y_i\right) =& \ell\left(f\left(\mathbf{x}_i r\right), y_i\right) \\
&+ \big(\left(A\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - A\left[\mathbf{x}_i\right]\right)\mathbf{x}_i \\
&\quad + \left(b\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - b\left[\mathbf{x}_i\right]\right) + A\left[\mathbf{x}_i + \mathbf{u}_{ij}\right]\mathbf{u}_{ij}\big)^{\top} \nabla_{f(\mathbf{x}_i)} \ell\left(f\left(\mathbf{x}_i\right), y_i\right).
\end{aligned}$$

Hence,

$$\begin{aligned}
\tilde{\ell}\left(f\left(\mathbf{x}_i + \mathbf{u}_{ij}\right), y_i\right) \leq& \ell\left(f\left(\mathbf{x}_i r\right), y_i\right) + RK_2 \left\|A\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - A\left[\mathbf{x}_i\right]\right\|_2 \\
&+ K_2 \left|b\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - b\left[\mathbf{x}_i\right]\right| + \epsilon K_1 K_2.
\end{aligned}$$

Therefore,

$$\begin{aligned}
\tilde{\mathcal{L}}_{\text{aug}} &= \frac{1}{\hat{n}+1} \sum_{i=1}^{n} \left(\ell\left(f\left(\mathbf{x}_i r\right), y_i\right) + \sum_{j=1}^{\hat{n}} \tilde{\ell}\left(f\left(\mathbf{x}_i + \mathbf{u}_{ij}\right), y_i\right) \right) \\
&\leq \mathcal{L} + \frac{RK_2}{\hat{n}+1} \sum_{i=1}^{n} \sum_{j=1}^{\hat{n}} \left\|A\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - A\left[\mathbf{x}_i\right]\right\|_2 \\
&\quad + \frac{K_2}{\hat{n}+1} \sum_{i=1}^{n} \sum_{j=1}^{\hat{n}} \left|b\left[\mathbf{x}_i + \mathbf{u}_{ij}\right] - b\left[\mathbf{x}_i\right]\right| + \frac{K_1 K_2 n \hat{n} \epsilon}{\hat{n}+1},
\end{aligned}$$

completing the proof. □

Remark 11.4.2. *Note how for $p = 1$, (11.3) is very similar to $\hat{C}(f)$, up to a normalisation constant. Moreover, the data augmentations methods are such that $\mathbf{x}_i + \mathbf{u}_{ij} \in \mathcal{M}$. Thus, a sufficiently rich set of small augmentations will have similar distribution and support to the uniform distribution over unit vectors on the subspace tangent to the manifold, as is the case in (11.3). Therefore, the Hessian-based rugosity measure (11.3) could act as an explicit regularisation term in training that yields similar benefits as data augmentation, namely minimising $\mathcal{L}_{aug}$.*

11.5 References

Section 11.2 from Humayun et al. (2024). Section 11.3 from Gamba et al. (2022). Section 11.4 from LeJeune et al. (2019).

12 Constraining Optimization

12.1 Introduction

By understanding the functional geometry of deep networks and how this relates to their properties we can impose certain properties on deep networks by intervening on this geometry. More specifically, by imposing constraints in the optimization process, one can provide guarantees on the behaviour of a trained network by guaranteeing its functional geometry. Here we explore how this is done in practice and its application to ensuring the robustness of deep networks around training points.

12.2 POLICE

Deep networks struggle on optimization problems involving exact constraints, such as

$$\begin{cases} \min_\theta \|f_\theta - f^*\| + \lambda \mathcal{R}(\theta) \\ f_\theta(\mathbf{x}) \text{ affine on } R. \end{cases}$$

Let $V = \left(\mathbf{v}_1, \ldots, \mathbf{v}_p\right)^\top$ be a matrix of p vertices, such that

$$R \approx \left\{ V^\top \alpha : \alpha_j \geq 0 \text{ for all } j \text{ and } \alpha^\top \mathbf{1} = 1 \right\}. \tag{12.1}$$

Let $\mathbf{v}_j^{(\ell)}$ be the mapping of the jth vertex through layers 1 through $\ell - 1$, similarly for $V^{(\ell)}$ and with $\mathbf{v}_j$ corresponding to $\ell = 1$.

Henceforth, for simplicity, we will assume that (12.1) holds exactly.

Theorem 12.2.1. *A necessary and sufficient condition for a continuous-piecewise affine spline deep network to stay affine within a region R, is to have the same pre-activation sign patterns between the vertices $\mathbf{v}_j$ for every $j = 1, \ldots, p$ and each layer.*

Proof. If $\text{sign}\left(f^{(1)}(\mathbf{v}_i)\right) = \text{sign}\left(f^{(1)}(\mathbf{v}_j)\right)$ for all $i, j \in \{1, \ldots, p\}$, then the vertices all lie within the same partition region of $\Omega^{(1)}$. That is, there exists $\omega^* \in \Omega^{(1)}$ such that $v_j \in \omega^*$ for all $j \in \{1, \ldots, P\}$.

Since, ω^* is convex due to Corollary 6.5.3, and R is convex by construction, it follows that $R \subseteq \omega^*$. Repeating this for each layer, we arrive at the conclusion that the input-output mapping of the deep network as affine within R. □

Proposition 12.2.2. *A sufficient condition for layer ℓ to be linear within the input space region given by R is*

$$0 \leq \min_{(i,j)\in[p]\times\left[d^{(\ell)}\right]} H_{i,j} s_j, \tag{12.2}$$

where $H = V^{(\ell)}\left(W^{(\ell)}\right)^\top + \mathbf{1}\left(b^{(\ell)}\right)^\top$ *and*

$$s_j = \begin{cases} \left|\left\{i : H_{i,j} > 0\right\}\right| \geq \frac{p}{2} \\ -1 & \textit{otherwise.} \end{cases}$$

Proof. The matrix $H \in \mathbb{R}^{[p]\times d^{(\ell)}}$ is simply the pre-activation of layer ℓ given input $V^{(\ell)}$. Therefore, by Theorem 12.2.1, it is sufficient to ensure that the rows of H have the same sign pattern. Clearly enforcing (12.2) ensures this, with the construction of s_j meaning that we are enforcing the sign based on majority. □

Algorithm 5, known as POLICE, implements condition (12.2) at train time to satisfy the condition of Theorem 12.2.1 which ensures the affinity of the input-output mapping on R. One can similarly implement the condition at test time using Algorithm 6.

Algorithm 5: Provably Optimal Linear Constraint Enforcement. Train time.

Require: $X \in \mathbb{R}^{n\times d}$, $V \in \mathbb{R}^{m\times d}$
  $V^{(1)} \leftarrow V$.
  $X \leftarrow X$.
  for $\ell = 1, \ldots, L$ **do**
    $H \leftarrow V^{(\ell)}\left(W^{(\ell)}\right)^\top + \mathbf{1}\left(b^{(\ell)}\right)^\top$.
    $s = \text{MajorityVote}\left(\text{sign}(H), \text{axis} = 0\right)$
    $c = \text{ReLU}\left(-H\text{diag}(s)\right).\max(\text{axis} = 0) \odot s$.
    $X^{(\ell+1)} \leftarrow \sigma\left(X^{(\ell)}\left(W^{(\ell)}\right)^\top + \mathbf{1}\left(b^{(\ell)} + c\right)^\top\right)$.
    $V^{(\ell+1)} \leftarrow \sigma\left(V^{(\ell)}\left(W^{(\ell)}\right)^\top + \mathbf{1}\left(b^{(\ell)} + c\right)^\top\right)$.
  end for

12.3 *Instance Robustness*

Definition 12.3.1. *A binary classifier deep network f is ϵ-robust at $\mathbf{x} \in \mathbb{R}^d$ if*

$$\left\{\mathbf{x}' \in B_\epsilon(\mathbf{x}) : f\left(\mathbf{x}'\right) \neq f(\mathbf{x})\right\} = \emptyset.$$

Algorithm 6: Provably Optimal Linear Constraint Enforcement. Test time.

Require: $V \in \mathbb{R}^{m \times d}$
$V^{(1)} \leftarrow V$.
for $\ell = 1, \ldots, L$ **do**
$H \leftarrow V^{(\ell)} \left(W^{(\ell)}\right)^\top + \mathbf{1} \left(b^{(\ell)}\right)^\top$.
$s = \text{MajorityVote}\left(\text{sign}(H), \text{axis} = 0\right)$
$c = \text{ReLU}\left(-H\text{diag}(s)\right).\max(\text{axis} = 0) \odot s$.
$V^{(\ell+1)} \leftarrow \sigma\left(V^{(\ell)} \left(W^{(\ell)}\right)^\top + \mathbf{1}\left(b^{(\ell)} + c\right)^\top\right)$.
$b^{(\ell)} \leftarrow b^{(\ell)} + c$.
end for

Remark 12.3.2.

1. *Definition 12.3.1 is formally saying that the minimum distance of* $\mathbf{x}$ *from the decision boundary of* f *is lower bounded in* ℓ_2 *norm by* ϵ.

2. *Recall that*
$$B_\epsilon(\mathbf{x}) = \left\{\mathbf{x}' : \left\|\mathbf{x}' - \mathbf{x}\right\| \leq \epsilon\right\}.$$
Note that the norm $\|\cdot\|$ *in this instance is unspecified.*

Theorem 12.3.3. *Consider a binary classifier* f. *For* $\mathbf{x} \in \mathbb{R}^d$, *if there exists a* d*-polytope containing* $\mathbf{x}$ *with* $p \geq d$ *vertices* $V = \{\mathbf{v}_j\}_{j=1}^p$ *of which* d *form a basis, such that* $\left\|\mathbf{v}_j - \mathbf{x}\right\|_1 > \epsilon$ *for each* $\mathbf{v}_j \in V$ *and*

$$\left|\sum_{j=1}^p \text{sign}\left(\left\langle \left[W^{(L)}\right]_{1,\cdot}, f^{(\ell-1)}(\mathbf{v}_j)\right\rangle + \left[b^{(L)}\right]_1\right)\right| = p, \tag{12.3}$$

then f *is* ϵ*-robust in* ℓ_1 *norm at* $\mathbf{x}$.

Proof. By construction $B_\epsilon(\mathbf{x}) \subseteq \text{conv}(V)$. Moreover, (12.3) ensures that the pre-activation pattern between the vertices are the same, so by Theorem 12.2.1 we have that the output of the vertices, and in particular $\text{conv}(V)$, are on the same side of the decision boundary. Meaning that the decision $B_\epsilon(\mathbf{x})$ remains on the same side of the decision boundary through the mapping, and hence the deep network is ϵ robust in ℓ_1 norm. □

From Theorem 12.3.3 its follows that one can use Algorithm 5 to train a deep network to be ϵ-robust at a particular instance $\mathbf{x} \in \mathbb{R}^d$. By applying constraints to multiple samples and ensuring that the signs off all the constraints are the same, one can train a deep network to be ϵ-robust at multiple samples.

12.4 References

Section 12.2 from Balestriero and LeCun (2023). Section 12.3 from Humayun et al. (2023b).

13 Formal Verification

13.1 Introduction

In Chapter 12 we looked at enforcing properties of deep networks by constraining the optimization process. In this chapter we focus on the similar problem of deriving guarantees on the properties of deep networks by analysing the optimized artefact. For example, just as we previously ensured the predictions of deep networks in convex hulls around training points in Section 12.3, in this chapter we will show how to guarantee predictions in regions of the input space. This broad field of guaranteeing deep network behaviour through mathematical arguments is known as formal verification.

13.2 Predictions

Theorem 13.2.1. *Consider a binary classifier f and let $\omega \in \Omega$ have vertices $\{\mathbf{v}_1, \ldots, \mathbf{v}_p\}$. Then if*

$$\left| \sum_{j=1}^{p} \operatorname{sign}\left(\left\langle \left[W^{(L)}\right]_{1,\cdot}, f^{(\ell-1)}\left(\mathbf{v}_j\right) \right\rangle + \left[b^{(L)}\right]_1 \right) \right| = p,$$

then f makes the same classification prediction for every $\mathbf{x} \in \omega$.

Proof. **TODO!** □

Remark 13.2.2.

1. *Note the similarity between Theorem 13.2.1 and Theorem 12.3.3.*

2. *From Theorem 13.2.1, it follows that it suffices to understand the predictions of the deep network at the vertices of a given region to make inferences on the predictions it will make for all points within the region ω.*

TODO!

Typically, regions $\omega \in \Omega$ induced by $f : \mathbb{R}^d \to \mathbb{R}$ are bounded by on the order of d vertices. Although it seems as though we can guarantee the behaviour of a deep network by only checking its behaviour

Figure 13.1: The training data is depicted as scatter points, colour-coded according to the class that they belong too. The highlighted regions correspond to regions for which it can be guaranteed that the deep network predicts the class associated to that colour.

on a relatively few number of determinable points, computing the points is computationally expensive. However, as shown in Section 10.3, this computations are tractable when we restrict ourselves to a two-dimensional slice of the input domain. If the input domain is two-dimensional, then we would be able to make statements such as the robustness of instances in our input domain.

Example 13.2.3. *Suppose that we have a deep network $f : \mathbb{R}^2 \to \mathbb{R}$ comprising of four fully connected operators composed with ReLU nonlinearities. We can train this deep network to perform two-class classification on a set of training data. Since f is a continuous affine spline, it induces a partition of the input space. Let P be a two-dimensional polytope encompassing the training data. Then using Algorithm 2 we can obtain the partition and thus the vertices bounding the regions of Ω intersecting P. We can then use Theorem 13.2.1 to identify regions on which we can guarantee the predictions of our deep network, Figure 13.2. Of course, we can make P arbitrarily larger to improve the strength of the inferred guarantees. However, this comes with additional computational costs as more regions ω would intersect P, resulting in the effects outlined in Proposition 10.3.1. For practical purposes, formal verification only really needs to be able to provide guarantees on likely inputs to our deep network. Clearly from observing the data in Figure 13.2, it is clear that P, and the verified region, captures a significant proportion of training data distribution.*

In the case where our input domain has more than two-dimensions all is not lost, as certain properties of the input domain lie on two-dimensional subspaces; meaning we can verify predictions for specific properties in the input domain.

Example 13.2.4.

1. *Suppose that our deep network is receiving flattened images as its input. Then for an image $\mathbf{x}$, verifying predictions in the two-dimensional subspace spanned by $\mathbf{v}_1 := \frac{\mathbf{x}}{\|\mathbf{x}\|}$ and $\mathbf{v}_2 =$ gramschmidtvector corresponds to verifying the predictions of the deep network as the brightness of the image $\mathbf{x}$ is varied.*

2. *It has been shown that features of a deep network can correspond directions in input space of deep networks, Section 18.3. Therefore, verifying predictions in the two-dimensional spanned by these directions and an orthogonal direction corresponds to verifying the predictions of the deep network for inputs expressing this feature to various degrees.*

13.3 *References*

14 Pruning

14.1 Introduction

In the classical statical sense deep networks are over-parametrised, and thus their optimisation should lead to degenerate solutions to the task at hand. However, it is observed in practice that deep networks arrive at remarkably generalisable solutions, despite having the capacity to memorise their training. This suggests that their in some redundancy in their capacity. Using our developed theory we can reason about how to judiciously reduce, or prune, parameters of the deep network in a manner that maintains its performance. Initially we formulate autoencoders to introduce the framework in which generative models operate in; then we focus on the generative component.

14.2 Characterisation of Techniques

Consider the ℓth layer of a deep network

$$f^{(\ell)}(\mathbf{x}) = \sigma\left(W^{(\ell)} + b^{(\ell)}\right),$$

where σ is a leaky ReLU with parameter η. Recall the induced a partition $\Omega^{(\ell)}$ of $\mathbb{R}^{d^{(\ell-1)}}$ given by (4.5), along with the functional decomposition of the layer operator (4.6).

A pruning strategy is equivalent to applying a mask onto the layer's parameters,

$$\left(W^{(\ell)}, b^{(\ell)}\right) \mapsto \left(M^{(\ell)} \odot W^{(\ell)}, m^{(\ell)} \odot b^{(\ell)}\right),$$

where $M^{(\ell)} \in \{0,1\}^{d^{(\ell)} \times d^{(\ell-1)}}$ and $m^{(\ell)} \in \{0,1\}^{d^{(\ell)}}$.

Example 14.2.1.

1. *Unit pruning would employ $M^{(\ell)} = \tilde{m}^{(\ell)}\mathbf{1}^\top$ and $m^{(\ell)} = \tilde{m}^{(\ell)}$.*
2. *Parameter pruning would treat each entry of $M^{(\ell)}$ and $m^{(\ell)}$ independently.*

14.3 Impact on Partition

Proposition 14.3.1. *Setting any entry of $M^{(\ell)}$ or $m^{(\ell)}$ to zero impacts the per-region affine mappings $A_\omega^{(\ell)}$ and $b_\omega^{(\ell)}$, as well as the input space partition Ω.*

Proof. This follows from the dependency of $A_\omega^{(\ell)}$, $b_\omega^{(\ell)}$ and $\Omega^{(\ell)}$ on $W^{(\ell)}$ and $b^{(\ell)}$ highlighted in (4.5) and (4.6). □

Let $\left|\Omega^{(\ell)}; W^{(\ell)}, b^{(\ell)}\right|$ denote the number of regions in $\Omega^{(\ell)}$ when the layer is parametrized by $W^{(\ell)}$ and $b^{(\ell)}$.

Theorem 14.3.2. *We have*

$$\left|\Omega^{(\ell)}; M^{(\ell)} \odot W^{(\ell)}, m^{(\ell)} \odot b^{(\ell)}\right| \leq \left|\Omega^{(\ell)}; W^{(\ell)}, b^{(\ell)}\right| - \sum_{k=1}^{d^{(\ell)}} \mathbf{1}\left(\left\langle \left[M^{(\ell)}\right]_{k,\cdot}, \mathbf{1}\right\rangle = 0\right).$$

In particular, for unit pruning we have

$$\max_{W,b} \left|\Omega^{(\ell)}; M^{(\ell)} \odot W, m^{(\ell)} \odot b\right| = 2^{d^{(\ell)} - \langle m^{(\ell)}, \mathbf{1}\rangle}.$$

Proof. struggling with the proof here, is the statement of the theorem correct? □

14.4 References

Chapter 14 from You et al. (2022).

15 Fine-tuning

15.1 Introduction

Parameter fine-tuning is either done through additive, selective, re-parametrized or hybrid methods. The first involves introducing additional parameters to the model which are then trained, the second identifies a selection of parameters to retrain, the third decomposes the parameters into a low-rank representation which is then trained, and the fourth involves a combination of the previous. By utilising the fine-control of the parameterisation of deep network through the spline theory, deep network tuning can be done architecturally in proposed method named curvature tuning.

15.2 Smoothing Out Region Assignment

Recall that region assignment in a deep network operators according to an operation of the form

$$\left[\mathbf{t}^{(\ell)}\right]_i = \underset{j=1,\ldots,r^{(\ell)}}{\operatorname{argmax}} \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + [B]_{i,j} \right), \tag{15.1}$$

which in Section 5.2 we viewed as vector quantization. This lead to the β-VQ inference formulation, which showed that region assignment can be probabilistically relaxed to smooth out the non-linearity of deep networks

$$\left[\mathbf{t}_\beta^{(\ell)}\right]_i = \frac{\exp\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i} \left(\left\langle \left[A^{(\ell)}\right]_{i,j,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,j} \right) \right)}{\sum_{t=1}^{r^{(\ell)}} \exp\left(\frac{\left[\beta^{(\ell)}\right]_i}{1-\left[\beta^{(\ell)}\right]_i} \left(\left\langle \left[A^{(\ell)}\right]_{i,t,\cdot}, \mathbf{z}^{(\ell-1)} \right\rangle + \left[B^{(\ell)}\right]_{i,t} \right) \right)}. \tag{15.2}$$

In particular, recall Proposition 5.5.1 which shows how we can manoeuvrer from the linear regime, to the ReLU activation operator and then to the sigmoid gated linear activation operator simply by varying the parameter $\beta^{(\ell)}$ from zero to one. This has the effect of smoothing out the non-linearity of the deep network.

However, there is an alternative perspective we can take to induce a similar smoothing effective. More specifically, we can use a smooth the maximum operation in (15.1) with

$$\left[\tilde{\mathbf{t}}_{\beta}^{(\ell)}\right]_i = \left(1-\beta^{(\ell)}\right)\log\left(\sum_{j=1}^{r^{(\ell)}}\exp\left(\frac{1}{1-\beta^{(\ell)}}\left(\left\langle\left[A^{(\ell)}\right]_{i,j,\cdot},\mathbf{z}^{(\ell-1)}\right\rangle\right)+\left[B^{(\ell)}\right]_{i,j}\right)\right). \tag{15.3}$$

In this case, as $\beta^{(\ell)}$ goes from zero to one we manoeuvrer from the linear regime to the ReLU activation operator.

15.3 *Curvature Tuning*

Given a trained deep network, we can parametrise its behaviour with the parameter $\beta^{(\ell)}$ using (15.2) or (15.3). In particular, note that regardless of which parameterisation we utilise, changing the value of $\beta^{(\ell)}$ will cause the decision boundary to shift. For tuning the deep network, this behaviour is undesirable. However, using an average of these parameterisation negates these shifts.

Curvature tuning utilises this and proposes to replace the ReLU activation operators of a trained deep network with

$$\sigma_\beta(x) = \frac{\sigma_{\text{sig}}\left(\frac{\beta x}{1-\beta}\right)x}{2} + \frac{\log\left(1+\exp\left(\frac{x}{1-\beta}\right)\right)(1-\beta)}{2},$$

and then tune the parameter β using a forward passes on the deep network and a test set.

References

Chapter 15 from Hu and Balestriero (2025).

16 Generative Modelling

16.1 Introduction

Deep networks can be utilised for various tasks, including classification and regression. Typically, this involves receiving an input and outputting a prediction. In this chapter we investigate deep networks used for generative applications, namely given an input the deep network generates a response.

16.2 Continuous Piecewise Affine Autoencoders

Generative models typically have an encoder and generator component. The encoder is responsible for extracting features and reducing the dimensionality of the input, the generator reconstructs a response from an *embedding*.

Definition 16.2.1. *An autoencoder f comprises non-linear components, namely the encoder $E : \mathbb{R}^d \to \mathbb{R}^{d^{(L_E)}}$ and generator $G : \mathbb{R}^{d^{(L_E)}} \to \mathbb{R}^{d^{(L_E+L_G)}}$, such that $f = G \circ E$. In particular, E is a deep network with L_E layers, and G is a deep network with L_G layers.*

Remark 16.2.2. *Usually $d^{(L_E)} < d$ so that the deep network is encouraged to extract the most salient features for reconstruction.*

Under the assumption that the encoder and generator utilise affine and piece-wise affine operators, we can utilise the notation convention from statement 2 of Remark 2.2.2 to write

$$\begin{aligned} f(\mathbf{x}) &= (G \circ E)(\mathbf{x}) \\ &= A^G[E(\mathbf{x})]E(\mathbf{x}) + b^G[E(\mathbf{x})] \\ &= \tilde{A}^G[\mathbf{x}]\left(A^E[\mathbf{x}]\mathbf{x} + b^E[\mathbf{x}]\right) + \tilde{b}^G[\mathbf{x}] \\ &= \tilde{A}^G[\mathbf{x}]A^E[\mathbf{x}]\mathbf{x} + \tilde{A}^G[\mathbf{x}]b^E[\mathbf{x}] + \tilde{b}^G[\mathbf{x}], \end{aligned}$$

where $A^E \in \mathbb{R}^{d^{(L_E)} \times d}$, $B^E \in \mathbb{R}^{d^{(L_E)}}$, $\tilde{A}^G \in \mathbb{R}^{d^{(L_E+L_G)} \times d^{(L_E)}}$ and

$\tilde{B}^G \in \mathbb{R}^{d^{(L_E+L_G)}}$. Equivalently,

$$f(\mathbf{x}) = \sum_{j=1}^{d^{(L_E)}} \left\langle \left[A^E[\mathbf{x}]\right]_{j,\cdot}, \mathbf{x} \right\rangle \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} + b^{E,D}[\mathbf{x}], \tag{16.1}$$

where $b^{E,D}[\mathbf{x}] := \tilde{A}^G[\mathbf{x}]b^E[\mathbf{x}] + \tilde{b}^G[\mathbf{x}]$. Then for samples in a region $\omega \in \Omega \subseteq \mathbb{R}^d$, (16.1) tells us that the samples are expressed in the basis given by $\tilde{A}^G$ in a representation induced by A^E, these expression are then shifted according to both encoder and generator parameters.

Reconstruction Guarantees

Reconstruction models are parametrised such that $d = d^{(L_E+L_G)}$. The deep network is then trained to reconstruct inputs given a bottleneck constraint, namely having $d^{(L_E)} < d$.

With our continuous piecewise affine perspective on autoencoder networks, we can derive guarantees on their reconstruction abilities. In particular, we consider zero-bias autoencoders, which are autoencoders where $b^{d^{(\ell)}} = \mathbf{0}$ for every $\ell \in \{1, \ldots, L_E + L_G\}$.

Proposition 16.2.3. *A necessary condition for a zero-bias autoencoder to reconstruct a piecewise linear data surface it to be bi-orthogonal, namely for every $\mathbf{x} \in \omega$ if $f(\mathbf{x}) = \mathbf{x}$ then $\left\langle \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j}, \left[A^E[\mathbf{x}]\right]_{j',\cdot} \right\rangle = \delta_{jj'}$ for every $j, j' \in \left\{1, \ldots, d^{(L_E)}\right\}$.*[1]

[1] We are using Kronecker delta notation.

Proof. Reconstruction implies that for every $\mathbf{x} \in \mathbb{R}^d$ we have

$$\mathbf{x} = \sum_{j=1}^{d^{(L_E)}} \left\langle \left[A^E[\mathbf{x}]\right]_{j,\cdot}, \mathbf{x} \right\rangle \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j}.$$

Therefore,

$$\begin{aligned}\sum_{j'=1}^{d^{(L_E)}} \left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \mathbf{x} \right\rangle \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j'} &= \sum_{j'=1}^{d^{(L_E)}} \left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \sum_{j=1}^{d^{(L_E)}} \left\langle \left[A^E[\mathbf{x}]\right]_{j,\cdot}, \mathbf{x} \right\rangle \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} \right\rangle \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j'} \\ &= \sum_{j'=1}^{d^{(L_E)}} \sum_{j=1}^{d^{(L_E)}} \left\langle \left[A^E[\mathbf{x}]\right]_{j,\cdot}, \mathbf{x} \right\rangle \left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} \right\rangle \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j'}.\end{aligned}$$

Hence, as reconstruction implies that $\tilde{A}^G[\mathbf{x}]A^E[\mathbf{x}]$ is injective on the region ω encompassing $\mathbf{x}$, it follows that

$$\left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} \right\rangle = \delta_{jj'}$$

for every $j, j' \in \left\{1, \ldots, d^{(L_E)}\right\}$. □

Remark 16.2.4. *What Proposition 16.2.3 is requiring is that the map obtained by applying the encoder and then generator is the identity map.*

Corollary 16.2.5. *Let $L_E = L_P = 1$ and suppose E and G employ ReLU activation operations. Let $W^{(1)} \in \mathbb{R}^{d^{(1)} \times d}$ and $W^{(2)} \in \mathbb{R}^{d \times d^{(1)}}$ be the encoders and generators weight matrices respectively. Then for every $\mathbf{x} \in \mathbb{R}^d$ and $j, j' \in \left\{1, \ldots, d^{(1)}\right\}$ distinct, a necessary condition for bi-orthogonality is that one of the following statements is satisfied.*

1. $\left\langle \left[W^{(1)}\right]_{j,\cdot}, \mathbf{x} \right\rangle \leq 0.$
2. $\left\langle \left[W^{(2)}\right]_{i,\cdot}, E(\mathbf{x}) \right\rangle \leq 0$ *for every* $i = 1, \ldots, d$.
3. $\left\langle \left[W^{(2)}\right]_{i,\cdot}, E(\mathbf{x}) \right\rangle > 0$ *and* $\left\langle \left[W^{(2)}\right]_{\cdot,j'}, \left[W^{(1)}\right]_{j,\cdot} \right\rangle = 0$ *for every* $i = 1, \ldots, d$.
4. $\sum_{i=1}^{d} \left[W^{(2)}\right]_{i,j'} \left[W^{(1)}\right]_{j,i} \mathbf{1}_{\left\{\left\langle \left[W^{(2)}\right]_{i,\cdot}, E(\mathbf{x}) \right\rangle > 0\right\}} = 0.$

Proof. In this setting,

$$\begin{aligned}
\left[A^E[\mathbf{x}]\right]_{j,\cdot} &= \left[Q^{(1)}[\mathbf{x}] W^{(1)}\right]_{j,\cdot} \\
&= \left[Q^{(1)}[\mathbf{x}]\right]_{j,\cdot} W^{(1)} \\
&= \mathbf{1}_{\left\{\left\langle \left[W^{(1)}\right]_{j,\cdot}, \mathbf{x} \right\rangle > 0\right\}} \left[W^{(1)}\right]_{j,\cdot}
\end{aligned}$$

and

$$\begin{aligned}
\left[\tilde{A}^D[\mathbf{x}]\right]_{\cdot,j} &= \left[Q^{(2)}[\mathbf{x}] W^{(2)}\right]_{\cdot,j} \\
&= Q^{(2)}[\mathbf{x}] \left[W^{(2)}\right]_{\cdot,j} \\
&= \operatorname{diag}\left(\begin{pmatrix} \mathbf{1}_{\left\{\left\langle \left[W^{(2)}\right]_{1,\cdots}, E(\mathbf{x}) \right\rangle > 0\right\}} \\ \vdots \\ \mathbf{1}_{\left\{\left\langle \left[W^{(2)}\right]_{d,\cdots}, E(\mathbf{x}) \right\rangle > 0\right\}} \end{pmatrix}\right) \left[W^{(2)}\right]_{\cdot,j},
\end{aligned}$$

where we are utilising Prop 4.6.5 and the notation of statement 3 Remark 4.6.6. From Proposition 16.2.3, we know that under the assumption of reconstruction and j, j' distinct, we have

$$\left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} \right\rangle = 0. \tag{16.2}$$

With the expansion

$$\left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} \right\rangle = \mathbf{1}_{\left\{\left\langle \left[W^{(1)}\right]_{j,\cdot}, \mathbf{x} \right\rangle > 0\right\}} \left\langle \left[W^{(1)}\right]_{j,\cdot}, Q^{(2)}[\mathbf{x}] \left[W^{(2)}\right]_{\cdot,j} \right\rangle,$$

we get statement 1 by setting $\mathbf{1}_{\left\{\left\langle \left[W^{(1)}\right]_{j,\cdot}, \mathbf{x} \right\rangle > 0\right\}} = 0$ and statement 2 by setting $Q^{(2)}[\mathbf{x}] = 0$. From the expansion

$$\left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} \right\rangle = \mathbf{1}_{\left\{\left\langle \left[W^{(1)}\right]_{j,\cdot}, \mathbf{x} \right\rangle > 0\right\}} \left\langle \left[W^{(2)}\right]_{\cdot,j}, Q^{(2)}[\mathbf{x}] \left[W^{(1)}\right]_{j,\cdot} \right\rangle,$$

we get the possibility that $Q^{(2)}[\mathbf{x}]\left[W^{(1)}\right]_{j,\cdot} = 0$ which gives statement 3. Similarly, the expansion

$$\left\langle \left[A^E[\mathbf{x}]\right]_{j',\cdot}, \left[\tilde{A}^G[\mathbf{x}]\right]_{\cdot,j} \right\rangle = \mathbf{1}_{\left\{\left\langle \left[W^{(1)}\right]_{j,\cdot}, \mathbf{x}\right\rangle > 0\right\}} \sum_{i=1}^{d} \left[W^{(2)}\right]_{i,j'} \left[W^{(1)}\right]_{j,i} \mathbf{1}_{\left\{\left\langle \left[W^{(2)}\right]_{i,\cdot}, E(\mathbf{x})\right\rangle > 0\right\}}$$

gives the possibility that $\sum_{i=1}^{d} \left[W^{(2)}\right]_{i,j'} \left[W^{(1)}\right]_{j,i} \mathbf{1}_{\left\{\left\langle \left[W^{(2)}\right]_{i,\cdot}, E(\mathbf{x})\right\rangle > 0\right\}} = 0$ which is statement 4. With the understanding that none of the rows or columns of $W^{(1)}$ and $W^{(2)}$ can be zero, this entertains all possibilities for satisfying (16.2), therefore, the proof is complete. □

Remark 16.2.6.

1. *Statement 1 corresponds to the case where the input of the jth unit in the bottleneck layer is negative.*
2. *Statement 2 corresponds to the case where the input of all output units is negative.*
3. *Statement 3 corresponds to the case of a linear generator and orthogonality in the weights.*
4. *Statement 4 corresponds to an orthogonality condition between the $(j')^{th}$ column of the generator weights and the jth row of the encoder weights, modulo the activations of the generator layer.*
5. *Note that if statement 2 and 3 hold for multiple regions, then the generator is linear with respect to the coordinate space and forms a linear manifold.*

Autoencoder Jacobian

The continuous piecewise affine perspective of autoencoders also allows us to reason about their Jacobians.

Proposition 16.2.7. *For an autoencoder $f = G \circ E$, we have*

$$J_f(\mathbf{x}) = \tilde{A}^G[\mathbf{x}] A^E[\mathbf{x}],$$

provided $\mathbf{x}$ is in the interior of its corresponding region ω.

Proof. Recall that $\left[J_f(\mathbf{x})\right]_{i,j} = \frac{\partial [f]_i}{\partial [\mathbf{x}]_j}$. Therefore,

$$\begin{aligned}
\frac{\partial [f]_i}{\partial [\mathbf{x}]_j} &= \lim_{\epsilon \to 0} \left(\frac{\left[f\left(\mathbf{x} + \epsilon \mathbf{e}_j\right)\right]_i - [f(\mathbf{x})]_i}{\epsilon} \right) \\
&\overset{(1)}{=} \lim_{\epsilon \to 0} \left(\frac{\left[\tilde{A}^G[\mathbf{x}] A^E[\mathbf{x}] \left(\mathbf{x} + \epsilon \mathbf{e}_j\right)\right]_i - \left[\tilde{A}^G[\mathbf{x}] A^E[\mathbf{x}]\right]_i}{\epsilon} \right) \\
&= \left[\tilde{A}^G[\mathbf{x}] A^E[\mathbf{x}] \mathbf{e}_j\right]_i \\
&= \left[\tilde{A}^G[\mathbf{x}] A^E[\mathbf{x}]\right]_{i,j},
\end{aligned}$$

where in (1) we are assuming ϵ is sufficiently small so that $\mathbf{x} + \epsilon \mathbf{e}_j \in \omega$ which can be done since $\mathbf{x}$ is in the interior of ω. It follows that $J_f(\mathbf{x}) = \tilde{A}^G[\mathbf{x}] A^E[\mathbf{x}]$. □

Similarly, one can obtain the Jacobian of the generator block on the embedding space of the autoencoder model as

$$J_G(\mathbf{z}) = A^G[\mathbf{z}],$$

where $\mathbf{z} \in \mathbb{R}^{d^{(L_E)}}$. Note that $J_G(\mathbf{z})$ is constant for all $\mathbf{z} \in \omega$, where $\omega \in \Omega^G$ is a region in $\mathbb{R}^{d^{(L_E)}}$ induced by G. Consequently, we'll adopt the notation $J_G(\omega)$ to denote $J_G(\mathbf{z})$ for $\mathbf{z} \in \omega$. This allows for a characterisation of the curvature of the induced data manifold using the per-region Hessian,

$$\|H_\omega\|_F = \sum_{\omega' \in \mathcal{N}(\omega)} \left\| J_G(\omega) - J_G\left(\omega'\right) \right\|_F,$$

where $\| \cdot \|_F$ is the Frobenius norm, and $\mathcal{N}(\omega)$ is the set of neighbours of the region ω.

16.3 *Continuous Piecewise Affine Variational Autoencoders*

Variational Bayes

Suppose we have some dataset $\{\mathbf{x}_i\}_{i=1}^n$, that is an independent and identically distributed sample from some discrete variable $\mathbf{x}$. In particular, suppose that the data is generated by a random process governed by a continuous unobserved random variable $\mathbf{z}$. That is, for each observation $\mathbf{x}_i$ a value $\mathbf{z}_i$ was generated and then used to generate $\mathbf{x}_i$. As is the case with most problems in deep learning, we are looking for a characterisation of the distribution of our dataset. Formally, we suppose the $\mathbf{z}_i$ are generated from $p_{\theta^*}(\mathbf{z})$, the observations are then generated from $p_{\theta^*}(\mathbf{x}|\mathbf{z})$, so that

$$p(\mathbf{x}) = \int p_{\theta^*}(\mathbf{x}|\mathbf{z}) p_{\theta^*}(\mathbf{z}) \, \mathrm{d}\mathbf{z}.$$

Typically, we suppose that we can model these distributions from a parametric family of distributions $p_\theta(\mathbf{z})$ and $p_\theta(\mathbf{x}|\mathbf{z})$, whose probability density functions are differentiable almost everywhere, allowing for learning. In particular, the log-likelihood is given by

$$\log\left(p_\theta(\mathbf{x})\right) = \sum_{i=1}^n \log\left(p_\theta(\mathbf{x}_i)\right) = \sum_{i=1}^n \log\left(\int p_\theta(\mathbf{x}_i|\mathbf{z}) p_\theta(\mathbf{z}) \, \mathrm{d}\mathbf{z}\right).$$

The Expectation-Maximization algorithm (citation) works in stages to perform this learning. The E-step infers the latent variables associated with respect to the observations by taking the expectation of

the log-likelihood, and then this is maximized in the M-step with respect to the parameters θ. This process is predicated on the posterior $p_\theta(\mathbf{z}|\mathbf{x})$ being tractable, which is rarely the case. Instead, we introduce a variational distribution $q_\gamma(\mathbf{z})$ to approximate the posterior. Then observe that

$$\log\left(p_\theta\left(\mathbf{x}_i\right)\right) = \mathrm{KL}\left(q_\gamma(\mathbf{z}); p_\theta\left(\mathbf{z}|\mathbf{x}_i\right)\right) + \underbrace{\mathbb{E}_{q_\gamma(\mathbf{z})}\left(-\log\left(q_\gamma(\mathbf{z})\right)\right) + \mathbb{E}_{q_\gamma(\mathbf{z})}\left(p_\theta\left(\mathbf{x}_i|\mathbf{z}\right)\right)}_{\text{evidence lower bound}}.$$

The intractability of the posterior means the first term of the right-hand side is still intractable; however, since it is non-negative, the evidence lower bound is a lower bound on the log-likelihood. Thus an alternative strategy is to apply the Expectation-Maximization algorithm to maximize the evidence lower bound, which can be shown to indirectly minimize the discrepancy between the variational distribution and the posterior. More specifically, maximizing the evidence lower bound with respect to γ goes toward ensuring the variational distribution approximates the true posterior, a variational E-step. Then, maximizing the evidence lower bound with respect to θ goes toward fitting our model to the data; depending on the tightness of this lower bound, which is dependent on the suitability of $q_\gamma(\mathbf{z})$ at approximating the posterior, this is an analogous M-step.

Variational Autoencoders

In a variational autoencoder, the conditional distribution is taken to be

$$p_\theta(\mathbf{x}|\mathbf{z}) = \phi\left(\mathbf{x}; G_\theta(\mathbf{z}), \sigma_\mathbf{x}\right),$$

and the prior is taken to be

$$p(\mathbf{z}) = \phi\left(\mathbf{z}; 0, \sigma_\mathbf{z}\right),$$

where G_θ is a deep generative model, and ϕ is the multivariate Gaussian density function with given mean and variance.[2] When considering the deep generative network to be a continuous affine spline, we can obtain analytic expressions for the conditional, marginal and posterior distributions.

[2] These seemingly strong assumptions are reasonable since any distribution can be sufficiently transformed by a sufficiently complex function into a normal distribution.

Conditional, Posterior and Marginal Distributions

Lemma 16.3.1. *If G_θ is a continuous piecewise affine spline, then*

$$p(\mathbf{x}|\mathbf{z}) = \sum_{\omega\in\Omega} \phi\left(\mathbf{x}; A_\omega \mathbf{z} + b_\omega, \sigma_\mathbf{x}\right) \mathbf{1}_{\{\mathbf{z}\in\omega\}}.$$

Proof. Observe that

$$
\begin{aligned}
p(\mathbf{x}|\mathbf{z}) &= \frac{1}{(2\pi)^{\frac{d}{2}}\sqrt{|\det(\sigma_{\mathbf{x}})|}} \exp\left(-\frac{1}{2}\left(\mathbf{x}-G_\theta(\mathbf{x})\right)^\top \sigma_{\mathbf{x}}^{-1}\left(\mathbf{x}-G_\theta(\mathbf{x})\right)\right) \\
&= \frac{1}{(2\pi)^{\frac{d}{2}}\sqrt{|\det(\sigma_{\mathbf{x}})|}} \exp\left(-\frac{1}{2}\left(\mathbf{x}-(A[\mathbf{z}]\mathbf{z}+b[\mathbf{z}])\right)^\top \sigma_{\mathbf{x}}^{-1}\left(x-(A[\mathbf{z}]\mathbf{z}+b[\mathbf{z}])\right)\right) \\
&= \frac{1}{(2\pi)^{\frac{d}{2}}\sqrt{|\det(\sigma_{\mathbf{x}})|}} \exp\left(-\frac{1}{2}\left(\sum_{\omega\in\Omega}\mathbf{1}_{\{\mathbf{z}\in\omega\}}\left(\mathbf{x}-(A_\omega\mathbf{z}+b_\omega\mathbf{z})\right)^\top \sigma_{\mathbf{x}}^{-1}\left(\mathbf{x}-(A_\omega\mathbf{z}+b_\omega)\right)\right)\right) \\
&= \sum_{\omega\in\Omega}\mathbf{1}_{\{\mathbf{z}\in\omega\}}\frac{1}{(2\pi)^{\frac{d}{2}}\sqrt{|\det(\sigma_{\mathbf{x}})|}} \exp\left(-\frac{1}{2}\left(\left(\mathbf{x}-(A_\omega\mathbf{z}+b_\omega)\right)^\top \sigma_{\mathbf{x}}^{-1}\left(\mathbf{x}-(A_\omega\mathbf{z}+b_\omega)\right)\right)\right) \\
&= \sum_{\omega\in\Omega}\phi\left(\mathbf{x}; A_\omega\mathbf{z}+b_\omega, \sigma_{\mathbf{x}}\right)\mathbf{1}_{\{\mathbf{z}\in\omega\}}.
\end{aligned}
$$

□

Theorem 16.3.2. *If G_θ is a continuous piecewise affine spline, then*

$$
\begin{cases}
p(\mathbf{x}) = \sum_{\omega\in\Omega}\phi\left(\mathbf{x}; b_\omega, \sigma_{\mathbf{x}}+A_\omega\sigma_{\mathbf{z}}A_\omega^\top\right)\Phi_\omega\left(\mu_\omega(\mathbf{x}), \sigma_\omega\right) \\
p(\mathbf{z}|\mathbf{x}) = p(\mathbf{x})^{-1}\sum_{\omega\in\Omega}\phi\left(\mathbf{x}; b_\omega, \sigma_{\mathbf{x}}+A_\omega\sigma_{\mathbf{z}}A_\omega^\top\right)\phi\left(\mathbf{z}; \mu_\omega(\mathbf{x}), \sigma_\omega\right),
\end{cases}
$$

where $\Phi_\omega\left(\mu_\omega(\mathbf{x}), \sigma_\omega\right)$ denotes the integral of the Gaussian with parameters $\mu_\omega(\mathbf{x})$ and σ_ω on the domain ω, with

$$
\begin{cases}
\mu_\omega(\mathbf{x}) = \sigma_\omega\left(A_\omega^\top\sigma_{\mathbf{x}}^{-1}\left(\mathbf{x}-b_\omega\right)\right) \\
\sigma_\omega = \left(\sigma_{\mathbf{z}}^{-1}+A_\omega^\top\sigma_{\mathbf{x}}^{-1}A_\omega\right)^{-1}.
\end{cases}
$$

Proof. Refer to §F.3 Balestriero et al. (2020). □

Corollary 16.3.3. *The posterior of a deep generative network is a mixture of $|\Omega|$ truncated Gaussians.*

Remark 16.3.4. *Corollary 16.3.3 has implications on the form of the variational distribution $q_\gamma(\mathbf{z})$ used to model the posterior. In particular, practitioners should favour forms able to approximate multimodal data; thus using a Gaussian distribution is perhaps not ideal.*

Gaussian Integration on Partitions

From Theorem 16.3.2, it is necessary to compute the Gaussian integral over regions ω. To do so, one can leverage known forms of the Gaussian integral by decomposing ω. More specifically, we utilise the Delaunay triangulation

$$
T(\omega) = \{\Delta_1, \ldots, \Delta_m\},
$$

where

$$
\begin{cases}
\bigcup_{i=1}^m \Delta_i = \omega \\
\Delta_i \cap \Delta_j = \emptyset \quad i \neq j,
\end{cases}
$$

and each Δ_i is a simplex given by half-spaces $\Delta_i = \bigcap_{s=1}^{d^{(L_E)}+1} H_{i,s}$.

Lemma 16.3.5. *For any integrable function g, on a polytopal region ω we have*

$$\int_\omega g(\mathbf{z})\,\mathrm{d}\mathbf{z} = \sum_{\Delta \in T(\omega)} \sum_{(s,V)\in H(\Delta)} s \int_V g(\mathbf{z})\,\mathrm{d}\mathbf{z},$$

where

$$H(\Delta_i) := \left\{ \left((-1)^{|J|+d^{(L_E)}}, \cap_{j\in J} H_{i,j} \right) : J \subseteq \left\{1,\ldots,d^{(L_E)}+1\right\}, |J| \leq d^{(L_E)} \right\}.$$

Proof. Refer to §F.7 Balestriero et al. (2020). □

We can then use known results of the Gaussian integral over simplices in conjunction with Lemma 16.3.5 to compute the Gaussian integral over polytopal regions ω, as required for Theorem 16.3.2

Expectation-Maximization Learning

With analytic derivations of the posterior distribution, that are computable, one can effectively implement Expectation-Maximization learning to the probabilistic modelling problem at hand.

Expectation Step

Proposition 16.3.6. *With $\mathbf{e}_\omega^{(0)}(\mathbf{x}) := \mathbb{E}_{p(\mathbf{z}|\mathbf{x})}\left(\mathbf{1}_{\{\mathbf{z}\in\omega\}}\right)$, $\mathbf{e}_\omega^{(1)}(\mathbf{x}) := \mathbb{E}\left(\mathbf{z}\mathbf{1}_{\{\mathbf{z}\in\omega\}}\right)$ and $\mathbf{e}_\omega^{(2)}(\mathbf{x}) := \mathbb{E}_{p(\mathbf{z}|\mathbf{x})}\left(\mathbf{z}\mathbf{z}^\top\mathbf{1}_{\{\mathbf{z}\in\omega\}}\right)$ we have*

$$\begin{aligned}
\mathbb{E}_{p(\mathbf{z}|\mathbf{x})}\left(\log(p(\mathbf{x}|\mathbf{z})p(\mathbf{z}))\right) = &-\frac{1}{2}\left((2\pi)^{d^{(L_E)}+d}\left|\det(\sigma_\mathbf{x})\right|\left|\det(\sigma_\mathbf{z})\right|\right) \\
&-\frac{1}{2}\mathrm{tr}\left(\sigma_\mathbf{z}^{-1}\mathbf{e}^{(2)}(\mathbf{x})\right) \\
&-\frac{1}{2}\left(\mathbf{x}^\top\sigma_\mathbf{x}^{-1}\mathbf{x} - 2\mathbf{x}^\top\sigma_\mathbf{x}^{-1}\left(\sum_\omega A_\omega \mathbf{e}_\omega^{(1)}(\mathbf{x}) + b_\omega \mathbf{e}_\omega^{(0)}(\mathbf{x})\right)\right) \\
&+\sum_\omega\left(\mathrm{tr}\left(A_\omega^\top\sigma_\mathbf{x}^{-1}A_\omega\mathbf{e}_\omega^{(2)}(\mathbf{x})\right) + \left(\mathbf{e}_\omega^{(0)}(\mathbf{x})b_\omega + 2A_\omega\mathbf{e}_\omega^{(1)}(\mathbf{x})\right)^\top\sigma_\mathbf{x}^{-1}b_\omega\right)\right).
\end{aligned}$$

Proof. Refer to §G.1 Balestriero et al. (2020). □

Maximization Step Given Proposition 16.3.6 and the Gaussian parameterization, one can perform the maximization step analytically in a gradient-free manner.

Proposition 16.3.7. *Let $A_\omega^{(L\to i)} := \left(A_\omega^{(i\leftarrow L)}\right)^\top$ be the back-propagation matrix to layer i, let*

$$r_\omega^{(\ell)}(\mathbf{x}) := \left(\mathbf{x}\mathbf{e}_\omega^{(0)}(\mathbf{x}) - \left(A_\omega\mathbf{e}_\omega^{(1)}(\mathbf{x}) + \sum_{i\neq\ell}\mathbf{e}_\omega^{(0)}(\mathbf{x})A_\omega^{(i+1\to L)}Q_\omega^{(i)}b^{(i)}\right)\right)$$

be the expected residual without $b^{(\ell)}$, and

$$\hat{\mathbf{z}}_{\omega}^{(\ell)} := Q_{\omega}^{(\ell-1)} \left(A_{\omega}^{(1\to\ell-1)} \mathbf{e}_{\omega}^{(1)}(\mathbf{x}) + b_{\omega}^{(1\to\ell-1)} \mathbf{e}_{\omega}^{(0)} \right),$$

be the expected feature map of layer ℓ. Then, the maximizing variance parameter is

$$\sigma_{\mathbf{x}}^{*} = \frac{1}{N} \sum_{i=1}^{n} \left(\mathbf{x}_i \mathbf{x}_i^{\top} + \sum_{\omega\in\Omega} b_{\omega} \left(b_{\omega} \mathbf{e}^{(0)}(\mathbf{x}_i) + 2A_{\omega} \mathbf{e}_{\omega}^{(1)}(\mathbf{x}_i) \right)^{\top} - 2\mathbf{x}_i \left(\hat{\mathbf{z}}_{\omega}(\mathbf{x}_i) \right)^{\top} + A_{\omega} \mathbf{e}_{\omega}^{(2)}(\mathbf{x}_i) A_{\omega}^{\top} \right),$$

the maximizing bias parameter is

$$\left(b^{(\ell)} \right)^{*} = \left(\sum_{i=1}^{n} \sum_{\omega\in\Omega} Q_{\omega}^{(\ell)} A_{\omega}^{(L\to\ell+1)} \sigma_{\mathbf{x}_i}^{-1} A_{\omega}^{(\ell+1\to L)} Q_{\omega}^{(\ell)} \right)^{-1} \left(\sum_{i=1}^{n} \sum_{\omega\in\Omega} Q_{\omega}^{(\ell)} A_{\omega}^{(L\to\ell+1)} \sigma_{\mathbf{x}_i}^{-1} r_{\omega}^{(\ell)}(\mathbf{x}_i) \right),$$

and the maximizing weight parameter is

$$\mathrm{vect}\left(\left(W^{(\ell)} \right)^{*} \right) = U_{\omega}^{-1} \mathrm{vect} \left(\sum_{i=1}^{n} \sum_{\omega\in\Omega} Q_{\omega}^{(\ell)} A^{(L\to\ell+1)} \sigma_{\mathbf{x}_i}^{-1} \left(\mathbf{x}_i - \sum_{j=\ell}^{L} A_{\omega}^{(j+1\to L)} Q_{\omega}^{(j)} b^{(j)} \right) \left(\hat{\mathbf{z}}_{\omega}^{(\ell)}(\mathbf{x}_i) \right)^{\top} \right),$$

*where U_{ω} =***TODO!***.*

Proof. Refer to §G.2 Balestriero et al. (2020). □

16.4 Density of Generated Manifold

In this section, Section 16.5, and Section 16.6 we'll focus on the generator component of a generative model. We'll update our notation with Definition 16.4.1.

Definition 16.4.1. *A deep generative network is a non-linear operator G_{θ} with parameters θ, mapping a latent input $\mathbf{z} \in \mathbb{R}^{d}$ to an observation $\mathbf{y} \in \mathbb{R}^{d^{(L)}}$ by composing L layers.*

Remark 16.4.2. *Just as in the case of deep networks utilising piecewise affine non-linearities, a deep generative network utilising piecewise affine non-linearities can be considered a continuous piecewise affine spline.*

Suppose $d \leq d^{(\ell)}$, then the image of G_{θ} is a continuous piecewise affine manifold of dimension at most d. In particular,

$$\mathrm{Im}\left(G_{\theta}\right) := \left\{ G_{\theta}(\mathbf{z}) : \mathbf{z} \in \mathbb{R}^{d} \right\} = \bigcup_{\omega\in\Omega} \mathrm{aff}_{\omega},$$

where $\mathrm{aff}_{\omega} = \{A_{\omega}\mathbf{z} + b_{\omega} : \mathbf{z} \in \omega\}$.

Henceforth, we will denote the volume of a region $\omega \in \Omega$ as $\mu(\omega)$. Moreover, we assume that the operator G_{θ} is bijective from its domain to its image, that is, each A_{ω} is full rank.

Lemma 16.4.3. *We have*

$$\frac{\mu\left(\mathrm{aff}_\omega\right)}{\mu(\omega)} = \sqrt{\det\left(A_\omega^\top A_\omega\right)}.$$

Proof. The per-region affine mapping, $M(\mathbf{z}) = A_\omega \mathbf{z} + b_\omega$ produces an affine subspace embedded in the Euclidean space of dimension $d^{(L)}$. The metric tensor is given by $g = A_\omega^\top A_\omega$, Theorem 3.14 (Sard's) Spivak (1965). Leading to

$$\begin{aligned}
\mu\left(\mathrm{aff}_\omega\right) &= \int_{\mathrm{aff}_\omega} \mathrm{d}\mathbf{x} \\
&= \frac{1}{\sqrt{\det\left(A_\omega^\top A_\omega\right)}} \int_\omega \mathrm{d}\mathbf{z} \\
&= \frac{\mu(\omega)}{\sqrt{\det\left(A_\omega^\top A_\omega\right)}}.
\end{aligned}$$

□

Theorem 16.4.4. *The probability density $p_{G_\theta}(\mathbf{y})$ generated by G_θ for latent space distribution $p_\mathbf{z}$ is given by*

$$p_{G_\theta}(\mathbf{y}) = \sum_{\omega\in\Omega} \frac{p_\mathbf{z}\left(\left(A_\omega^\top A_\omega\right)^{-1} A_\omega^\top\left(\mathbf{y} - b_\omega\right)\right)}{\sqrt{\det\left(A_\omega^\top A_\omega\right)}} \mathbf{1}_{\{\mathbf{y}\in\mathrm{aff}_\omega\}},$$

where $A_\omega^\dagger := \left(A_\omega^\top A_\omega\right)^{-1} A_\omega^\top$.[3]

[3] This is also known as the Moore-Penrose inverse.

Proof. Let $\varrho_i(\cdot)$ denote the ith singular value of a matrix.

Lemma 16.4.5. *Let A be a matrix, then*

$$\sqrt{\det\left(A^\top A\right)} = \prod_{i:\varrho_i(A)\neq 0} \varrho_i(A).$$

Proof. Let A have singular value decomposition $USV^\top$. Then

$$\begin{aligned}
\sqrt{\det\left(A^\top A\right)} &= \sqrt{\det\left(\left(USV^\top\right)^\top\left(USV^\top\right)\right)} \\
&= \sqrt{\det\left(VS^\top U^\top USV^\top\right)} \\
&= \sqrt{\det\left(VS^\top SV^\top\right)} \\
&= \sqrt{\det\left(S^\top S\right)} \\
&= \prod_{i:\varrho_i(A)\neq 0} \varrho_i(A).
\end{aligned}$$

□

Lemma 16.4.6. *We have*

$$\varrho_i\left(A^\dagger\right) = \varrho_i(A)^{-1}.$$

Proof. Let A have singular value decomposition $USV^\top$. Then

$$\begin{aligned}
A^\dagger &= \left(A^\top A\right)^{-1} A^\top \\
&= \left(\left(USV^\top\right)^\top \left(USV^\top\right)\right)^{-1} \left(USV^\top\right)^\top \\
&= \left(VS^\top U^\top USV^\top\right)^{-1} \left(USV^\top\right)^\top \\
&= \left(VS^\top SV^\top\right)^{-1} VS^\top U^\top \\
&= V\left(S^\top S\right)^{-1} S^\top U^\top,
\end{aligned}$$

which is a singular value decomposition of $A^\dagger$. In particular, the singular values of $A^\dagger$ are given by the diagonal values of $\left(S^\top S\right)^{-1} S^\top$. Therefore,

$$\varrho_i\left(A^\dagger\right) = \frac{1}{\varrho_i(A)^2}\varrho_i(A) = \varrho_i(A)^{-1}.$$

□

Note that $J_{G_\theta^{-1}}(\mathbf{y}) = A_\omega^\dagger$. Due to the full rank assumptions,

$$p_{G_\theta(\mathbf{z})}(\mathbf{y} \in v) = p_{\mathbf{z}}\left(\mathbf{z} \in G_\theta^{-1}(v)\right) = \int_{G^{-1}(v)} p_{\mathbf{z}}(\mathbf{z})\,\mathrm{d}\mathbf{z}.$$

Therefore,

$$\begin{aligned}
p_{G_\theta}(\mathbf{y} \in v) &= \sum_{\omega\in\Omega} \int_{\omega\cap v} p_{\mathbf{z}}\left(G_\theta^{-1}(\mathbf{y})\right) \sqrt{\det\left(J_{G_\theta^{-1}}(\mathbf{y})^\top J_{G_\theta^{-1}}(\mathbf{y})\right)}\,\mathrm{d}\mathbf{y} \\
&= \sum_{\omega\in\Omega} \int_{\omega\cap v} p_{\mathbf{z}}\left(G_\theta^{-1}(\mathbf{y})\right) \sqrt{\det\left(\left(A_\omega\right)^\top A_\omega^\dagger\right)}\,\mathrm{d}\mathbf{y} \\
&\overset{\text{Lem 16.4.5}}{=} \sum_{\omega\in\Omega} \int_{\omega\cap v} p_{\mathbf{z}}\left(G_\theta^{-1}(\mathbf{y})\right) \left(\prod_{i:\varrho_i\left(A_\omega^\dagger\right)>0} \varrho_i\left(A_\omega^\dagger\right)\right)\,\mathrm{d}\mathbf{y} \\
&\overset{\text{Lem 16.4.6}}{=} \sum_{\omega\in\Omega} \int_{\omega\cap v} p_{\mathbf{z}}\left(G_\theta^{-1}(\mathbf{y})\right) \left(\prod_{i:\varrho_i(A_\omega)>0} \varrho_i\left(A_\omega^\dagger\right)\right)^{-1}\,\mathrm{d}\mathbf{y} \\
&\overset{\text{Lem 16.4.5}}{=} \sum_{\omega\in\Omega} \int_{\omega\cap v} p_{\mathbf{z}}\left(G_\theta^{-1}(\mathbf{y})\right) \frac{1}{\sqrt{\det\left(A_\omega^\top A_\omega\right)}}\,\mathrm{d}\mathbf{y} \\
&= \sum_{\omega\in\Omega} \int_{\omega\cap v} p_{\mathbf{z}}\left(\left(A_\omega^\top A_\omega\right)^{-1} A_\omega^\top\left(\mathbf{y} - b_\omega\right)\right) \frac{1}{\sqrt{\det\left(A_\omega^\top A_\omega\right)}}\,\mathrm{d}\mathbf{y}
\end{aligned}$$

as required. □

Corollary 16.4.7. *In the setting of Theorem 16.4.4, when the density of the*

latent distribution is uniform on a bounded domain $\mathcal{D}$, $\mathbf{z} \sim U(\mathcal{D})$, we have

$$p_{G_\theta}(\mathbf{y}) \propto \sum_{\omega \in \Omega} \frac{1}{\sqrt{\det\left(A_\omega^\top A_\omega\right)}} \mathbf{1}_{\{\mathbf{y} \in \mathrm{aff}_{\omega \cap \mathcal{D}}\}}.$$

Proof. Follows directly from Theorem 16.4.4 and the fact that

$$p_{\mathbf{z}}(\mathbf{z}) = \frac{1}{\mu(\mathcal{D})}.$$

□

16.5 Polarity Sampling

From Corollary 16.4.7 we can directly parametrise $p_{\mathbf{z}}$ to control the distribution of samples on the generated manifold.

Example 16.5.1.

1. *With $\mathbf{z} \sim U(\bar{\omega})$, where $\bar{\omega} = \mathrm{argmin}_{\omega \in \Omega}\left(\det\left(A_\omega^\top A_\omega\right)\right)$, we sample from the mode of the deep generative models generated distribution.*
2. *With $\mathbf{z} \sim U(\bar{\omega})$, where $\bar{\omega} = \mathrm{argmax}_{\omega \in \Omega}\left(\det\left(A_\omega^\top A_\omega\right)\right)$, we sample from the anti-mode of the deep generative models generated distribution.*

Corollary 16.5.2. *The latent distribution*

$$p_\rho(\mathbf{z}) \propto \sum_{\omega \in \Omega} \det\left(A_\omega^\top A_\omega\right)^{\frac{\rho}{2}} \mathbf{1}_{\{\mathbf{z} \in \omega\}},$$

on the domain $\mathcal{D}$, where ρ is the so-called polarity parameter, ensures that the output distribution of the deep generative model is such that

$$p_{G_\theta}(\mathbf{y}) \propto \sum_{\omega \in \Omega} \det\left(A_\omega^\top A_\omega\right)^{\frac{\rho-1}{2}} \mathbf{1}_{\{\mathbf{y} \in \mathrm{aff}_{\omega \cap \mathcal{D}}\}}.$$

Proof. Follows directly from Theorem 16.4.4. □

Remark 16.5.3. *In Corollary 16.5.2, with $\rho = 0$ in we sample from the standard deep generative network distribution, with $\rho \to -\infty$ we recover mode sampling from statement 1 of Example 16.5.1, and with $\rho \to \infty$ we recover anti-mode sampling from statement 2 of Example 16.5.1.*

Practical Implementation

From Corollary 16.5.2, it is clear that we need to determine the regions ω in the domain $\mathcal{D}$, the corresponding A_ω and then $\det\left(A_\omega^\top A_\omega\right)$.

Computing A_ω. Given a $\mathbf{z} \in \omega$, this can be done using 10.1 as $A_\omega = J_{G_\theta}\left(G_\theta(\mathbf{z})\right)$.

Discovering regions $\omega \in \Omega$. For state-of-the-art deep generative network this is challenging due to the exponential nature of the regions. As an approximation one can form a large sample n of latents $\mathbf{z} \sim U(\mathcal{D})$ and compute the corresponding $A_{\omega(\mathbf{z})}$ as above, in an attempt to account for all the regions.

Computing $\det\left(A_\omega^\top A_\omega\right)$. Due to a support lemma of Theorem 16.4.4, this computation reduces to computing the singular values of A_ω, for which there are known computational algorithms that can be done in $O\left(\min\left(d, d^{(\ell)}\right)^3\right)$ operations. In practice, only the top-k singular values are relevant, speeding up computations to $O\left(dk^2\right)$.

Algorithm 7: Polarity sampling.

Require: $d^{(\ell)}$, d, $n \gg d$, G_θ, $\mathcal{D}$, ρ.
  $\mathcal{Z} \leftarrow []$.
  $\mathcal{S} \leftarrow []$.
  $\mathcal{R} \leftarrow []$.
  for $i = 1, \ldots, n$ **do**
    $\mathbf{z} \sim U(\mathcal{D})$.
    $\varrho = \text{SingularValues}\left(J_{\mathbf{z}} G_\theta(\mathbf{z})\right)$ ▷ Ordered in a decreasing manner.
    Append $\mathbf{z}$ to $\mathcal{Z}$.
    Append $\rho \sum_{j=1}^{d^{(\ell)}} \log\left(\varrho_j + \epsilon\right)$
  end for
  for $i = 1, \ldots, d$ **do**
    $j \sim \text{Cat}(\text{softmax}(\mathcal{S}))$.
    Append $\mathcal{Z}_j$ to $\mathcal{R}$
  end for
  return $\mathcal{R}$.

16.6 ScaLES

Latent Space Optimisation

Let $\mathcal{V} \subseteq \{0,1\}^{l \times c}$ be a discrete and structured space, where each instance of $\mathcal{V}$ can be thought of as a sequence of l c-dimensional one-hot vectors representing categorical variables.

Let $\mathcal{M} : \mathcal{V} \to \mathbb{R}$ be the objective function. Then latent space optimisation is the problem

$$\operatorname*{argmax}_{\mathbf{v} \in \mathcal{V}} \mathcal{M}(\mathbf{v}).$$

In particular, a continuous representation of the problem is learned, and then optimisation techniques are applied in the continuous space

before being decoded back to the discrete space.

Let E_θ be a deep encoder model, let G_θ be a deep generative model, and let $\mathcal{D} = \{\mathbf{v}_i, y_i\}_{i=1}^n$ be an initial set of labelled data which is encoded into the latent space as $\mathcal{D}^{\mathbf{z}} = \{\mathbf{z}_i = E_\theta(\mathbf{v}_i), y_i\}_{i=1}^n$.

Typically evaluations on $\mathcal{M}$ are expensive, thus a surrogate model $\hat{f}$ is utilised[4]. The acquisition function is then

[4] Typically a Gaussian process.

$$\mathcal{A}\left(\hat{f}(\mathbf{z})\right) = \mathbb{E}\left(\max\left(\hat{f}(\mathbf{z}) - f^*, 0\right)\right),$$

where f^* is the best observed objective value and the expectation is with respect to the posterior of $\hat{f}$.

Algorithm 8: Latent space optimisation.

Require: T, $\mathcal{D}^{\mathbf{z}}$
  for $t = 1, \ldots, T$ **do**
    Fit the surrogate model to $\mathcal{D}^{\mathbf{z}}$.
    Generate a batch of query points $\mathbf{z}^{(\text{new})}$ $\text{argmax}_{\mathbf{z}} = \mathcal{A}\left(\hat{f}(\mathbf{z})\right)$.
    Decode $\mathbf{v}^{(\text{new})} = G_\theta\left(\mathbf{z}^{(\text{new})}\right)$.
    Obtain the true objective value $y^{(\text{new})} = \mathcal{M}\left(\mathbf{v}^{(\text{new})}\right)$.
    Update $\mathcal{D}^{\mathbf{z}}$ with $\left(\mathbf{z}^{(\text{new})}, y^{(\text{new})}\right)$.
  end for

An issue with Algorithm 8, is that it ignores the structure of $\mathcal{V}$, a problem referred to as over-exploration.

Definition 16.6.1. *Let $G_\theta : \mathcal{Z} \to \mathbb{R}^{l \times c}$ be a deep generative network, and let $\mathcal{V} \subseteq \mathbb{R}^{l \times c}$ be a set of valid sequences. The valid set of the latent space $\mathcal{Z}$ is*

$$\{\mathbf{z} : G_\theta(\mathbf{z}) \in \mathcal{V}\}.$$

Scalable Latent Exploration Score

Deep generative networks G_θ for discrete sequences typically output a matrix of logits, which are then transformed into normalised scores using the softmax function,

$$G_\theta(\mathbf{z}) = \text{softmax}\left(L_\theta(\mathbf{z})\right),$$

applied to every row of $L_\theta(\mathbf{z})$. Henceforth, we extend our notation with

$$\tilde{G}_\theta(\mathbf{z}) = \text{softmax}_+\left(L_\theta(\mathbf{z})\right) := \left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(1)}, \tilde{c}_{\mathbf{z}}^{(1)}, \ldots, \tilde{\mathbf{p}}_{\mathbf{z}}^{(l)}, \tilde{c}_{\mathbf{z}}^{(l)}\right)^\top,$$

where

$$\begin{cases} \tilde{\mathbf{p}}_{\mathbf{z}}^{(i)} = \text{softmax}\left(L_\theta(\mathbf{z})\right)_{i,\cdot} \\ \tilde{c}_{\mathbf{z}}^{(i)} = \sum_{j=1}^c \exp\left(L_\theta(\mathbf{z})_{ij}\right). \end{cases}$$

Theorem 16.6.2. *Assume L_θ is bijective and expressible as the continuous piecewise affine function*

$$L_\theta(\mathbf{z}) = \sum_{\omega \in \Omega} (A_\omega \mathbf{z} + b_\omega) \mathbf{1}_{\{\mathbf{z} \in \omega\}}.$$

Moreover, suppose that $\mathbf{z} \sim p_{\mathbf{z}}$. Then

$$p_{\tilde{G}_\theta}(\mathbf{v}) = p_{\mathbf{z}}\left(\tilde{G}_\theta^{-1}(\mathbf{v})\right) \sqrt{\det\left(\sum_{i=1}^{l} \left(A_i^\dagger\right)^\top B_i^\top B_i A_i^\dagger\right)},$$

where

$$\begin{cases} B_i = \left(\operatorname{diag}\left(\frac{1}{\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(i)}\right)_1}, \ldots, \frac{1}{\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(i)}\right)_d}\right), \mathbf{1}\frac{1}{\tilde{c}_{\mathbf{z}}^{(i)}}\right)^\top \\ A_i^\dagger = \left(A_\omega^{(1)}, \ldots, A_\omega^{(l)}\right)^\dagger_{(i \cdot d):((i+1) \cdot d)} \end{cases}.$$

Proof. Consider the following.

Lemma 16.6.3. *In the setting of Theorem 16.6.2, we have*

$$J\left(\tilde{G}_\theta^{-1}\right)(\mathbf{v}) = \left(\begin{pmatrix} B_1 & \ldots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \ldots & B_l \end{pmatrix} A_\omega^\dagger\right)^\top.$$

Proof. Recall that $\tilde{G}_\theta(\mathbf{z}) = \mathrm{softmax}_+\left(L_\theta(\mathbf{z})\right)$. Since L_θ is bijective and hence invertible, we have

$$\left(\tilde{G}_\theta\right)^{-1}(\mathbf{z}) = L_\theta^{-1}\left(\mathrm{softmax}_+^{-1}(\mathbf{z})\right).$$

In particular,

$$L_\theta^{-1}\left(\mathrm{softmax}_+^{-1}(\mathbf{v})\right) = \left(\mathrm{softmax}_+^{-1}(\mathbf{v}) - b_\omega\right) A_\omega^\top.$$

Moreover, observe that

$$\begin{aligned}
&\mathrm{softmax}_+\left(\begin{pmatrix} \log\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(1)}\right) + \log\left(\tilde{c}_{\mathbf{z}}^{(1)}\right) \\ \vdots \\ \log\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(l)}\right) + \log\left(\tilde{c}_{\mathbf{z}}^{(l)}\right) \end{pmatrix}\right) \\
&= \begin{pmatrix} \frac{1}{\hat{c}_{\mathbf{z}}^{(1)}} e^{\left(\log\left(\left[\tilde{\mathbf{p}}_{\mathbf{z}}^{(1)}\right]_1\right) + \log\left(\tilde{c}_{\mathbf{z}}^{(1)}\right)\right)} & & & & \\ & \hat{c}_{\mathbf{z}}^{(1)} & & & \\ & & \ddots & & \\ & & & \hat{c}_{\mathbf{z}}^{(l)} & \\ & & & & e^{\left(\log\left(\left[\tilde{\mathbf{p}}_{\mathbf{z}}^{(l)}\right]_c\right) + \log\left(\tilde{c}_{\mathbf{z}}^{(l)}\right)\right)} \end{pmatrix} \\
&= \left(\frac{\tilde{c}_{\mathbf{z}}^{(1)}}{\hat{c}_{\mathbf{z}}^{(1)}}\tilde{\mathbf{p}}_{\mathbf{z}}^{(1)}, \hat{c}_{\mathbf{z}}^{(1)}, \ldots, \frac{\tilde{c}_{\mathbf{z}}^{(l)}}{\hat{c}_{\mathbf{z}}^{(l)}}\tilde{\mathbf{p}}_{\mathbf{z}}^{(l)}, \hat{c}_{\mathbf{z}}^{(l)}\right)^\top \\
&= \left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(1)}, \tilde{c}_{\mathbf{z}}^{(1)}, \ldots, \tilde{\mathbf{p}}_{\mathbf{z}}^{(l)}, \tilde{c}_{\mathbf{z}}^{(l)}\right)^\top.
\end{aligned}$$

where operations are component-wise operations and

$$\hat{c}_{\mathbf{z}}^{(i)} = \sum_{j=1}^{c} e^{\left(\log\left(\left[\tilde{\mathbf{p}}_{\mathbf{z}}^{(i)}\right]_j\right) + \log\left(\tilde{c}_{\mathbf{z}}^{(i)}\right)\right)} = \tilde{c}_{\mathbf{z}}^{(i)} \sum_{j=1}^{c} \left[\tilde{\mathbf{p}}^{(i)}\right]_j = \tilde{c}_{\mathbf{z}}^{(i)}.$$

So that,

$$\mathrm{softmax}_+^{-1}(\mathbf{v}) = \left(\log\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(1)}\right) + \log\left(\tilde{c}_{\mathbf{z}}^{(1)}\right), \ldots, \log\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(l)}\right) + \log\left(\tilde{c}_{\mathbf{z}}^{(l)}\right)\right)^\top.$$

Therefore,

$$\frac{\partial}{\partial \mathbf{v}} \mathrm{softmax}_+^{-1}(\mathbf{v}) = \begin{pmatrix} B_1 & \ldots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \ldots & B_l \end{pmatrix}$$

where

$$B_i = \left(\mathrm{diag}\left(\frac{1}{\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(i)}\right)_1}, \ldots, \frac{1}{\left(\tilde{\mathbf{p}}_{\mathbf{z}}^{(i)}\right)_d}\right), \mathbf{1}\frac{1}{\tilde{c}_{\mathbf{z}}^{(i)}}\right)^\top,$$

thus the result follows. □

Due to invertability, we can proceed as follows,

$$\begin{aligned}
\mathbb{P}(\mathbf{v} \in v) &= \left(\mathbf{v} \in \tilde{G}_\theta^{-1}(v)\right) \\
&= \sum_{\omega \in \Omega} \mathbb{P}\left(\mathbf{z} \in \left(\tilde{G}_\theta^{-1}(v) \cap \omega\right)\right) \\
&= \sum_{\omega \in \Omega} \int_{\tilde{G}_\theta^{-1}(v) \cap \omega} p_{\mathbf{z}}(\mathbf{z})\, \mathrm{d}\mathbf{z} \\
&= \sum_{\omega \in \Omega} \int_{v \cap \tilde{G}_\theta(\omega)} p_{\mathbf{z}}\left(\tilde{G}_\theta^{-1}(\mathbf{v})\right) \sqrt{\det\left(J\left(\tilde{G}_\theta^{-1}(\mathbf{v})\right) J\left(\tilde{G}_\theta^{-1}(\mathbf{v})\right)^\top\right)} \\
&\overset{\text{Lem 16.6.3}}{=} \int_v \sum_{\omega \in \Omega} p_{\mathbf{z}}\left(\tilde{G}_\theta^{-1}(\mathbf{v})\right) \sqrt{\det\left(\sum_{i=1}^{l} \left(A_i^\dagger\right)^\top B_i^\top B_i A_i^\dagger\right)}.
\end{aligned}$$

□

Definition 16.6.4. *The ScaLES with weight parameter ρ is*

$$\mathcal{S}_\rho(\mathbf{z}) = \log\left(p_{\mathbf{z}}(\mathbf{z})\right) + \rho \log\left(\sqrt{\det\left(\sum_{i=1}^{l} \left(A_i^\dagger\right)^\top B_i^\top B_i A_i^\dagger\right)}\right).$$

ScaLES is theoretically and empirically verified to obtain higher scores in regions of the latent space's valid set, Definition 16.6.1. Therefore, it seems reasonable to utilise this score as a way to mitigate the over-exploration issue of Algorithm 8.

Moreover, the ScaLES is differentiable, making it amenable to optimisation. More specifically, one modifies Algorithm 8 with

$$\mathbf{z}^{(\text{new})} = \text{argmax}_{\mathbf{z}} \mathcal{A}\left(\hat{f}(\mathbf{z})\right) + \lambda \mathcal{S}(\mathbf{z}).$$

16.7 References

Section 16.2 from Cosentino et al. (2022). Section 16.3 from Balestriero et al. (2020), with some of Section 16.3 from Kingma and Welling (2022). Section 16.4 from Humayun et al. (2022a). Section 16.5 from Humayun et al. (2022b). Section 16.6 from Ronen et al. (2024).

17 Applications to Language Models

17.1 Introduction

Language models commonly employ some form of transformer architecture, thus in this chapter we'll be considering these types of models. Even though some language model components employ smooth activation operators, their multi-layer perceptron components can be configured with piecewise affine activation operators, and thus are conducive to a spline orientated investigations. We use this perspective to understand how different aspects of the model's output relates to the underlying geometry of the model.

17.2 The Architecture

The ℓth transformer block is configured to receive as input as collection of token embeddings $\mathbf{Z}^{(\ell-1)} \in \mathbb{R}^{T \times d^{(\ell-1)}}$ and generate a similarly structure set of token embeddings $\mathbf{Z}^{(\ell)} \in \mathbb{R}^{T \times d^{(\ell)}}$. More specifically, the transformer block uses H attention mechanisms on the input, which it then combines using a multi-layer perception. This is then added to the residual stream of the model to generate the required embeddings. Namely,

$$\begin{cases} \mathsf{HEAD}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right) := \mathrm{softmax}_{\text{causal}}\left(\mathbf{Z}^{(\ell-1)} Q_h^{(\ell)} \left(\mathbf{Z}^{(\ell-1)} K_h^{(\ell)}\right)^\top\right) \mathbf{Z}^{(\ell-1)} V_h^{(\ell)} & h = 1, \ldots, H \\ \mathsf{MHA}^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right) := \sum_{h=1}^{H} \mathsf{HEAD}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right) O_h^{(\ell)} \\ \mathsf{LAYER}^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right) := \mathsf{MLP}^{(\ell)}\left(\mathsf{LayerNorm}^{(\ell)}\left(\mathsf{MHA}^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right) + \mathbf{Z}^{(\ell-1)}\right)\right) + \mathbf{Z}^{(\ell-1)}. \end{cases} \tag{17.1}$$

We explicitly highlight the attention mechanism of the heads of the transformer block with[1]

$$\mathsf{Attn}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right) := \mathrm{softmax}_{\text{causal}}\left(\mathbf{Z}^{(\ell-1)} Q_h^{(\ell)} \left(\mathbf{Z}^{(\ell-1)} K_h^{(\ell)}\right)^\top\right).$$

[1] The causal subscript reflects the fact that the attention mechanism implements masking.

For an input $\mathbf{X} \in \mathbb{R}^{T \times d}$ of token embeddings, the language model provides a similarly structure output of token embeddings $\mathbf{Y} \in \mathbb{R}^{T \times d^{(L)}}$ by composing L transformer layers,[2]

$$\mathsf{LLM}\left(\mathbf{X}\right) = \left(\mathsf{Layer}^{(L)} \circ \cdots \circ \mathsf{Layer}^{(1)}\right)(\mathbf{X}).$$

[2] Strictly speaking the language model also involves encoder and decoder blocks to transform the tokens into embeddings and embeddings into token, however, we omit those here for simplicity.

17.3 The Geometry of Attention

Lemma 17.3.1. *The ith row of* $\mathsf{Head}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)$ *lies within the convex hull*

$$\operatorname{conv}\left(\left\{\left(V_h^{(\ell)}\right)^\top \left[\mathbf{Z}^{(\ell-1)}\right]_{j,\cdot} : j = 1, \ldots, i\right\}\right),$$

and is of effective dimension at most

$$\left|\left\{\left[\mathsf{Attn}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)\right]_{i,j} > 0 : j = 1, \ldots, i\right\}\right|.$$

Proof. Since the causal mask hides the tokens $i+1, \ldots, T$ from the attention mechanism at token i, and the softmax provides attention values normalised to a probability distribution, it follows that

$$\left[\mathsf{Head}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)\right]_{i,\cdot} = \sum_{j=1}^{i} \alpha_i \left[\mathbf{Z}^{(\ell-1)} V_h^{(\ell)}\right]_{j,\cdot},$$

where $\alpha_j \geq 0$ for $j = 1, \ldots, i$ with $\sum_{j=1,\ldots,i} \alpha_j = 1$. Hence,

$$\left[\mathsf{Head}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)\right]_{i,\cdot} \in \operatorname{conv}\left(\left\{\left(V_h^{(\ell)}\right)^\top \left[\mathbf{Z}^{(\ell-1)}\right]_{j,\cdot} : j = 1, \ldots, i\right\}\right).$$

More specifically, the effective dimension is at most the number of non-zero α_js for $j = 1, \ldots, i$, thus the effective dimension is at most

$$\left|\left\{\left[\mathsf{Attn}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)\right]_{i,j} > 0 : j = 1, \ldots, i\right\}\right|.$$

□

Let

$$\mathcal{A}_h^{(\ell)}(i) := \operatorname{conv}\left(\left\{\left(V_h^{(\ell)} O_h^{(\ell)}\right)^\top \left[\mathbf{Z}^{(\ell-1)}\right]_{j,\cdot} : j = 1, \ldots, i\right\}\right),$$

namely the convex hull of the head mapping onto $O^{(\ell)}$.

Theorem 17.3.2. *The ith row of* $\mathsf{MHA}^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)$ *lies within the Minkowski sum*[3] $\mathcal{A}_1^{(\ell)}(i) + \cdots + \mathcal{A}_H^{(\ell)}(i)$*, and is of effective dimension at most*

$$\sum_{h=1}^{H} \left|\left[\mathsf{Attn}_h^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)\right]_{i,j} > 0 : j = 1, \ldots, i\right|. \tag{17.2}$$

[3] For sets $\mathcal{A}$ and $\mathcal{B}$, their Minkowski sum $\mathcal{A} + \mathcal{B}$ is $\{a + b : \text{for all } (a, b) \in \mathcal{A} \times \mathcal{B}\}$.

Proof. As noted in the proof of Lemma 17.3.1, we have

$$\left[\mathsf{MHA}^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)\right]_{i,\cdot} = \sum_{h=1}^{H} \alpha_i^{(h)} \left[\mathbf{Z}^{(\ell-1)} V_h^{(\ell)} O_h^{(\ell)}\right],$$

where $\alpha_j^{(h)} \geq 0$ for $j = 1, \ldots, i$ and $\sum_{j=1,\ldots,i} \alpha_j^{(h)} = 1$ for each $h = 1, \ldots, H$. Thus, it follows that

$$\left[\mathsf{MHA}^{(\ell)}\left(\mathbf{Z}^{(\ell-1)}\right)\right]_{i,\cdot} \in \mathcal{A}_1^{(\ell)}(i) + \cdots + \mathcal{A}_H^{(\ell)}(i),$$

with the required effective dimension due to similar arguments to those made in Lemma 17.3.1. □

Remark 17.3.3. *From Lemma 17.3.1 and Theorem 17.3.2 we observe that the effective dimension of the multi-head output for each token is upper bounded by the number of tokens proceeding multiplied by the number of heads being used. Moreover, the effective dimension of a token embedding increases with the number of non-zero attention it has with its proceeding tokens.*

The intrinsic dimension of an embedding space refers to the minimum number of parameters required for its characterisation whilst maintaining its structure. Theorem 17.3.2 offers a way to estimate the intrinsic dimension of the embeddings of tokens from multi-head mechanisms, namely,

$$\mathrm{ID}_{\epsilon}^{(\ell)}(i) := \left| \left\{ \left[\mathsf{Attn}_h^{(\ell)} \left(\mathbf{Z}^{(\ell-1)} \right) \right]_{i,j} > \epsilon : j = 1, \ldots, i \right\} \right|.$$

That is, an estimate for the intrinsic dimension is taken to be the number of tokens that influential, beyond a threshold ϵ, for the ith embedding.

17.4 Expressivity

An important observation to make regarding the multi-layer perceptron component of the transformer layer presented in (17.1), is that it is applied independently on each row $\mathbf{Z}^{(\ell-1)}$. That is,

$$\mathsf{MLP}^{(\ell)} \left(\mathbf{Z}^{(\ell-1)} \right) = \left(\mathsf{MLP}^{(\ell)} \left(\left[\mathbf{Z}^{(\ell-1)} \right]_{1,\cdot} \right), \ldots, \mathsf{MLP}^{(\ell)} \left(\left[\mathbf{Z}^{(\ell-1)} \right]_{T,\cdot} \right) \right)^{\top}.$$

Supposing that the multi-layer perceptron component implements piecewise-affine operators, it can be considered as a continuous piecewise affine spline, namely

$$\mathsf{MLP}^{(\ell)} \left(\left[\mathbf{Z}^{(\ell-1)} \right]_{j,\cdot} \right) = \sum_{\omega \in \Omega^{(\ell)}} \left(A_{\omega}^{(\ell)} \left[\mathbf{Z}^{(\ell-1)} \right]_{j,\cdot} + b_{\omega}^{(\ell)} \right) \mathbf{1}_{\left\{ \left[\mathbf{Z}^{(\ell-1)} \right]_{j,\cdot} \in \omega \right\}},$$

where $\Omega^{(\ell)}$ is a partition of $\mathbb{R}^{d^{(\ell-1)}}$. The additional layer normalisation and skip-connection present in (17.1) does not affect this interpretation for our analysis, since partition statistics such as the number of regions, determining whether points are in the same region, or the identification of which region an input belongs to, remains unchanged with these added operations.

Since the regions of $\Omega^{(\ell)}$ provide a measure of expressivity, with this perspective, we can understand the expressivity of these multi-layer perceptron blocks.

Proposition 17.4.1. *The expressivity of a transformer layer's multi-layer perceptron that can be reached by the multi-head attention's output, grows exponentially with the multi-head attention's intrinsic dimension as measured by* (17.2).

Proof. This follows from Theorem 17.3.2 and observations from Section 4.6 which identify the exponential relationship between regions and dimensions. □

Proposition 17.4.1 provides a necessary but not sufficient condition for chain-of-thought reasoning or in-context learning.

These insights can be used to derive first order geometric features to characterise prompts to language models to understand their toxicity Balestriero et al. (2023), or to improve their reasoning abilities Cosentino and Shekkizhar (2024).

17.5 *References*

Chapter 17 from Balestriero et al. (2023).

18 Interpretability

18.1 Introduction

Deep network interpretability is the study of reverse-engineering the process of deep networks in a contextual manner. Interpretability seeks to understanding what computations a deep network is processing to arrive at its outputs. Here we explore how the characterisation of deep network geometry using spline theory can shed light on these endeavours.

18.2 Neurons as Fundamental Units

Since the architecture of deep networks is a collection of neurons connected by weights, it may be reasonable to explore the behaviour of these individual neurons to get a sense of the computations that a deep network is performing when it receives an input. After all, this has been a popular doctrine for scientists to employ when investigating how neural activity in the brain correlates with the behaviour of organisms.

Even though the architecture of a deep network would seem to favour the deep network assigning the components of its computation to neurons, there is no immediate mathematical reason as to why this should be the case[1]. In fact, it may even be a hindrance for these models to constrain their behaviour to the activity of individual neurons[2]. Furthermore, apart from collections of neurons, referred to as circuits, being identified as operating particular computational algorithms, there is limited empirical evidence that deep networks use neurons as their fundamental units of computations. What is observed is that neurons tend to fire on multiple different concepts, and thus display polysemanticity rather than monosemanticity.

[1] Due to this idea that the architecture of the deep network should favour computational units aligning with neurons, results in the basis spanned by neurons being called the privileged basis.

[2] Explore the notion of superposition and the (add lemma name) lemma to

18.3 Directions as Fundamental Units

Deep network are a series of affine transformations interspersed with non-linearities. In particular, the last layer of the deep network is

an affine transformation. Therefore, it seem reasonable that the intermediate layers of a deep network are working towards linearly separating pertinent features for the intended task. Consequently, a focus of work in interpretability has been on extracting features of the deep network's computation using linear methods, that is attributing features to directions. In particular, techniques such as sparse autoencoders Huben et al. (2024) (move this reference to spline theory library) have been surprisingly successful at extracting important features from transformer based deep networks, evidenced by their utility in downstream tasks; providing evidence for the linear representation hypothesis Park et al. (2024) (move this reference to spline theory library), which posits that the units used by the deep network for computations can be represented as direction in some underlying vector space.

A prominent source of evidence for the linear representation hypothesis is the identification of polysemantic neurons, that is neurons that are active on various different concepts. Typically this polysemanticity occurs on semantically unrelated concepts. The explanation here would be that the network is storing these concepts with similar directions since they are unlikely to interfere and impact the behaviour of the deep network. More specifically, since the concepts are unrelated, they are unlikely to be present simultaneously and thus generally do not interfere. Moreover, any low level interference can be ablated by the non-linearities.

However, there has been some evidence suggesting that this perspective on the deep network's computation is not the full story. In particular, the fact that polysemanticity is a useful property for the deep network lies in non-linearity. Furthermore, under the linear representation hypothesis, it follows that deep networks would be invariant to scaling. However, in practice, this is observed not be the case. Therefore, there needs to be a "distribution of validity" component of the linear representation hypothesis to explain more of the deep network's behaviour.

18.4 Polytopes as Fundamental Units

The polytope lens puts partitioning of the input space as the focus for understanding the behaviour of deep networks. The appeal of this perspective is that the polytopes provide a complete characterisation of the input space of deep network's, and thus offer a way for completely characterising the behaviour of these models.

Within each polytope the inputs are subject to the same affine transformation, thus we have an invariance within the polytopes. Moreover, the affine transformations do not differ drastically between

neighbouring polytopes. This local invariance helps us reason about the semantics of the model's input.

To better consolidate these ideas we introduce the spline code of a given polytope. For this we focus on deep networks using ReLU, leaky ReLU or absolute value activation functions.

Definition 18.4.1. *The spline code of a polytope (region) $\omega \in \Omega$ is the vector $\mathbf{s} = \left(\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)}\right)$ which is constructed be concatenating the vectors*

$$\left[\mathbf{s}^{(\ell)}\left(\mathbf{z}^{(\ell-1)}(\mathbf{x})\right)\right]_j = \begin{cases} 1 & \left\langle \left[W^{(\ell)}\right]_{j,\cdot}, \mathbf{z}^{(\ell-1)}(\mathbf{x}) \right\rangle + \left[b^{(\ell)}\right]_j > 0 \\ 0 & \textit{otherwise.} \end{cases}$$

Remark 18.4.2. *Note the similarity in notation to that develop in Section 4.6. We just update slightly the notation here for subsequent analyses.*

The spline codes do more than just help identify individual regions, they help elucidate their semantics. More specifically, note that neighbouring polytopes differ in splines codes by just one entry. Therefore, polytopes that are close together have *close* spline codes. One can quantify the closeness of spline codes using Hamming distance.

Definition 18.4.3. **TODO!** *add Hamming distance definition.*

From this perspective we take features of the deep network to be a group of nearby polytopes that admit similar affine transformations.

Connection to the directions hypothesis. The polytope lens can be seen as a slight augmentation of the linear representation hypothesis. Vectors in nearby polytopes share high similarity with those in *similar* or nearby polytopes. In particular, within the same polytope the polytope lens exactly matches the features as directions hypothesis.

Predictions of the polytope lens. With the understanding that semantics is characterised by clusters of polytopes, we can make certain predictions on what this means for model behaviour.

1. Polysemantic directions overlap with multiple monosemantic polytope regions.
2. Polytope boundaries reflect semantic boundaries. More specifically, polytopes will be clustered around regions of the input space with distinct semantics.
3. Polytopes define the distributions of when a features is associated to a particular direction.

Black et al. (2022) provides evidence supporting these predictions to various extents.

18.5 Mechanistic Interpretability

Recall in Section 18.2 we noted that it has been observed that there exists clusters of neurons whose activity patterns can be understood as performing algorithmic operations (add citations). This does not necessarily conform with the neurons as units perspective, since it comprises the interaction of multiple neurons. Similarly, it cannot be understood well through the directions as unit perspective, since as the feature spans multiple neurons it is necessarily non-linear due to the activation functions. However, there is an explanation that the polytopes as units perspective can provide.

More specifically, recall that the affine transformation operation on a region ω is given by the parameters A_ω and b_ω as given by Proposition 4.6.4 and Proposition 4.6.5. In particular, Proposition 4.6.4 and Proposition 4.6.5 highlight how the affine transformation acting on ω is a function of the activity of the other neurons in the deep network, indeed this is precisely what $\mathbf{s}$ captures.[3]More specifically, the local complexity measure introduced in Section 11.2, can be seen as a measure of the density of unique circuits formed in a locality of the input space.

[3] Here we are departing from the notation of $\mathbf{s}$ introduced in Section 18.4 and recalling previous notation.

Similarly, recall that from the polytope lens it follows that clusters of polytopes are meaningful. Therefore, since the non-linearities of individual neurons form the hyperplanes which generate these polytopes, one can see how a cluster of polytopes can be constructed from a cluster of neurons. Therefore, analysing the input space partition may facilitate circuit discovery, or help understand the nature of a circuit by identify the semantics of the inputs that arise in its corresponding cluster of polytopes.

What we see then is that spline theory elucidates the interaction between neurons and how this manifests as structure in the input space of deep networks. Consequently, we connect this structure to the behaviour of the deep network by using it to augment the established observation that directions in the input space are pertinent to the deep network's behaviour.[4]

[4] Throughout these discussion we have been explicitly referring to the deep network and its resulting partition on its input space. However, hopefully it is clear to see that one can equally investigate the computational units of the deep network at intermediary stages by analysing the partitions of the input space of the layers instead.

18.6 References

Section 18.2, Section 18.3 and Section 18.4 are inspired from Black et al. (2022). Section 18.5 inspired from Humayun et al. (2024).

19 Computation

19.1 Introduction

In general, considering the functional geometry of deep networks is computational intractable due to the combinatorial nature of the regions, and the necessity of solving large linear programs to determine their exact structure. Moreover, modern deep networks operate in high-dimensional spaces, and thus the curse of dimensionality becomes quickly evident. However, there are methods to circumvent these restrictions when focusing on computing specific statistics. Similarly, one can develop approximate heuristics that can provide valuable information on the underlying structure of the deep network's geometry.

19.2 Jacobian-Vector Products

For an operator $f : \mathbb{R}^d \to \mathbb{R}^k$, the right-operation at $\mathbf{x} \in \mathbb{R}^d$ denoted $\mathrm{Rop}_{(f,\mathbf{x})} : \mathbb{R}^d \to \mathbb{R}^k$ is given by

$$\mathrm{Rop}_{(f,\mathbf{x})}(\mathbf{u}) := J_f(\mathbf{x})\mathbf{u},$$

where $J_f(\mathbf{x})$ is the Jacobian of f evaluated at $\mathbf{x}$.[1] Explicitly,

$$J_f(\mathbf{x}) = \begin{pmatrix} \frac{\partial [f]_1}{\partial [\mathbf{x}]_1}(\mathbf{x}) & \dots & \frac{\partial [f]_1}{\partial [\mathbf{x}]_d}(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial [f]_k}{\partial [\mathbf{x}]_1}(\mathbf{x}) & \dots & \frac{\partial [f]_k}{\partial [\mathbf{x}]_d}(\mathbf{x}) \end{pmatrix},$$

however, with regards to computing Jacobian-vector products it is not necessary to determine this.

Another common Jacobian-vector product is the left-operation at $\mathbf{x} \in \mathbb{R}^d$ denoted $\mathrm{Lop}_{(f,\mathbf{x})} : \mathbb{R}^d \to \mathbb{R}^k$ and given by

$$\mathrm{Lop}_{(f,\mathbf{x})}(\mathbf{u}) := \mathbf{u}^\top J_f(\mathbf{x}).$$

Proposition 19.2.1. *Right-operations can be constructed using left-operations as follows.*

[1] Throughout, we will assume that f is differentiable at $\mathbf{x}$.

1. *Let* $\{\mathbf{e}_1, \ldots, \mathbf{e}_k\}$ *be the canonical basis vectors of* $\mathbb{R}^k$*, then*

$$\mathrm{Rop}_{(f,\mathbf{x})}(\mathbf{u}) = \begin{pmatrix} \mathrm{Lop}_{(f,\mathbf{x})}(\mathbf{e}_1) \\ \vdots \\ \mathrm{Lop}_{(f,\mathbf{x})}(\mathbf{e}_k) \end{pmatrix} \mathbf{u}.$$

2. $\mathrm{Rop}_{(f,\mathbf{x})}(\mathbf{u}) = \left(\mathrm{Lop}_{\left(\mathrm{Lop}_{(f,\mathbf{x})},\mathbf{v}\right)}(\mathbf{u})\right)^\top$

Proof.

1. Indeed,

$$\begin{aligned} \begin{pmatrix} \mathrm{Lop}_{(f,\mathbf{x})}(\mathbf{e}_1) \\ \vdots \\ \mathrm{Lop}_{(f,\mathbf{x})}(\mathbf{e}_k) \end{pmatrix} \mathbf{u} &= \begin{pmatrix} \mathbf{e}_1^\top J_f(\mathbf{x}) \\ \vdots \\ \mathbf{e}_k^\top J_f(\mathbf{x}) \end{pmatrix} \mathbf{u} \\ &= \begin{pmatrix} \left[J_f(\mathbf{x})\right]_{1,\cdot} \\ \vdots \\ \left[J_f(\mathbf{x})\right]_{k,\cdot} \end{pmatrix} \mathbf{u} \\ &= J_f(\mathbf{x})\mathbf{u}) \\ &= \mathrm{Rop}_{(f,\mathbf{x})}(\mathbf{u}). \end{aligned}$$

2. Since $\mathrm{Lop}_{(f,\mathbf{x})}$ is a linear operator, we just have

$$J_{\mathrm{Lop}_{(f,\mathbf{x})}}(\mathbf{v}) = J_f(\mathbf{x})^\top$$

for any $\mathbf{v} \in \mathbb{R}^d$. Therefore,

$$\begin{aligned} \left(\mathrm{Lop}_{\left(\mathrm{Lop}_{(f,\mathbf{x})},\mathbf{v}\right)}(\mathbf{u})\right)^\top &= \left(\mathbf{u}^\top J_{\mathrm{Lop}_{(f,\mathbf{x})}}(\mathbf{v})\right)^\top \\ &= \left(\mathbf{u}^\top J_f(\mathbf{x})^\top\right)^\top \\ &= J_f(\mathbf{x})\mathbf{u} \\ &= \mathrm{Rop}_{(f,\mathbf{x})}(\mathbf{u}). \end{aligned}$$

□

Remark 19.2.2. *Note how statement 1 of Proposition 19.2.1 requires k Jacobian-vector computations, whereas statement 2 of Proposition 19.2.1 requires only two Jacobian-vector computations. The utility of Proposition 19.2.1 is that left-operations are more optimised on current hardware, due to their similarity to deep network operations. In practice, the optimal approach for performing Jacobian-vector operations depends on the hardware being used and the computational graph of* f*.*

For a deep network that is a continuous piecewise affine operator, it is clear that

$$\mathrm{Rop}_{(f,\mathbf{x})}(\mathbf{u}) = A_{\omega_{\mathbf{t}(\mathbf{x})}}\mathbf{u}.$$

Although this may seem promising as an approach for computing $\mathrm{Rop}_{(f,\mathbf{x})}$, computing $A_{\omega_{\mathbf{t}(\mathbf{x})}}$ suffers similar drawbacks to computing $J_f(\mathbf{x})$ making it largely infeasible.

However, for deep networks implementing ReLU, leaky ReLU or absolute value activation operators we can extract the matrices $Q^{(\ell)}[\mathbf{x}]$ from Proposition 4.6.5 using a single forward pass of the deep network on $\mathbf{x}$ as per statement 2 of Remark 4.6.6. Question: After observing these matrices, what stops us from just using them to compute A_ω and then using that for the Rop calculation? We can then create a clone of the deep network, where the parameter matrices are fixed using the matrices $\mathrm{diag}\left(q^{(\ell)}_{\omega_{\mathbf{s}(\mathbf{x})}}\right)$ as per Proposition 4.6.5 despite the input of the forward pass, we denote this cloned deep network $f_{\mathbf{x}}$. Then observe that

$$\begin{aligned}\mathrm{Rop}_{(f,\mathbf{x})}(\mathbf{u}) &= A_{\omega_{\mathbf{s}(\mathbf{x})}}\mathbf{u}\\ &= A_{\omega_{\mathbf{s}}(\mathbf{x})}\mathbf{x} + b_{\omega_{\mathbf{s}(\mathbf{x})}} - \left(A_{\omega_{\mathbf{s}(\mathbf{x})}}(\mathbf{x}-\mathbf{u}) + b_{\omega_{\mathbf{s}(\mathbf{x})}}\right)\\ &= f(\mathbf{x}) - f_{\mathbf{x}}(\mathbf{x}-\mathbf{u}),\end{aligned}$$

meaning the right-operation on a vector $\mathbf{u}$ at $\mathbf{x}$ can be computed using two forward passes of the deep network.

Proposition 19.2.3. *With the cloning procedure outlined above,*

19.3 *References*

Section 19.2 from Balestriero and Baraniuk (2021).

Bibliography

Alexander Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1998.

Randall Balestriero and Richard Baraniuk. Mad Max: Affine Spline Insights into Deep Learning, November 2018.

Randall Balestriero and Richard Baraniuk. Fast Jacobian-Vector Product for Deep Networks, April 2021.

Randall Balestriero and Richard G. Baraniuk. Batch Normalization Explained, September 2022.

Randall Balestriero and Yann LeCun. Police: Provably Optimal Linear Constraint Enforcement For Deep Neural Networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, June 2023.

Randall Balestriero, Romain Cosentino, Behnaam Aazhang, and Richard Baraniuk. The Geometry of Deep Networks: Power Diagram Subdivision. In *Neural Information Processing Systems*, May 2019.

Randall Balestriero, Sébastien Paris, and Richard Baraniuk. Analytical Probability Distributions and Exact Expectation-Maximization for Deep Generative Networks. In *Neural Information Processing Systems*, 2020.

Randall Balestriero, Romain Cosentino, and Sarath Shekkizhar. Characterizing Large Language Model Geometry Helps Solve Toxicity Detection and Generation. In *International Conference on Machine Learning*, December 2023.

Sid Black, Lee Sharkey, Leo Grinsztajn, Eric Winsor, Dan Braun, Jacob Merizian, Kip Parker, Carlos Ramón Guevara, Beren Millidge, Gabriel Alfour, and Connor Leahy. Interpreting Neural Networks through the Polytope Lens, November 2022.

L. Breiman. Hinging hyperplanes for regression, classification, and function approximation. *IEEE Transactions on Information Theory*, 39 (3), May 1993. ISSN 1557-9654. doi: 10.1109/18.256506.

Romain Cosentino and Sarath Shekkizhar. Reasoning in Large Language Models: A Geometric Perspective, July 2024.

Romain Cosentino, Randall Balestriero, Richard Baranuik, and Behnaam Aazhang. Deep Autoencoders: From Understanding to Generalization Guarantees. In *Proceedings of the 2nd Mathematical and Scientific Machine Learning Conference*, April 2022.

Matteo Gamba, Adrian Chmielewski-Anders, Josephine Sullivan, Hossein Azizpour, and Marten Bjorkman. Are all linear regions created equal? In Gustau Camps-Valls, Francisco J. R. Ruiz, and Isabel Valera, editors, *Proceedings of the 25th Iinternational Conference on Artificial Intelligence and Statistics*, volume 151, pages 6573–6590. PMLR, March 2022.

Ziteng Gao, Limin Wang, and Gangshan Wu. LIP: Local Importance-Based Pooling. *International Journal of Computer Vision*, 131(1), January 2023.

Leyang Hu and Randall Balestriero. Curvature Tuning: Provable Training-free Model Steering From a Single Parameter, February 2025.

Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2024.

Ahmed Imtiaz Humayun, Randall Balestriero, and Richard Baraniuk. MaGNET: Uniform Sampling from Deep Generative Network Manifolds Without Retraining. In *International Conference on Learning Representations*, January 2022a.

Ahmed Imtiaz Humayun, Randall Balestriero, and Richard Baraniuk. Polarity Sampling: Quality and Diversity Control of Pre-Trained Generative Networks via Singular Values. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, June 2022b.

Ahmed Imtiaz Humayun, Randall Balestriero, Guha Balakrishnan, and Richard Baraniuk. SplineCam: Exact Visualization and Characterization of Deep Network Geometry and Decision Boundaries. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, June 2023a.

Ahmed Imtiaz Humayun, Josue Casco-Rodriguez, Randall Balestriero, and Richard Baraniuk. Provable Instance Specific Robustness via Linear Constraints. In *The Second Workshop on New Frontiers in Adversarial Machine Learning*, August 2023b.

Ahmed Imtiaz Humayun, Randall Balestriero, and Richard Baraniuk. Deep Networks Always Grok and Here is Why. In *High-Dimensional Learning Dynamics 2024: The Emergence of Structure and Reasoning*, June 2024.

Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes, December 2022.

Daniel LeJeune, Randall Balestriero, Hamid Javadi, and Richard G. Baraniuk. Implicit Rugosity Regularization via Data Augmentation, October 2019.

Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Forty-First International Conference on Machine Learning*, 2024.

Nasim Rahaman, A. Baratin, Devansh Arpit, Felix Dräxler, Min Lin, F. Hamprecht, Yoshua Bengio, and Aaron C. Courville. On the Spectral Bias of Neural Networks. In *International Conference on Machine Learning*, June 2018.

Omer Ronen, Ahmed Imtiaz Humayun, Randall Balestriero, Richard Baraniuk, and Bin Yu. ScaLES: Scalable Latent Exploration Score for Pre-Trained Generative Networks, June 2024.

Michael Spivak. *Calculus On Manifolds: A Modern Approach To Classical Theorems Of Advanced Calculus*. Avalon Publishing, 1965. ISBN 978-0-8053-9021-6.

Zichao Wang, Randall Balestriero, and Richard Baraniuk. A Max-Affine Spline Perspective of Recurrent Neural Networks. In *International Conference on Learning Representations*, 2019.

Haoran You, Randall Balestriero, Zhihan Lu, Yutong Kou, Huihong Shi, Shunyao Zhang, Shang Wu, Yingyan Lin, and Richard Baraniuk. Max-affine spline insights into deep network pruning. *Transactions on Machine Learning Research*, 2022.